

Otomachi Una C++ 模板库

XCPC 考场代码手册

Junlin Ye

2026-09-23

收录当前模板库的全部 72 份主模板。每节给出调用方式、最小示例和完整代码；模板有依赖时，先抄依赖，再抄本节代码。标题中的路径用于对应源码，实际抄写以本手册代码为准。

考场版改动：SA 使用计数排序倍增，构建 $O(n \log n)$ ；poly-fast、poly-new 和 MTT 使用普通二进制位逆序迭代 NTT，去掉大段预制根表和展开蝶形。其余算法保留原实现。源码目录中的高效版本不受影响。

选择多项式：常规卷积及 $\ln/\exp$ 用 poly-fast；需要开方、带余除法或多点求值选 poly-new；非 NTT 目标模数选 MTT。poly-old 为旧接口，保留备查。各节代码完整，模整数定义也分别保留，使用时只能选其中一份。

编译与调试：GNU C++17。basic/src.cpp 是含头文件和 main 的底稿，其他代码抄在 main 前。LOCAL 控制 Otomachi_Una.in/out，DEBUG 控制 cerr、debug、debugl。不要把全部模板拼成一个程序。

抄写提示：行号不属于代码，排版换行不代表源码换行。长模板会跨页，按页码接续。清空对象使用 set/setN，几何与点分治用 output 调试；FastIO 仅适用于 Linux 的普通文件输入，额外头文件见对应节。

使用顺序：按目录选模板 → 核对下标和依赖 → 抄写代码 → 跑最小示例 → 替换题目逻辑。

目录

1	基础与数值类型	4
1.1	basic/src.cpp	4
1.2	basic/ST table.cpp	6
1.3	basic/ST table (linear).cpp	7
1.4	basic/DSU.cpp	8
1.5	basic/int128_I0.cpp	9
1.6	basic/BigInt.cpp	10
1.7	basic/FastI0.cpp	13
1.8	basic/Coordinate Compression.cpp	16
1.9	basic/ModInt.cpp	18
1.10	basic/ModInt-nonstaticMod.cpp	20
1.11	basic/roll back saver (int).cpp	22
1.12	basic/roll back saver (int or ll).cpp	23
1.13	basic/DSU (with undo).cpp	24
2	数据结构	26
2.1	ds/BIT.cpp	26
2.2	ds/BIT/Sum.cpp	27
2.3	ds/BIT/Max.cpp	28
2.4	ds/BIT/Min.cpp	29
2.5	ds/Segment Tree/AddSum.cpp	30
2.6	ds/Segment Tree/MaxAdd.cpp	32
2.7	ds/Segment Tree/MinAdd.cpp	34
2.8	ds/Segment Tree(with lazytag).cpp	36
2.9	ds/Segment Tree(without lazytag).cpp	40
2.10	ds/zkw.cpp	43
2.11	ds/Dynamic Segment Tree.cpp	45
2.12	ds/Segment Tree Merge.cpp	46
2.13	ds/lichao.cpp	48
2.14	ds/ODT.cpp	50
2.15	ds/FastSet.cpp	51
2.16	ds/FastODT.cpp	54
2.17	ds/FHQ.cpp	55
2.18	ds/pq2stack.cpp	57
2.19	ds/xihe_tree.cpp	59
3	图论	62
3.1	graph/flow.cpp	62
3.2	graph/mcmf.cpp	64
3.3	graph/sssp.cpp	67
3.4	graph/2-sat.cpp	68

3.5	graph/block-cut-tree.cpp	69
3.6	graph/Edge BCC.cpp	71
3.7	graph/blossom.cpp	73
3.8	graph/bipartite-matching.cpp	75
3.9	graph/regular-bipartite.cpp	77
3.10	graph/directed-mst.cpp	79
3.11	graph/chordal.cpp	82
4	树上算法	84
4.1	trees/HLD.cpp	84
4.2	trees/DC.cpp	86
4.3	trees/cartesian.cpp	90
5	数论	91
5.1	number theory/basic.cpp	91
5.2	number theory/linear sieve.cpp	92
5.3	number theory/pollard-rho.cpp	93
5.4	number theory/floor sum.cpp	96
5.5	number theory/du jiao sieve.cpp	96
6	多项式	99
6.1	polynomial/poly-fast.cpp	99
6.2	polynomial/poly-new.cpp	105
6.3	polynomial/poly-old.cpp	114
6.4	polynomial/MTT.cpp	119
6.5	polynomial/FWT.cpp	125
7	字符串	128
7.1	string/sa.cpp	128
7.2	string/sam.cpp	130
7.3	string/ac.cpp	132
7.4	string/gsam.cpp	133
7.5	string/kmp.cpp	135
7.6	string/hash.cpp	136
7.7	string/pam.cpp	138
7.8	string/manacher.cpp	140
7.9	string/runs.cpp	141
7.10	string/lyndon.cpp	143
8	线性代数	144
8.1	math/gauss.cpp	144
8.2	math/linear basis/xor.cpp	145
8.3	math/linear basis/xor-prefix.cpp	146
8.4	math/linear basis/linear.cpp	147

8.5	math/linear basis/linear-prefix.cpp	148
9	计算几何	150
9.1	geometry/geometry-main.cpp	150

1 基础与数值类型

1.1 basic/src.cpp

公共底稿

什么时候用

提供类型别名、调试宏和其他模板依赖的辅助函数。自带标准头文件和 main，可直接作为提交程序的底稿。

调用顺序与接口

- 直接复制本文件，在 main 前粘贴算法，并在其中填写输入输出与求解逻辑。
- debug(...) / debugl(tag,...) 接受 1 至 10 个参数；编译时加 -DDEBUG 才输出，关闭时参数不求值。
- chkmin/chkmax 更新最值并返回是否修改。first_bit/last_bit 返回最低/最高置位的位置，参数必须非零。
- inf<int>、inf<ll> 已特化；其他类型默认值为零，不能当无穷大。

最小示例（放入 main）：

```
1 int a=7; chkmin(a,3);
2 assert(a==3 && first_bit(12)==2 && last_bit(12)==3);
3 debug(a);
4 cerr << "diagnostic\n";
```

使用边界

定义 LOCAL 时从 Otomachi_Una.in 读入并写入 Otomachi_Una.out，打开失败返回 1；否则使用标准输入输出。DEBUG 独立控制调试输出。未定义时通过宏屏蔽裸写的 cerr，不要写 std::cerr。向量输入前必须先分配非空长度；向量输出带方括号并截断长向量，正式答案应自己循环输出。

完整代码

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 using ll=long long;
4 using ull=unsigned long long;
5 // head file
6 #ifndef DEBUG
7     #define cerr for(;;false;) cerr
8 #endif
9 template<typename T>
10 inline constexpr T inf=T{};
11 template<>
12 inline constexpr int inf<int> = 1010000000;
13 template<>
14 inline constexpr long long inf<long long> = 2020000000000000000LL;
15 template<typename T,size_t S>
16 istream& operator >>(istream &o,array<T,S> &I){
17     assert(S>0);
18     for(int i=0;i<S;i++) o>>I[i];
19     return o;
20 }
21 template<typename T,size_t S>
22 ostream& operator <<(ostream &o,const array<T,S> &I){
23     o<<"[";
24     for(int i=0;i<S;i++){
25         if(i==50){o<<"...(total "<<S<<" elements)";break;}
26         o<<I[i];
```

```

27     if(i+1<I.size()) o<<" ";
28 }
29 o<<" ";
30 return o;
31 }
32 template<typename T>
33 istream& operator >>(istream &o,vector<T> &I){
34     assert(!I.empty());
35     for(int i=0;i<I.size();i++) o>>I[i];
36     return o;
37 }
38 template<typename T>
39 ostream& operator <<(ostream &o,const vector<T> &I){
40     o<<"[";
41     for(int i=0;i<I.size();i++){
42         if(i==50){o<<"...(total "<<I.size()<<" elements)";break;}
43         o<<I[i];
44         if(i+1<I.size()) o<<" ";
45     }
46     o<<"]";
47     return o;
48 }
49
50 #define DEBUG_1(x) #x"<<(x)
51 #define DEBUG_2(x,y) DEBUG_1(x)<<" , "<<DEBUG_1(y)
52 #define DEBUG_3(x,y,z) DEBUG_2(x,y)<<" , "<<DEBUG_1(z)
53 #define DEBUG_4(x,y,z,w) DEBUG_3(x,y,z)<<" , "<<DEBUG_1(w)
54 #define DEBUG_5(x,y,z,w,a) DEBUG_4(x,y,z,w)<<" , "<<DEBUG_1(a)
55 #define DEBUG_6(x,y,z,w,a,b) DEBUG_5(x,y,z,w,a)<<" , "<<DEBUG_1(b)
56 #define DEBUG_7(x,y,z,w,a,b,c) DEBUG_6(x,y,z,w,a,b)<<" , "<<DEBUG_1(c)
57 #define DEBUG_8(x,y,z,w,a,b,c,d) DEBUG_7(x,y,z,w,a,b,c)<<" , "<<DEBUG_1(d)
58 #define DEBUG_9(x,y,z,w,a,b,c,d,e) DEBUG_8(x,y,z,w,a,b,c,d)<<" , "<<DEBUG_1(e)
59 #define DEBUG_10(x,y,z,w,a,b,c,d,e,f) DEBUG_9(x,y,z,w,a,b,c,d,e)<<" , "<<DEBUG_1(f)
60 #define GET_MACRO(_1,_2,_3,_4,_5,_6,_7,_8,_9,_10,NAME,...) NAME
61 #ifdef DEBUG
62 #define debug(...) do { \
63     cerr<<"(line "<< __LINE__ <<"): "; \
64     cerr<<GET_MACRO(__VA_ARGS__,DEBUG_10,DEBUG_9,DEBUG_8, \
65         DEBUG_7,DEBUG_6,DEBUG_5,DEBUG_4, \
66         DEBUG_3,DEBUG_2,DEBUG_1)(__VA_ARGS__); \
67     cerr<<endl; \
68 } while(0)
69 #define debugl(s,...) do { \
70     cerr<<"(line "<< __LINE__ <<"): <<s<<" "; \
71     cerr<<GET_MACRO(__VA_ARGS__,DEBUG_10,DEBUG_9,DEBUG_8, \
72         DEBUG_7,DEBUG_6,DEBUG_5,DEBUG_4, \
73         DEBUG_3,DEBUG_2,DEBUG_1)(__VA_ARGS__); \
74     cerr<<endl; \
75 } while(0)
76 #else
77 #define debug(...) ((void)0)
78 #define debugl(...) ((void)0)
79 #endif
80 inline int last_bit(ull x){assert(x);return 63-__builtin_clzll(x);}
81 inline int first_bit(ull x){assert(x);return __builtin_ctzll(x);}

```

```

82 inline int popc(ull x){return __builtin_popcountll(x);}
83 template<typename A,typename B> inline bool chkmax(A &x,const B &y){if(x<y){x=y;return true;}return
    false;}
84 template<typename A,typename B> inline bool chkmin(A &x,const B &y){if(x>y){x=y;return true;}return
    false;}
85 int main(){
86     #ifdef LOCAL
87         if(!freopen("Otomachi_Una.in","r",stdin)) return 1;
88         if(!freopen("Otomachi_Una.out","w",stdout)) return 1;
89     #endif
90     ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
91     return 0;
92 }

```

1.2 basic/ST table.cpp

静态区间最值

什么时候用

数组不修改、需要多次查询闭区间最大值时使用。预处理 $O(n \log n)$ ，查询 $O(1)$ ，空间 $O(n \log n)$ 。

调用顺序与接口

- `ST_table t; t.setN(n)` 分配下标 $1..n$ ，要求 $n \geq 1$ 。
- 填入 `t.a[0][i]` 后调用 `t.build()`。
- `t.ask(l,r)` 返回最大值；要维护最小值，先把 `chk` 改成 `min`。

最小示例（放入 main）：

```

1 ST_table t; t.setN(3);
2 t.a[0][1]=4; t.a[0][2]=1; t.a[0][3]=8;
3 t.build(); assert(t.ask(1,2)==4);

```

使用边界

修改原数组后必须重新 build。文件末尾已有全局对象 `S`，可以直接用它，也可以另建局部对象。

完整代码

```

1 //
2 // template for ST table
3 // version 1.0 (Last Update Jun 23rd, 2026)
4 //
5 // usage:
6 //   S.setN(n); fill S.a[0][1..n]; S.build();
7 //   int mx = S.ask(l, r); // static RMQ, tweak chk() for min
8 //
9 struct ST_table{
10 int n=0;
11 vector<vector<int>> a;
12 void setN(int _n){
13     assert(1<=_n&&_n<INT_MAX);n=_n;
14     a.assign(__lg(n)+1,vector<int>(n+1));
15 }
16 inline int chk(const int &x,const int &y){return max(x,y);} // compare
17 void build(){
18     assert(n>0);

```

```

19   for(int i=1;i<(int)a.size();i++) for(int j=1;j<=n-(1<<i)+1;j++)
20       a[i][j]=chk(a[i-1][j],a[i-1][j+(1<<(i-1))]);
21   }
22   int ask(int l,int r){
23       assert(1<=l&&l<=r&&r<=n);
24       int k=__lg(r-l+1);
25       return chk(a[k][l],a[k][r-(1<<k)+1]);
26   }
27   }S;

```

1.3 basic/ST table (linear).cpp

四毛子 RMQ：线性预处理与空间

什么时候用

静态数组最值。采用块内单调栈位掩码、块间 ST 表；在机器字位运算为 $O(1)$ 的模型下，预处理及空间 $O(n)$ ，查询 $O(1)$ 。

调用顺序与接口

`Linear_RMQ<int> t; t.build(a)` 接受从 0 开始的 vector，内部保存副本。`ask(l,r)` 查询闭区间最小值，`pos(l,r)` 返回最小值下标；相等取最左。`Linear_RMQ<int,true>` 改为最大值，其他接口相同。

最小示例（放入 main）：

```

1   vector<int> a{4,1,1,3};
2   Linear_RMQ<int> mn; mn.build(a);
3   assert(mn.ask(0,3)==1 && mn.pos(0,3)==1);
4   Linear_RMQ<int,true> mx; mx.build(a);
5   assert(mx.ask(1,3)==3);

```

使用边界

块长 $B=\max(1, \text{floor}(\log_2 n))$ ，不是固定常数；块数 $m=\text{ceil}(n/B)$ ，块间表 $O(m \log m)=O(n)$ 。每个元素存一份块前缀的单调栈掩码，一次清除低位和 `ctz` 即可查询块内最左最值。 $n < \text{INT_MAX}$ ，块长最多 30，使用 unsigned 位运算。可 build 空数组，但不能查询；修改数组后须重新 build。`set()` 清空。不需要普通 ST 表或笛卡尔树作为依赖。

完整代码

```

1   //////////////////////////////////////
2   //
3   // template for linear RMQ
4   //
5   // usage:
6   //   Linear_RMQ<int> t; t.build(a); t.ask(l,r); t.pos(l,r);
7   //   0-indexed, inclusive; use Linear_RMQ<int,true> for maximum.
8   //   O(n) build / space, O(1) query on the word RAM; ties choose the leftmost.
9   //
10  //////////////////////////////////////
11  template<typename T,bool MAX=false> struct Linear_RMQ{
12  private:
13      int n=0,B=1;
14      vector<T> a;
15      vector<unsigned> mask;
16      vector<vector<int>>> st;
17      bool better(int x,int y)const{if constexpr(MAX)return a[y]<a[x];else return a[x]<a[y];}
18      int chk(int x,int y)const{return better(y,x)?y:x;}
19      int small(int l,int r)const{

```

```

20 unsigned s=mask[r]&(~0u<<(1%B));
21 return r/B*B+__builtin_ctz(s);
22 }
23 public:
24 void set(){n=0;B=1;a.clear();mask.clear();st.clear();}
25 void build(const vector<T> &v){
26     assert(v.size()<(size_t)INT_MAX);a=v;n=a.size();mask.resize(n);st.clear();
27     if(!n){B=1;return;}
28     // B grows with log(n); the block sparse table has only O(n) entries.
29     B=max(1,31-__builtin_clz((unsigned)n));
30     int m=(n-1)/B+1;st.resize(32-__builtin_clz((unsigned)m));st[0].resize(m);
31     unsigned s=0;
32     for(int i=0;i<n;i++){
33         if(i%B==0)s=0;
34         while(s){
35             int j=31-__builtin_clz(s);
36             if(!better(i,i/B*B+j))break;
37             s^=1u<<j;
38         }
39         mask[i]=s|1u<<(i%B);
40         st[0][i/B]=i/B*B+__builtin_ctz(s);
41     }
42     for(int k=1;k<(int)st.size();k++){
43         st[k].resize(m-(1<<k)+1);
44         for(int i=0;i<(int)st[k].size();i++)st[k][i]=chk(st[k-1][i],st[k-1][i+(1<<(k-1))]);
45     }
46 }
47 int pos(int l,int r)const{
48     assert(0<=l&&l<=r&&r<n);
49     int x=l/B,y=r/B;if(x==y)return small(l,r);
50     int ans=small(l,(x+1)*B-1);
51     if(x+1<y){
52         int k=31-__builtin_clz((unsigned)(y-x-1));
53         ans=chk(ans,chk(st[k][x+1],st[k][y-(1<<k)]));
54     }
55     return chk(ans,small(y*B,r));
56 }
57 T ask(int l,int r)const{return a[pos(l,r)];}
58 };
59 // end for basic/ST table (linear).cpp
60 //////////////////////////////////////

```

1.4 basic/DSU.cpp

普通并查集

什么时候用

维护集合连通关系，路径压缩配合按大小合并。操作均摊近似 $O(1)$ ，空间 $O(n)$ 。

调用顺序与接口

- DSU d; d.setN(n) 创建 1..n 的独立集合。
- find(x) 返回代表元; same(x,y) 判断同集合。
- merge(x,y) 成功返回新代表元，已经连通返回 0。siz[find(x)] 为集合大小。

最小示例（放入 main）：

```

1 DSU d; d.setN(4);
2 d.merge(1,2); assert(d.same(1,2));
3 assert(d.siz[d.find(1)]==2 && !d.same(1,3));

```

使用边界

再次 setN 会清空关系。普通版不能回滚；需要撤销时选择 unionfind。

完整代码

```

1 //
2 // template for Disjoint Set Union (DSU)
3 //
4 // usage:
5 //   DSU dsu; dsu.setN(n); dsu.merge(u, v);
6 //   dsu.same(u, v); int r = dsu.find(u); // merge returns new root or 0
7 //
8 struct DSU{
9   int n=0;
10  vector<int> fa,siz;
11  void setN(int _n){
12    assert(0<=_n&&_n<INT_MAX);n=_n;
13    fa.resize(n+1);iota(fa.begin(),fa.end(),0);siz.assign(n+1,1);siz[0]=0;
14  }
15  inline int find(int x){
16    assert(1<=x&&x<=n);
17    while(x^fa[x])x=fa[x]=fa[fa[x]];return x;
18  }
19  inline int merge(int x,int y){
20    assert(1<=min(x,y)&&max(x,y)<=n);
21    x=find(x),y=find(y);
22    if(x==y) return 0;
23    if(siz[x]<siz[y]) swap(x,y);
24    siz[x]+=siz[y],fa[y]=x;return x;
25  }
26  inline bool same(int x,int y){
27    assert(1<=min(x,y)&&max(x,y)<=n);
28    return find(x)==find(y);
29  }
30 };

```

1.5 basic/int128_I0.cpp**128 位整数输入输出****什么时候用**

为有符号 __int128 提供十进制流输入输出，仍然是定长整数。

调用顺序与接口

- 类型别名 lll 等于 __int128。
- 粘贴后可直接使用 cin >> x、cout << x。

最小示例（放入 main）：

```

1 lll x=1;
2 x*=1000000000000000000LL; x*=100;
3 ostream out; out << x;
4 assert(out.str()=="10000000000000000000");

```

使用边界

需要 GCC/Clang 的 `__int128` 支持；输入和运算结果都必须在有符号 128 位范围内。乘法之前就要提升类型，例如 `(__int128)a*b`。

完整代码

```

1 //
2 // template for __int128 I/O
3 //
4 // usage:
5 //   using lll = __int128; lll x; cin >> x; cout << x;
6 //   GCC/Clang only; decimal I/O
7 //
8 using lll=__int128;
9 istream& operator >>(istream &o, lll &x){
10  string s;if(!(o>>s))return o;
11  bool neg=s[0]=='-';size_t i=(neg||s[0]=='+');
12  assert(i<s.size());__uint128_t v=0,lim=((__uint128_t)1<<127)-!neg;
13  for(;i<s.size();i++){
14    assert('0'<=s[i]&&s[i]<='9');unsigned d=s[i]-'0';
15    assert(v<=(lim-d)/10);v=v*10+d;
16  }
17  x=neg?(lll)(__uint128_t(0)-v):(lll)v;
18  return o;
19 }
20 ostream& operator <<(ostream &o, lll x){
21  array<int,40> a;int n=0;
22  if(x==0){o<<0;return o;}
23  __uint128_t v=x;
24  if(x<0){o<<"-";v=__uint128_t(0)-v;}
25  while(v) a[++n]=v%10,v/=10;
26  for(int i=n;i>=1;i--) o<<a[i];
27  return o;
28 }

```

1.6 basic/BigInt.cpp

高精度有符号整数

什么时候用

十进制大整数，vector 按 10^9 分块。加减 $O(n)$ ，普通乘法及长除法最坏 $O(nm)$ ， n 、 m 是分块数。

调用顺序与接口

- `BigInt x=123` 从 long long 构造；更长整数用流读入。
- 支持 `+` `-` `*` `/` `%`、复合赋值和比较。
- `a.divmod(b)` 同时返回商和余数，避免分别执行除法与取模。

最小示例（放入 main）：

```

1 BigInt a,b;
2 istringstream in("-12345678901234567890 100");
3 in >> a >> b;
4 auto [q,r]=a.divmod(b);
5 assert(q*b+r==a);
6 ostringstream out; out << q << ' ' << r;
7 assert(out.str()=="-123456789012345678 -90");

```

使用边界

除法向零取整，余数与被除数同号；除数不能为零。这不是 FFT 高精度乘法，也不支持小数。

完整代码

```

1 struct BigInt{
2     static constexpr int base=1000000000;
3     vector<int> a;
4     int sign=1;
5     BigInt(long long x=0){
6         unsigned long long y=x;
7         if(x<0)sign=-1,y=0-y;
8         for(;;y/=base)a.push_back(y%base);
9     }
10    void trim(){while(!a.empty()&&!a.back())a.pop_back();if(a.empty())sign=1;}
11    explicit operator bool()const{return !a.empty();}
12    int cmp_abs(const BigInt &b)const{
13        if(a.size()!=b.a.size())return a.size()<b.a.size()?-1:1;
14        for(int i=(int)a.size()-1;i>=0;i--)if(a[i]!=b.a[i])return a[i]<b.a[i]?-1:1;
15        return 0;
16    }
17    int cmp(const BigInt &b)const{return sign!=b.sign?(sign<b.sign?-1:1):sign*cmp_abs(b);}
18    bool operator <(const BigInt &b)const{return cmp(b)<0;}
19    bool operator >(const BigInt &b)const{return cmp(b)>0;}
20    bool operator <=(const BigInt &b)const{return cmp(b)<=0;}
21    bool operator >=(const BigInt &b)const{return cmp(b)>=0;}
22    bool operator ==(const BigInt &b)const{return sign==b.sign&&a==b.a;}
23    bool operator !=(const BigInt &b)const{return !(*this==b);}
24    BigInt operator -(const BigInt &b){BigInt b=*this;if(b)b.sign=-b.sign;return b;}
25    BigInt operator +(const BigInt &b){return *this;}
26    BigInt operator +(const BigInt &b)const{
27        if(!b)return *this;
28        if(!*this)return b;
29        if(sign!=b.sign)return *this-(-b);
30        BigInt c;c.sign=sign;c.a.resize(max(a.size(),b.a.size()));
31        int carry=0;
32        for(int i=0;i<(int)c.a.size();i++){
33            long long x=(long long)carry+(i<(int)a.size()?a[i]:0)+(i<(int)b.a.size()?b.a[i]:0);
34            c.a[i]=x%base;carry=x/base;
35        }
36        if(carry)c.a.push_back(carry);
37        return c;
38    }
39    BigInt operator -(const BigInt &b)const{
40        if(!b)return *this;
41        if(!*this)return -b;
42        if(sign!=b.sign)return *this+(-b);
43        if(cmp_abs(b)<0)return -(b-*this);

```

```

44     BigInt c=*this;int carry=0;
45     for(int i=0;i<(int)a.size();i++){
46         int x=a[i]-carry-(i<(int)b.a.size()?b.a[i]:0);
47         carry=x<0;c.a[i]=x+(carry?base:0);
48     }
49     c.trim();return c;
50 }
51 BigInt mul(int x)const{
52     assert(0<=x&&x<base);
53     BigInt c;c.sign=sign;c.a.resize(a.size());long long carry=0;
54     for(int i=0;i<(int)a.size();i++){
55         long long y=1ll*a[i]*x+carry;c.a[i]=y%base;carry=y/base;
56     }
57     if(carry)c.a.push_back(carry);
58     c.trim();return c;
59 }
60 BigInt operator *(const BigInt &b)const{
61     BigInt c;if(!*this||!b)return c;
62     c.sign=sign*b.sign;c.a.resize(a.size()+b.a.size());
63     for(int i=0;i<(int)a.size();i++){
64         long long carry=0;
65         for(int j=0;j<(int)b.a.size();j++){
66             long long x=c.a[i+j]+1ll*a[i]*b.a[j]+carry;
67             c.a[i+j]=x%base;carry=x/base;
68         }
69         c.a[i+b.a.size()]=carry;
70     }
71     c.trim();return c;
72 }
73 // Division truncates towards zero; remainder has the dividend's sign.
74 pair<BigInt,BigInt> divmod(const BigInt &b)const{
75     assert(b);
76     if(cmp_abs(b)<0)return {0,*this};
77     if(b.a.size()==1){
78         BigInt q=*this;long long rem=0;
79         for(int i=(int)a.size()-1;i>=0;i--){
80             long long x=rem*base+a[i];q.a[i]=x/b.a[0];rem=x%b.a[0];
81         }
82         q.sign=sign*b.sign;q.trim();return {q,BigInt(rem*sign)};
83     }
84     int norm=base/(b.a.back()+1);
85     BigInt x=mul(norm),y=b.mul(norm),q,r;x.sign=y.sign=1;
86     q.a.resize(x.a.size());
87     for(int i=(int)x.a.size()-1;i>=0;i--){
88         r.a.insert(r.a.begin(),x.a[i]);r.trim();
89         int n=y.a.size();
90         long long hi=r.a.size()>(size_t)n?r.a[n]:0;
91         long long lo=r.a.size()>=(size_t)n?r.a[n-1]:0;
92         int d=min((hi*base+lo)/y.a.back(),(long long)base-1);
93         r=r-y.mul(d);
94         while(r.sign<0)r=r+y,--d;
95         q.a[i]=d;
96     }
97     long long carry=0;
98     for(int i=(int)r.a.size()-1;i>=0;i--){

```

```

99     long long x=r.a[i]+carry*base;r.a[i]=x/norm;carry=x%norm;
100 }
101 q.sign=sign*b.sign;r.sign=sign;q.trim();r.trim();return {q,r};
102 }
103 BigInt operator /(const BigInt &b)const{return divmod(b).first;}
104 BigInt operator %(const BigInt &b)const{return divmod(b).second;}
105 BigInt& operator +=(const BigInt &b){return *this=*this+b;}
106 BigInt& operator -=(const BigInt &b){return *this=*this-b;}
107 BigInt& operator *=(const BigInt &b){return *this=*this*b;}
108 BigInt& operator /=(const BigInt &b){return *this=*this/b;}
109 BigInt& operator %=(const BigInt &b){return *this=*this%b;}
110 friend istream& operator >>(istream &in,BigInt &x){
111     string s;if(!(in>>s))return in;
112     int l=(s[0]=='+'||s[0]=='-');
113     if(l==(int)s.size()){in.setstate(ios::failbit);return in;}
114     for(int i=l;i<(int)s.size();i++)if(s[i]<'0'||s[i]>'9'){
115         in.setstate(ios::failbit);return in;
116     }
117     BigInt y;y.sign=s[0]=='?'-1:1;y.a.reserve((s.size()-l+8)/9);
118     for(int r=s.size();r>l;r-=9){
119         int v=0;for(int i=max(l,r-9);i<r;i++)v=v*10+s[i]-'0';
120         y.a.push_back(v);
121     }
122     y.trim();x=move(y);return in;
123 }
124 friend ostream& operator <<(ostream &out,const BigInt &x){
125     if(!x)return out<<'0';
126     string s=x.sign<0?"-":"";s+=to_string(x.a.back());
127     for(int i=(int)x.a.size()-2;i>=0;i--){
128         string t=to_string(x.a[i]);s.append(9-t.size(),'0');s+=t;
129     }
130     return out<<s;
131 }
132 };

```

1.7 basic/FastIO.cpp

普通文件上的 mmap 快读快写

什么时候用

适合 Linux 下大规模整数输入，输入必须是可映射的普通文件。

调用顺序与接口

- 额外包含 `sys/stat.h`、`sys/mman.h`、`unistd.h`。
- 重定向输入输出之后调用 `input.set(stdin)` 和 `output.set(stdout)`。
- 整数使用 `input >> x`、`output << x`；`output.flush()` 主动刷新。

最小示例（放入 main）：

```

1 // Input file: 2 10 20
2 input.set(stdin); output.set(stdout);
3 int n; input >> n; ll sum=0;
4 for(int i=0;i<n;i++){ll x; input >> x; sum+=x;}
5 output << sum << '\n'; output.flush(); // 30

```

使用边界

不适用于交互题、终端或管道输入，也不能原生运行于 Windows。不读浮点和字符串。不要和 cin 混读同一个输入源。本手册此例因当前检查环境为 Windows，未执行 Linux 运行验证。

完整代码

```

1 //
2 // template for FastIO (mmap)
3 // requires sys/stat.h, sys/mman.h, unistd.h
4 //
5 // usage:
6 //   input.set(stdin); output.set(stdout); // after freopen
7 //   input >> x; output << x; // integers only, Linux/WSL
8 //
9 // -----
10 // Fast mmap-based reader / buffered writer (from the __yzlf code)
11 // -----
12 using u8 = unsigned char;
13 using u16 = unsigned short;
14 using u32 = unsigned;
15 using i64 = long long;
16 using u64 = unsigned long long;
17
18 constexpr std::size_t BUF_DEF_SIZE = 262144;
19 constexpr std::size_t BUF_FLUSH_THRESHOLD = 32;
20 constexpr u64 E16 = 10000000000000000ULL;
21 constexpr u64 E12 = 1000000000000000ULL;
22 constexpr u64 E8 = 1000000000ULL;
23 constexpr u64 E4 = 1000000ULL;
24
25 struct _io_t {
26     std::array<u8, 1 << 15> t_i; // two-digit decode table
27     std::array<u32, 10000> t_o; // four ASCII digits packed in u32
28     _io_t() : t_i{}, t_o{} {
29         t_i.fill(u8(-1));
30         for (int i = 0; i < 10; ++i)
31             for (int j = 0; j < 10; ++j)
32                 t_i[0x3030 + 256 * j + i] = j + 10 * i;
33         int idx = 0;
34         for (int e0 = (48 << 0); e0 < (58 << 0); e0 += (1 << 0))
35             for (int e1 = (48 << 8); e1 < (58 << 8); e1 += (1 << 8))
36                 for (int e2 = (48 << 16); e2 < (58 << 16); e2 += (1 << 16))
37                     for (int e3 = (48 << 24); e3 < (58 << 24); e3 += (1 << 24))
38                         t_o[idx++] = e0 ^ e1 ^ e2 ^ e3;
39     }
40     void get(char* s, u32 p) const { memcpy(s, &t_o[p], sizeof(u32)); }
41 };
42 _io_t _iot;
43
44 struct Reader {
45     char* bg=nullptr;
46     char* p=nullptr;
47     char* ed=nullptr;
48     std::size_t sz=0;
49     Reader() = default;
50     Reader(const Reader&)=delete;

```

```

51 Reader& operator=(const Reader&)=delete;
52 void set(FILE* f = stdin) {
53     assert(f);
54     if(bg)munmap(bg,sz);
55     int fd=fileno(f);struct stat st;
56     int res=fstat(fd,&st);assert(res==0&&S_ISREG(st.st_mode));
57     sz=st.st_size;bg=p=ed=nullptr;
58     if(!sz)return;
59     bg=static_cast<char*>(mmap(nullptr,sz,PROT_READ,MAP_PRIVATE,fd,0));
60     assert(bg!=MAP_FAILED);madvice(bg,sz,MADV_SEQUENTIAL);p=bg;ed=bg+sz;
61 }
62
63 ~Reader(){if(bg)munmap(bg,sz);}
64
65 inline void skip_space() { while(p!=ed&&*p<=' ')++p; }
66
67 template <typename T>
68 Reader& operator>>(T& x) {
69     skip_space();assert(p!=ed);bool neg=false;
70     if(*p=='+')++p;
71     else if constexpr(std::is_signed_v<T>){neg=(*p=='-');p+=neg;}
72     assert(p!=ed&&'0'<=*p&&*p<='9');
73     using U=std::make_unsigned_t<T>;U v=*p++-'0';
74     while(ed-p>=2){
75         u16 z;memcpy(&z,p,sizeof(z));
76         u32 y=z<_iot.t_i.size()?_iot.t_i[z]:255;
77         if(y>99)break;
78         v=v*100+y;p+=2;
79     }
80     if(p!=ed&&'0'<=*p&&*p<='9')v=v*10+(*p++-'0');
81     x=neg?T(U(0)-v):T(v);return *this;
82 }
83 };
84
85 struct Writer {
86     std::vector<char> buf;
87     char* bg=nullptr;char* p=nullptr;char* ed=nullptr;FILE* f=nullptr;
88     Writer() = default;
89     Writer(const Writer&)=delete;
90     Writer& operator=(const Writer&)=delete;
91
92     void set(FILE* fi = stdout, std::size_t sz = BUF_DEF_SIZE) {
93         assert(fi&&sz>BUF_FLUSH_THRESHOLD);if(bg)flush();
94         f = fi;buf.resize(sz);
95         bg = buf.data();
96         p = bg;
97         ed = bg + sz - BUF_FLUSH_THRESHOLD;
98     }
99     ~Writer() { if(bg)flush(); }
100     inline void flush() { assert(bg&&f);fwrite(bg, 1, p - bg, f); p = bg; }
101     inline void chk() { if (p > ed) flush(); }
102
103     inline void put_block(u32 x) { _iot.get(p, x); p += 4; }
104
105     inline void put2(u32 x) {

```

```

106     if (x > 9) { _iot.get(p, x * 100); p += 2; }
107     else *p++ = char('0' + x);
108 }
109 inline void put4(u32 x) {
110     if (x > 99) { if (x > 999) put_block(x); else { _iot.get(p, x * 10); p += 3; } }
111     else put2(x);
112 }
113 inline void write_u32(u32 x) {
114     if (x >= E8) { put2(x / E8); x %= E8; put_block(x / E4); put_block(x % E4); }
115     else if (x >= E4) { put4(x / E4); put_block(x % E4); }
116     else put4(x);
117     chk();
118 }
119 inline void write_u64(u64 x) {
120     if (x >= E8) {
121         u64 q0 = x / E8, r0 = x % E8;
122         if (x >= E16) { u64 q1 = q0 / E8, r1 = q0 % E8; put4(q1); put_block(r1 / E4); put_block(r1 % E4); }
123         else if (x >= E12) { put4(q0 / E4); put_block(q0 % E4); }
124         else put4(q0);
125         put_block(r0 / E4); put_block(r0 % E4);
126     } else if (x >= E4) { put4(x / E4); put_block(x % E4); }
127     else put4(static_cast<u32>(x));
128     chk();
129 }
130
131 Writer& operator<<(u32 x) { write_u32(x); return *this; }
132 Writer& operator<<(u64 x) { write_u64(x); return *this; }
133 Writer& operator<<(int x) { if (x < 0) { *p++ = '-'; write_u32(-static_cast<u32>(x)); } else write_u32
    (static_cast<u32>(x)); return *this; }
134 Writer& operator<<(long long x) { if (x < 0) { *p++ = '-'; write_u64(-static_cast<u64>(x)); } else
    write_u64(static_cast<u64>(x)); return *this; }
135 Writer& operator<<(char c) { *p++ = c; chk(); return *this; }
136 };
137 inline Reader input;
138 inline Writer output;

```

1.8 basic/Coordinate Compression.cpp

离散化并原地回写

什么时候用

先收集所有会用到的值，再排序去重，把已登记变量改写成 1..k 的排名。时间 $O(n \log n)$ 。

调用顺序与接口

- `compress<T> c; c.set()` 清空。`c.pb(x)` 要求可修改左值，会保存 `x` 的地址。
- `c.build()` 原地改写 `x`；`c.id(original)` 找已登记原值的排名。
- `c[i]` 由排名恢复原值；`range(l,r)` 返回落在原值闭区间中的排名范围，左端大于右端表示空。

最小示例（放入 `main`）：

```

1 compress<int> c; c.set();
2 int a=30,b=10,d=30;
3 c.pb(a); c.pb(b); c.pb(d); c.build();

```

```

4 assert(a==2 && b==1 && d==2 && c[2]==30);
5 assert(c.id(30)==2);

```

使用边界

pb 后、build 前变量必须存活且地址稳定：不要登记即将离开作用域的局部变量，也不要让所属 vector 扩容。id 参数是原值，不是已改写后的排名。range 当前端点参数为 int。

完整代码

```

1 //
2 // template for Coordinate Compression
3 // version 2.0 (Last Update Jul 3rd, 2026)
4 //
5 // usage:
6 //   compress<int> cc; cc.set(); cc.pb(x); cc.build();
7 //   cc.id(x); cc[i]; cc.range(l,r); // call set() each problem
8 //
9 template<typename T>
10 struct compress{
11     vector<T> I;
12     vector<T*> II;
13     // make compress values to 1 ~ size
14     void set(){I.clear();II.clear();}
15     void pb(T &x){I.push_back(x);II.push_back(&x);}
16     void pb(const T &x){I.push_back(x);}
17     int id(T x)const{
18         auto it=lower_bound(I.begin(),I.end(),x);
19         assert(it!=I.end()&&*it==x);
20         return it-I.begin()+1;
21     }
22     void build(){
23         sort(I.begin(),I.end());
24         I.erase(unique(I.begin(),I.end()),I.end());
25         vector<int> a; a.reserve(II.size());
26         for(auto p:II)a.push_back(id(*p));
27         for(int i=0;i<(int)II.size();i++)*II[i]=a[i];
28         II.clear();
29     }
30     int size()const{return I.size();}
31     array<int,2> range(const T &l,const T &r)const{
32         int a=lower_bound(I.begin(),I.end(),l)-I.begin();
33         int b=upper_bound(I.begin(),I.end(),r)-I.begin();
34         return {a+1,b};
35     }
36     T operator [](int x)const{
37         assert(1<=x&&x<=I.size());
38         return I[x-1];
39     }
40 };
41 // !!!!!!!!!!! before each use, use cc.set() to clear !!!!!!!!!!!
42 // !!!!!!!!!!! 如果元素被插入, cc.build() 会自动重编号 !!!!!!!!!!!

```

1.9 basic/ModInt.cpp

固定模数及组合数

什么时候用

默认模数 998244353。模数写在源码中的 MOD；阶乘表按需扩容。

调用顺序与接口

- mint 支持四则、pow(k)、inv()；除数必须可逆。
- init(n) 保证 fac、ifac、inv 覆盖到 n，组合数调用会自动预处理。
- C(k,n) 和 binom(n,k) 都表示从 n 个中选 k 个；这里 C 的参数顺序与常见记法相反。

最小示例（放入 main）：

```
1 init(10);
2 assert(C(2,5).x==10 && binom(5,2).x==10);
3 mint a=3; assert((a*a.inv()).x==1);
4 assert(a.pow(4).x==81);
```

使用边界

阶乘方案用于质数模数且 $n < \text{MOD}$ ；比较时可读 .x，此版未定义 mint 的 ==。输入运算符直接读 .x，读负数或未归一化数据时先读 long long 再构造 mint。避免 pow(LLONG_MIN)。多项式文件已内置本类，不要重复粘贴。

完整代码

```
1 template <unsigned _M> struct ModInt{
2     static constexpr unsigned MOD=_M;
3     static_assert(1<MOD&&MOD<=INT_MAX);
4     unsigned x;
5     constexpr ModInt():x(0){}
6     constexpr ModInt(unsigned y):x(y%MOD){}
7     constexpr ModInt(int y):x((y%=static_cast<int>(MOD))<0?y+MOD:y){}
8     constexpr ModInt(unsigned long long y):x(y%MOD){}
9     constexpr ModInt(long long y):x((y%=static_cast<long long>(MOD))<0?y+MOD:y){}
10    ModInt& operator +=(const ModInt &a){x=((x+=a.x)>=MOD)?x-MOD:x;return *this;}
11    ModInt& operator -=(const ModInt &a){x=((x-=a.x)>=MOD)?x+MOD:x;return *this;}
12    ModInt& operator *=(const ModInt &a){x=static_cast<unsigned long long>(x)*a.x%MOD;return *this;}
13    ModInt& operator /=(const ModInt &a){return (*this)*=a.inv();}
14    bool operator ==(const ModInt &a)const{return x==a.x;}
15    bool operator !=(const ModInt &a)const{return x!=a.x;}
16    explicit operator bool() const { return x != 0; }
17    bool operator!() const { return x == 0; }
18    ModInt inv() const {
19        unsigned a=MOD,b=x;ll y=0,z=1;
20        while(b){
21            const unsigned q=a/b,c=a-q*b;
22            a=b,b=c;
23            const ll w=y-(ll)q*z;
24            y=z,z=w;
25        }
26        assert(a==1);return ModInt(y);
27    }
28    ModInt pow(long long k) const {
29        unsigned long long e=k;
30        ModInt a=*this,b=1;
31        if(k<0){a=a.inv();e=0-e;}
```

```

32   for(;e>=1){if(e&1)b*=a;a*=a;}
33   return b;
34 }
35 ModInt& operator++() { *this+=1;return *this;}
36 ModInt& operator--() { *this-=1;return *this;}
37 ModInt operator--(int){ModInt t=*this;--*this;return t;}
38 ModInt operator++(int){ModInt t=*this;++*this;return t;}
39 ModInt operator +()const{return *this;}
40 ModInt operator -()const{ModInt a;a.x=(x?MOD-x:0u);return a;}
41 ModInt operator +(const ModInt &a)const{return ModInt(*this)+=a;}
42 ModInt operator -(const ModInt &a)const{return ModInt(*this)-=a;}
43 ModInt operator *(const ModInt &a)const{return ModInt(*this)*=a;}
44 ModInt operator /(const ModInt &a)const{return ModInt(*this)/=a;}
45
46 template<typename T> ModInt friend operator +(T a,const ModInt &b){return ModInt(a)+=b;}
47 template<typename T> ModInt friend operator -(T a,const ModInt &b){return ModInt(a)-=b;}
48 template<typename T> ModInt friend operator *(T a,const ModInt &b){return ModInt(a)*=b;}
49 template<typename T> ModInt friend operator /(T a,const ModInt &b){return ModInt(a)/=b;}
50 friend istream& operator >> (istream &i,ModInt &x){ll y;if(i>>y)x=ModInt(y);return i;}
51 friend ostream& operator << (ostream &o,const ModInt &x){o<<x.x;return o;}
52 };
53 // Basic function, fac, ifac, binom
54 // init(n) prepares fac / ifac / inv through n
55 const int MOD=998244353;
56 using mint=ModInt<MOD>;
57 vector<mint> fac{1},ifac{1},inv{0,1};
58 void init(int n=0){
59     assert(0<=n&&n<MOD);
60     int m=fac.size();if(n<m)return;
61     n=max(n,(int)min(2ll*m,(ll)MOD-1));
62     fac.resize(n+1);ifac.resize(n+1);inv.resize(max(n+1,2));
63     for(int i=m;i<=n;i++){
64         fac[i]=fac[i-1]*i;
65         if(i>1)inv[i]=-(MOD/i)*inv[MOD%i];
66         ifac[i]=ifac[i-1]*inv[i];
67     }
68 }
69 inline mint C(int x,int y){
70     // choose x from y
71     if(x<0||y<x)return 0;
72     init(y);return fac[y]*ifac[x]*ifac[y-x];
73 }
74 inline mint binom(int y,int x){return C(x,y);}
75
76 inline mint Apple_in_Box(int n,int m){
77     // n apples into m boxes, each box must be non-empty
78     return C(m-1,n-1);
79 }
80 inline mint Apple_in_Box(int n,int m,int k){
81     // n apples into m boxes,
82     // the first k boxes must be non-empty (others can be empty)
83     assert(0<=k&&k<=m);
84     return Apple_in_Box(n+(m-k),m);
85 }

```

1.10 basic/ModInt-nonstaticMod.cpp

运行时模数及组合数

什么时候用

同一份程序需要选定不同模数时使用。设置模数后再创建该轮使用的值。

调用顺序与接口

- `mint::setM(p)` 修改模数。
- `init_fact(n)` 重建 $0..n$ 阶乘、逆阶乘，要求 $n!$ 可逆。
- `C(k,n) / binom(n,k)` 查询前必须已预处理到 n 。

最小示例（放入 main）：

```
1 mint::setM(101); init_fact(10);
2 assert(C(2,5).x==10);
3 mint a=100; assert((a+2).x==1);
```

使用边界

修改模数不会转换已有 `mint` 变量；丢弃旧值并重建阶乘。与固定模数版、各多项式版本的 `mint / fac` 等全局名字冲突。

完整代码

```
1 //
2 // template for ModInt (Mod isn't assign)
3 // version 1.0 (Last Update Jun 16th, 2026)
4 //
5 // usage:
6 //   mint::setM(mod); init_fact(n); mint x; cin >> x;
7 //
8 struct mint {
9     static unsigned M;
10    static unsigned long long NEG_INV_M;
11    static void setM(unsigned m) {
12        assert(1<m&&m<=INT_MAX);M = m;
13        NEG_INV_M = -1ULL / M;
14    }
15    unsigned x;
16    mint() : x(0U) {}
17    mint(unsigned x_) : x(x_ % M) {}
18    mint(unsigned long long x_) : x(x_ % M) {}
19    mint(int x_) : x(((x_ %= static_cast<int>(M)) < 0) ? (x_ + static_cast<int>(M)) : x_) {}
20    mint(long long x_) : x(((x_ %= static_cast<long long>(M)) < 0) ? (x_ + static_cast<long long>(M)) : x_
        ) {}
21    mint& operator+=(const mint &a) {
22        x = ((x += a.x) >= M) ? (x - M) : x;
23        return *this;
24    }
25    mint& operator-=(const mint &a) {
26        x = ((x -= a.x) >= M) ? (x + M) : x;
27        return *this;
28    }
29    mint& operator*=(const mint &a) {
30        const unsigned long long y = static_cast<unsigned long long>(x) * a.x;
31        const unsigned long long q = static_cast<unsigned long long>(
```

```

32     (static_cast<unsigned __int128>(NEG_INV_M) * y) >> 64
33 );
34 const unsigned long long r = y - M * q;
35 x = r - M * (r >= M);
36 return *this;
37 }
38 mint& operator/=(const mint &a) { return (*this *= a.inv()); }
39 mint pow(long long e) const {
40     unsigned long long k=e;
41     mint a=*this,b=1;
42     if(e<0){a=a.inv();k=0-k;}
43     for(;k>=1){if(k&1)b*=a;a*=a;}
44     return b;
45 }
46 mint inv() const {
47     unsigned a = M, b = x;
48     ll y = 0, z = 1;
49     for (; b; ) {
50         const unsigned q = a / b;
51         const unsigned c = a - q * b;
52         a = b; b = c;
53         const ll w = y - (ll)q * z;
54         y = z; z = w;
55     }
56     assert(a == 1U);
57     return mint(y);
58 }
59 mint operator+() const { return *this; }
60 mint operator-() const { mint a; a.x = x ? (M - x) : 0U; return a; }
61 mint operator+(const mint &a) const { return (mint(*this) += a); }
62 mint operator-(const mint &a) const { return (mint(*this) -= a); }
63 mint operator*(const mint &a) const { return (mint(*this) *= a); }
64 mint operator/(const mint &a) const { return (mint(*this) /= a); }
65 mint& operator++() { *this += 1; return *this; }
66 mint& operator--() { *this -= 1; return *this; }
67 bool operator!() const { return x == 0; }
68 explicit operator bool() const { return x != 0; }
69 bool operator==(const mint &a) const { return (x == a.x); }
70 bool operator!=(const mint &a) const { return (x != a.x); }
71 template<class T> friend mint operator+(T a, const mint &b) { return (mint(a) += b); }
72 template<class T> friend mint operator-(T a, const mint &b) { return (mint(a) -= b); }
73 template<class T> friend mint operator*(T a, const mint &b) { return (mint(a) *= b); }
74 template<class T> friend mint operator/(T a, const mint &b) { return (mint(a) /= b); }
75 friend istream& operator>>(istream &i, mint &a) { ll x;if(i>>x)a=mint(x);return i; }
76 friend ostream& operator<<(ostream &o, const mint &a) { return o << a.x; }
77 };
78 unsigned mint::M = 998244353;
79 unsigned long long mint::NEG_INV_M = -1ULL / mint::M;
80
81 // !!!!!!!!!!!!!Use mint::setM(<new mod>)!!!!!!!!!!!!
82
83 vector<mint> fac,ifac,inv;
84 unsigned fact_mod=0;
85 void init_fact(int n=0){
86     assert(0<=n&&(unsigned)n<mint::M);

```

```

87 fac.assign(n+1,1);ifac.resize(n+1);inv.assign(n+1,0);
88 for(int i=1;i<=n;i++)fac[i]=fac[i-1]*i;
89 ifac[n]=fac[n].inv();
90 for(int i=n;i>=1;i--)ifac[i-1]=ifac[i]*i;
91 for(int i=1;i<=n;i++)inv[i]=ifac[i]*fac[i-1];
92 fact_mod=mint::M;
93 }
94 inline mint C(int x,int y){
95     if(x<0||y<x)return 0;
96     assert(fact_mod==mint::M&&y<(int)fac.size());
97     return fac[y]*ifac[x]*ifac[y-x];
98 }
99 inline mint binom(int y,int x){return C(x,y);}

```

1.11 basic/roll back saver (int).cpp

记录并撤销变量修改

什么时候用

保存变量的地址和旧值，用版本号撤销一段修改。每次保存和撤销一次记录都是 $O(1)$ 。

调用顺序与接口

- 修改前先 `saver.save(x)`。
- `saver.version()` 返回当前记录数，`saver.roll_back(t)` 逆序恢复到该记录数。
- `saver.set()` 仅丢弃历史，不恢复变量。

最小示例（放入 `main`）：

```

1 saver.set(); int x=7;
2 auto t=saver.version(); saver.save(x); x=20;
3 saver.roll_back(t); assert(x==7);

```

使用边界

保存期间变量地址必须保持有效，不能让包含它的 `vector` 扩容。每个版本号只能用于当前历史；两种 `saver` 不能同时粘贴。

完整代码

```

1 //
2 // template for roll back saver
3 //
4 // usage:
5 //     saver.set(); saver.save(x); int t=saver.version(); saver.roll_back(t);
6 //     int only; pair with DSU (with undo)
7 //     saved elements must stay at the same address until rollback
8 //
9 struct rollback_saver{
10     vector<pair<int*,int>> buf;
11     void set(){buf.clear();}
12     void save(int &x){buf.push_back(make_pair(&x,x));}
13     int version(){return buf.size();}
14     void roll_back(int t){
15         assert(0<=t&&t<=version());
16         while(buf.size()>t){

```

```

17     auto [a,b]=buf.back();buf.pop_back();
18     *a=b;
19 }
20 }
21 };
22 rollback_saver saver;

```

1.12 basic/roll back saver (int or ll).cpp

记录并撤销变量修改

什么时候用

保存变量的地址和旧值，用版本号撤销一段修改。每次保存和撤销一次记录都是 $O(1)$ 。

调用顺序与接口

- 修改前先 `saver.save(x)`。
- `saver.version()` 返回当前记录数，`saver.roll_back(t)` 逆序恢复到该记录数。
- `saver.set()` 仅丢弃历史，不恢复变量。

最小示例（放入 `main`）：

```

1 saver.set(); ll x=7;
2 auto t=saver.version(); saver.save(x); x=20;
3 saver.roll_back(t); assert(x==7);

```

使用边界

保存期间变量地址必须保持有效，不能让包含它的 `vector` 扩容。每个版本号只能用于当前历史；两种 `saver` 不能同时粘贴。

完整代码

```

1 //
2 // template for roll back saver
3 //
4 // usage:
5 //     saver.set(); saver.save(x); saver.save(y); saver.roll_back(t);
6 //     supports int and ll; do not paste with (int) saver
7 //     saved elements must stay at the same address until rollback
8 //
9 using ll=long long;
10 struct rollback_saver{
11     vector<pair<int*,int>> buf;
12     vector<pair<ll*,ll>> buf1;
13     int saved_ele=0;
14     vector<char> R=vector<char>(1);
15     void set(){saved_ele=0;buf.clear();buf1.clear();R.assign(1,false);}
16     void save(int &x){buf.push_back(make_pair(&x,x));R.push_back(false);++saved_ele;}
17     void save(ll &x){buf1.push_back(make_pair(&x,x));R.push_back(true);++saved_ele;}
18     int version(){return saved_ele;}
19     void roll_back(int t){
20         assert(0<=t&&t<=version());
21         while(saved_ele>t){
22             if(R[saved_ele]){
23                 auto [a,b]=buf1.back();buf1.pop_back();

```

```

24     *a=b;
25 }else{
26     auto [a,b]=buf.back();buf.pop_back();
27     *a=b;
28 }
29 saved_ele--;R.pop_back();
30 }
31 }
32 };
33 rollback_saver saver;

```

1.13 basic/DSU (with undo).cpp

可撤销并查集

什么时候用

适合线段树分治或搜索中加入连边后回退。按大小合并、不做路径压缩，单次查找 $O(\log n)$ 。

调用顺序与接口

- 先粘贴一种 saver，再粘贴本文件。
- `unionfind d; d.setN(n)` 初始化；`find/merge` 与普通版类似。
- 通过全局 `saver.version/saver.roll_back` 撤销 `merge`，而不是调用对象成员。

额外依赖（按此顺序粘贴）：`basic/roll back saver (int).cpp`。

最小示例（放入 main）：

```

1 saver.set(); unionfind d; d.setN(3);
2 auto t=saver.version(); d.merge(1,2);
3 assert(d.find(1)==d.find(2));
4 saver.roll_back(t); assert(d.find(1)!=d.find(2));

```

使用边界

`setN` 前先清空旧历史，不能在旧历史仍引用数组时重新分配。不要自行加入路径压缩。

完整代码

```

1 //
2 // template for unionfind (accept undo, O(nlogn))
3 // saver.version 1.0 (Last Update Jun 28th, 2026)
4 //
5 // usage:
6 //   Paste roll back saver (int).cpp first.
7 //   unionfind uf; uf.setN(n); int t=saver.version(); uf.merge(u,v); saver.roll_back(t);
8 //
9 struct unionfind{
10 int n=0;
11 vector<int> fa,siz;
12 inline void setN(int _n){
13     // saved addresses must not survive setN()
14     assert(saver.version()==0);
15     assert(0<=_n&&_n<INT_MAX);n=_n;
16     fa.resize(n+1);iota(fa.begin(),fa.end(),0);siz.assign(n+1,1);siz[0]=0;
17 }
18 inline int find(int x){assert(1<=x&&x<=n);while(x^fa[x])x=fa[x];return x;}

```

```
19 inline int merge(int x,int y){
20     assert(1<=min(x,y)&&max(x,y)<=n);
21     x=find(x),y=find(y);
22     if(x==y) return 0;
23     if(siz[x]<siz[y]) swap(x,y);
24     saver.save(siz[x]);saver.save(fa[y]);
25     siz[x]+=siz[y];fa[y]=x;
26     return x;
27 }
28 };
```

2 数据结构

2.1 ds/BIT.cpp

自定义下标的树状数组

什么时候用

同时提供单点加的 BIT 和区间加的双 BIT。初始化 $O(n)$ ，单次操作 $O(\log n)$ ，空间 $O(n)$ 。

调用顺序与接口

两种结构都用 `setRange(L,R)` 设置闭区间，`setN(n)` 等价于 `setRange(1,n)`。BIT 的 `add(i,v)` 单点加，`ask(r)` 返回 $[L,r]$ 的和，`ask(l,r)` 返回区间和。rars 的 `add(l,r,v)` 区间加，`ask(l,r)` 查区间和。

最小示例（放入 main）：

```
1 BIT b; b.setRange(-2,2); b.add(0,7);
2 assert(b.ask(0)==7 && b.ask(-1,1)==7);
3 rars t; t.setRange(-2,2); t.add(-1,1,3);
4 assert(t.ask(-2,0)==6);
5 b.setN(3); assert(b.ask(3)==0);
```

使用边界

下标 0 或负数只要在设定区间内就合法；内部会转成正下标。`ask(L-1)` 返回零，`add(R+1,v)` 是差分哨兵，不计入查询。极端坐标的 $L-1/R+1$ 用 ll 计算。rars 的加权差分使用相对位置，不依赖坐标绝对值；值、加权积与答案须在 ll 范围内。`set()` 清空；重新 `setRange/setN` 丢弃旧值。文件末尾保留全局 rars 对象 T。

完整代码

```
1 //////////////////////////////////////
2 //
3 // template for BIT
4 //
5 // usage:
6 //   BIT: setRange(l,r); add(i,v); ask(r); ask(l,r).
7 //   setN(n) = setRange(1,n); add(vr+1,v) is a difference sentinel.
8 //   rars: setRange(l,r); add(l,r,v); ask(l,r).
9 //
10 //////////////////////////////////////
11 struct BIT{
12 private:
13     struct info{
14         ll x=0;
15         info(){}
16         info(ll _x):x(_x){}
17         info& operator +=(const info &y){x+=y.x;return *this;}
18         info operator +(const info &y)const{return info(*this)+=y;}
19     };
20     int vl=1,vr=0,n=0;vector<info> a;
21     info query(int x)const{
22         info ans;for(;x;--x)ans+=a[x];return ans;
23     }
24 public:
25     void set(){vl=1,vr=n=0;a.clear();}
26     void setRange(int l,int r){
27         ll m=(ll)r-l+1;assert(0<=m&&m<INT_MAX-1);
28         vl=l;vr=r;n=m;a.assign(n+2,info());
29     }
```

```

30 void setN(int n){setRange(1,n);}
31 void add(ll x,ll v){
32     assert(!a.empty()&&v1<=x&&x<=(ll)vr+1);
33     for(ll i=x-v1+1;i<=n+1;i+=i&-i)a[i]+=info(v);
34 }
35 ll ask(ll x)const{assert((ll)v1-1<=x&&x<=vr);return query(x-v1+1).x;}
36 ll ask(int l,int r)const{assert(v1<=l&&l<=r&&r<=vr);return ask(r)-ask((ll)l-1);}
37 };
38 struct rars{
39     int v1=1,vr=0;BIT T0,T1;
40     void set(){v1=1;vr=0;T0.set();T1.set();}
41     void setRange(int l,int r){v1=l;vr=r;T0.setRange(l,r);T1.setRange(l,r);}
42     void setN(int n){setRange(1,n);}
43     void add(int l,int r,ll x){
44         assert(v1<=l&&l<=r&&r<=vr);
45         ll a=(ll)l-v1+1,b=(ll)r-v1+2;
46         T0.add(l,x);T0.add((ll)r+1,-x);
47         T1.add(l,a*x);T1.add((ll)r+1,-b*x);
48     }
49     ll query(ll x)const{return (x-v1+2)*T0.ask(x)-T1.ask(x);}
50     ll ask(int l,int r)const{assert(v1<=l&&l<=r&&r<=vr);return query(r)-query((ll)l-1);}
51 }T;
52 // end for ds/BIT.cpp
53 //////////////////////////////////////////////////

```

2.2 ds/BIT/Sum.cpp

树状数组：和特化

什么时候用

初始化 $O(n)$ ，单点修改和前缀查询 $O(\log n)$ ，空间 $O(n)$ 。

调用顺序与接口

Sum_BIT t; t.setRange(L,R); setN(n) 调用 setRange(1,n)。add(i,v) 单点加; ask(r) 查询 $[L,r]$ 的和; ask(l,r) 查询区间和。

最小示例（放入 main）：

```

1 Sum_BIT t; t.setRange(-2,2); t.add(0,5); t.add(1,-3);
2 assert(t.ask(0)==5 && t.ask(0,1)==2);

```

使用边界

初始值全零；允许 add(R+1,v) 差分哨兵及 ask(L-1) 空前缀。支持负坐标，查询不会修改树。三个类名不同，可组合粘贴；set() 清空。

完整代码

```

1 //////////////////////////////////////////////////
2 //
3 // template for BIT (sum)
4 //
5 // usage:
6 //     Sum_BIT t; t.setRange(l,r); // setN(n) = setRange(1,n)
7 //     add(i,v); ask(r); ask(l,r); initial values are zero.
8 //     add(vr+1,v) is allowed as a difference sentinel.
9 //

```

```

10 //////////////////////////////////////////////////
11 struct Sum_BIT{
12 private:
13 int vl=1,vr=0,n=0;vector<ll> a;
14 public:
15 void set(){vl=1;vr=n=0;a.clear();}
16 void setRange(int l,int r){
17 ll m=(ll)r-l+1;assert(0<=m&&m<INT_MAX-1);
18 vl=l;vr=r;n=m;a.assign(n+2,0);
19 }
20 void setN(int n){setRange(1,n);}
21 void add(ll x,ll v){
22 assert(!a.empty()&&vl<=x&&x<=(ll)vr+1);
23 for(ll i=x-vl+1;i<=n+1;i+=i&-i)a[i]+=v;
24 }
25 ll ask(ll x)const{
26 assert((ll)vl-1<=x&&x<=vr);ll ans=0;
27 for(int i=x-vl+1;i; i-=i&-i)ans+=a[i];
28 return ans;
29 }
30 ll ask(int l,int r)const{assert(vl<=l&&l<=r&&r<=vr);return ask(r)-ask((ll)l-1);}
31 };
32 // end for ds/BIT/Sum.cpp
33 //////////////////////////////////

```

2.3 ds/BIT/Max.cpp

树状数组：最大值特化

什么时候用

初始化 $O(n)$ ，单点修改和前缀查询 $O(\log n)$ ，空间 $O(n)$ 。

调用顺序与接口

Max_BIT t; t.setRange(L,R); setN(n) 调用 setRange(1,n)。upd(i,v) 把第 i 项取最大值，并非赋值；ask(r) 查询 [L,r] 的最大值。

最小示例（放入 main）：

```

1 Max_BIT t; t.setRange(-2,2); t.upd(0,5); t.upd(0,3);
2 assert(t.ask(1)==5);

```

使用边界

初值和空前缀返回 LLONG_MIN。只支持单点 chmax；不能把已有大值减小。没有 ask(l,r)，最值不能像和一样用两个前缀相减；任意点赋值或一般区间最值请用线段树。三个类名不同，可组合粘贴；set() 清空。

完整代码

```

1 //////////////////////////////////////////////////
2 //
3 // template for BIT (max)
4 //
5 // usage:
6 //   Max_BIT t; t.setRange(l,r); // setN(n) = setRange(1,n)
7 //   upd(i,v) is chmax, not assignment; ask(r) queries [vl,r].
8 //   Initial / empty-prefix value: LLONG_MIN. No inverse for interval queries.
9 //

```

```

10 //////////////////////////////////////////////////
11 struct Max_BIT{
12 private:
13 int vl=1,vr=0,n=0;vector<ll> a;
14 public:
15 void set(){vl=1;vr=n=0;a.clear();}
16 void setRange(int l,int r){
17     ll m=(ll)r-l+1;assert(0<=m&&m<INT_MAX-1);
18     vl=l;vr=r;n=m;a.assign(n+2,LLONG_MIN);
19 }
20 void setN(int n){setRange(1,n);}
21 void upd(ll x,ll v){
22     assert(vl<=x&&x<=vr);
23     for(ll i=x-vl+1;i<=n;i+=i&-i)a[i]=max(a[i],v);
24 }
25 ll ask(ll x)const{
26     assert((ll)vl-1<=x&&x<=vr);ll ans=LLONG_MIN;
27     for(int i=x-vl+1;i; i-=i&-i)ans=max(ans,a[i]);
28     return ans;
29 }
30
31 };
32 // end for ds/BIT/Max.cpp
33 //////////////////////////////////

```

2.4 ds/BIT/Min.cpp

树状数组：最小值特化

什么时候用

初始化 $O(n)$ ，单点修改和前缀查询 $O(\log n)$ ，空间 $O(n)$ 。

调用顺序与接口

Min_BIT t; t.setRange(L,R); setN(n) 调用 setRange(1,n)。upd(i,v) 把第 i 项取最小值，并非赋值；ask(r) 查询 $[L,r]$ 的最小值。

最小示例（放入 main）：

```

1 Min_BIT t; t.setRange(-2,2); t.upd(0,5); t.upd(0,3);
2 assert(t.ask(1)==3);

```

使用边界

初值和空前缀返回 LLONG_MAX。只支持单点 chmin；不能把已有小值增大。没有 ask(l,r)，最值不能像和一样用两个前缀相减；任意点赋值或一般区间最值请用线段树。三个类名不同，可组合粘贴；set() 清空。

完整代码

```

1 //////////////////////////////////////////////////
2 //
3 // template for BIT (min)
4 //
5 // usage:
6 //     Min_BIT t; t.setRange(l,r); // setN(n) = setRange(1,n)
7 //     upd(i,v) is chmin, not assignment; ask(r) queries [vl,r].
8 //     Initial / empty-prefix value: LLONG_MAX. No inverse for interval queries.
9 //

```

```

10 //////////////////////////////////////////////////
11 struct Min_BIT{
12 private:
13 int vl=1,vr=0,n=0;vector<ll> a;
14 public:
15 void set(){vl=1;vr=n=0;a.clear();}
16 void setRange(int l,int r){
17 ll m=(ll)r-l+1;assert(0<=m&&m<INT_MAX-1);
18 vl=l;vr=r;n=m;a.assign(n+2,LLONG_MAX);
19 }
20 void setN(int n){setRange(1,n);}
21 void upd(ll x,ll v){
22 assert(vl<=x&&x<=vr);
23 for(ll i=x-vl+1;i<=n;i+=i&-i)a[i]=min(a[i],v);
24 }
25 ll ask(ll x)const{
26 assert((ll)vl-1<=x&&x<=vr);ll ans=LLONG_MAX;
27 for(int i=x-vl+1;i; i-=i&-i)ans=min(ans,a[i]);
28 return ans;
29 }
30
31 };
32 // end for ds/BIT/Min.cpp
33 //////////////////////////////////

```

2.5 ds/Segment Tree/AddSum.cpp

区间加、区间和

什么时候用

默认闭区间下标 1..n、初值全零，初始化 $O(n)$ ，单次修改查询 $O(\log n)$ 。

调用顺序与接口

- `setN(n)` 或 `set(l,r)` 指定有效闭区间。
- `upd(i,v)` 是赋值，`add(i,v)` 是加法；`add(l,r,v)` 区间加。
- `ask(l,r)` 查询，`ask(i)` 查单点。
- `find_first(k)` 从左累计达到 k 的位置；`find_last(k)` 从右累计达到 k 的位置。

最小示例（放入 `main`）：

```

1 AddSum_segt t; t.setN(3);
2 t.upd(2,5); t.add(1,2,2);
3 assert(t.ask(1,3)==9);

```

使用边界

这是区间加的特化实现；要更换信息或标记类型，使用外面的两份通用模板。`set/setN` 直接初始化全零节点，不递归建树。区间长度由递归参数计算，不存入节点；`ask` 和二分携带祖先标记，不下传、不改树。使用二分查询时所有点值必须非负，且 $1 \leq k \leq$ 总和； k 参数为 `ll`。三个类名不同，可以组合粘贴。

完整代码

```

1 //////////////////////////////////////////////////
2 //
3 // template for Segment Tree — 区间加 / 区间和 / 前缀和二分
4 // version 1.1 (Last Update Jun 27th, 2026)
5 //
6 // usage:
7 //   AddSum_segt st; st.setN(n); // or set(l,r), 1-indexed
8 //   upd/add point; add(l,r,v) range; ask; find_first/last
9 //
10 //////////////////////////////////////
11 struct AddSum_segt{
12 private:
13   struct node{ll s=0,a=0;};
14   int vl=1,vr=0;
15   vector<node> tr;
16   inline void pushup(int id){tr[id].s=tr[id<<1].s+tr[id<<1|1].s;}
17   inline void push(int id,ll x,int len){
18     tr[id].s+=x*len;
19     if(len>1)tr[id].a+=x;
20   }
21   inline void pushdown(int id,int l,int r){
22     ll x=tr[id].a;if(!x)return;
23     int mid=l+((ll)r-l)/2;
24     push(id<<1,x,mid-l+1);push(id<<1|1,x,r-mid);
25     tr[id].a=0;
26   }
27   void update(int x,ll v,bool assign,int id,int l,int r){
28     if(l==r){if(assign)tr[id].s=v;else tr[id].s+=v;tr[id].a=0;return;}
29     int mid=l+((ll)r-l)/2;pushdown(id,l,r);
30     if(x<=mid)update(x,v,assign,id<<1,l,mid);
31     else update(x,v,assign,id<<1|1,mid+1,r);
32     pushup(id);
33   }
34   void modify(int ql,int qr,ll x,int id,int l,int r){
35     if(ql<=l&&r<=qr){push(id,x,r-l+1);return;}
36     int mid=l+((ll)r-l)/2;pushdown(id,l,r);
37     if(ql<=mid)modify(ql,qr,x,id<<1,l,mid);
38     if(mid<qr)modify(ql,qr,x,id<<1|1,mid+1,r);
39     pushup(id);
40   }
41   ll query(int ql,int qr,int id,int l,int r,ll a)const{
42     // carry ancestor tags; queries do not change the tree
43     if(ql<=l&&r<=qr)return tr[id].s+a*(r-l+1);
44     int mid=l+((ll)r-l)/2;a+=tr[id].a;
45     if(qr<=mid)return query(ql,qr,id<<1,l,mid,a);
46     if(mid<ql)return query(ql,qr,id<<1|1,mid+1,r,a);
47     ll x=query(ql,qr,id<<1,l,mid,a),y=query(ql,qr,id<<1|1,mid+1,r,a);
48     return x+y;
49   }
50   int find(ll x,bool rev)const{
51     int id=1,l=vl,r=vr;ll a=0;
52     while(l<r){
53       int mid=l+((ll)r-l)/2;a+=tr[id].a;
54       ll s=tr[id<<1|rev].s+a*(rev?r-mid:mid-l+1);

```

```

55     bool right=rev;
56     if(x>s){x-=s;right=!right;}
57     id=id<<1|right;
58     if(right)l=mid+1;else r=mid;
59 }
60 return l;
61 }
62 public:
63 void set(){vl=1;vr=0;tr.clear();}
64 void set(int l,int r){
65     ll n=(ll)r-l+1;assert(1<=n&&n<=INT_MAX/4);
66     vl=l;vr=r;tr.assign(4*n,node());
67 }
68 void setN(int n){set(1,n);}
69 void upd(int x,ll v){assert(vl<=x&&x<=vr);update(x,v,true,1,vl,vr);}
70 void add(int x,ll v){assert(vl<=x&&x<=vr);update(x,v,false,1,vl,vr);}
71 void add(int l,int r,ll v){
72     assert(vl<=l&&(ll)l<=1ll+r&&r<=vr);
73     if(l<=r)modify(l,r,v,1,vl,vr);
74 }
75 ll ask(int l,int r)const{assert(vl<=l&&l<=r&&r<=vr);return query(l,r,1,vl,vr,0);}
76 ll ask(int x)const{return ask(x,x);}
77 // all leaf values must be non-negative
78 int find_first(ll x)const{assert(!tr.empty()&&0<x&&x<=tr[1].s);return find(x,false);}
79 int find_last(ll x)const{assert(!tr.empty()&&0<x&&x<=tr[1].s);return find(x,true);}
80 };
81 // end for ds/Segment Tree/AddSum.cpp
82 //////////////////////////////////////

```

2.6 ds/Segment Tree/MaxAdd.cpp

区间加、区间最大值

什么时候用

默认闭区间下标 1..n、初值全零，初始化 $O(n)$ ，单次修改查询 $O(\log n)$ 。

调用顺序与接口

- `setN(n)` 或 `set(l,r)` 指定有效闭区间。
- `upd(i,v)` 是赋值，`add(i,v)` 是加法；`add(l,r,v)` 区间加。
- `ask(l,r)` 查询，`ask(i)` 查单点。

最小示例（放入 main）：

```

1 MaxAdd_segt t; t.setN(3);
2 t.upd(2,5); t.add(1,2,2);
3 assert(t.ask(1,3)==7);

```

使用边界

这是区间加的特化实现；要更换信息或标记类型，使用外面的两份通用模板。`set/setN` 直接初始化全零节点，不递归建树。若初始数组不是全零，应逐点 `upd`；负数数组不需要改初始化，只要把所有位置正确赋值。三个类名不同，可以组合粘贴。

完整代码

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // template for Segment Tree — 区间加 / 区间最大
4 // version 1.0 (Last Update Jun 27th, 2026)
5 //
6 // usage:
7 //   MaxAdd_segt st; st.setN(n); upd/add; ask(l,r) for range max
8 //   default leaf value 0
9 //
10 ///////////////////////////////////////////////////////////////////
11 struct MaxAdd_segt{
12 private:
13   struct node{ll s=0,a=0;};
14   int vl=1,vr=0;
15   vector<node> tr;
16   inline void pushup(int id){tr[id].s=max(tr[id<<1].s,tr[id<<1|1].s);}
17   inline void push(int id,ll x){
18     tr[id].s+=x;
19     tr[id].a+=x;
20   }
21   inline void pushdown(int id){
22     ll x=tr[id].a;if(!x)return;
23     push(id<<1,x);push(id<<1|1,x);
24     tr[id].a=0;
25   }
26   void update(int x,ll v,bool assign,int id,int l,int r){
27     if(l==r){if(assign)tr[id].s=v;else tr[id].s+=v;tr[id].a=0;return;}
28     int mid=l+((ll)r-l)/2;pushdown(id);
29     if(x<=mid)update(x,v,assign,id<<1,l,mid);
30     else update(x,v,assign,id<<1|1,mid+1,r);
31     pushup(id);
32   }
33   void modify(int ql,int qr,ll x,int id,int l,int r){
34     if(ql<=l&&r<=qr){push(id,x);return;}
35     int mid=l+((ll)r-l)/2;pushdown(id);
36     if(ql<=mid)modify(ql,qr,x,id<<1,l,mid);
37     if(mid<qr)modify(ql,qr,x,id<<1|1,mid+1,r);
38     pushup(id);
39   }
40   ll query(int ql,int qr,int id,int l,int r){
41     if(ql<=l&&r<=qr)return tr[id].s;
42     int mid=l+((ll)r-l)/2;pushdown(id);
43     if(qr<=mid)return query(ql,qr,id<<1,l,mid);
44     if(mid<ql)return query(ql,qr,id<<1|1,mid+1,r);
45     ll x=query(ql,qr,id<<1,l,mid),y=query(ql,qr,id<<1|1,mid+1,r);
46     return max(x,y);
47   }
48
49 public:
50   void set(){vl=1;vr=0;tr.clear();}
51   void set(int l,int r){
52     ll n=(ll)r-l+1;assert(1<=n&&n<=INT_MAX/4);
53     vl=l;vr=r;tr.assign(4*n,node());
54   }

```

```

55 void setN(int n){set(1,n);}
56 void upd(int x,ll v){assert(vl<=x&&x<=vr);update(x,v,true,1,vl,vr);}
57 void add(int x,ll v){assert(vl<=x&&x<=vr);update(x,v,false,1,vl,vr);}
58 void add(int l,int r,ll v){
59     assert(vl<=l&&(ll)l<=1ll+r&&r<=vr);
60     if(l<=r)modify(l,r,v,1,vl,vr);
61 }
62 ll ask(int l,int r){assert(vl<=l&&l<=r&&r<=vr);return query(l,r,1,vl,vr);}
63 ll ask(int x){return ask(x,x);}
64
65 };
66 // end for ds/Segment Tree/MaxAdd.cpp
67 //////////////////////////////////////

```

2.7 ds/Segment Tree/MinAdd.cpp

区间加、区间最小值

什么时候用

默认闭区间下标 1..n、初值全零，初始化 $O(n)$ ，单次修改查询 $O(\log n)$ 。

调用顺序与接口

- `setN(n)` 或 `set(l,r)` 指定有效闭区间。
- `upd(i,v)` 是赋值，`add(i,v)` 是加法；`add(l,r,v)` 区间加。
- `ask(l,r)` 查询，`ask(i)` 查单点。

最小示例（放入 main）：

```

1 MinAdd_segt t; t.setN(3);
2 t.upd(2,5); t.add(1,2,2);
3 assert(t.ask(1,3)==0);

```

使用边界

这是区间加的特化实现；要更换信息或标记类型，使用外面的两份通用模板。`set/setN` 直接初始化全零节点，不递归建树。若初始数组不是全零，应逐点 `upd`；负数数组不需要改初始化，只要把所有位置正确赋值。三个类名不同，可以组合粘贴。

完整代码

```

1 //////////////////////////////////////
2 //
3 // template for Segment Tree — 区间加 / 区间最小
4 // version 1.0 (Last Update Jun 27th, 2026)
5 //
6 // usage:
7 //   MinAdd_segt st; st.setN(n); upd/add; ask(l,r) for range min
8 //   needs chkmin; set leaf INF in build for min queries
9 //
10 //////////////////////////////////////
11 struct MinAdd_segt{
12 private:
13     struct node{ll s=0,a=0;};
14     int vl=1,vr=0;
15     vector<node> tr;

```

```

16 inline void pushup(int id){tr[id].s=min(tr[id<<1].s,tr[id<<1|1].s);}
17 inline void push(int id,ll x){
18     tr[id].s+=x;
19     tr[id].a+=x;
20 }
21 inline void pushdown(int id){
22     ll x=tr[id].a;if(!x)return;
23     push(id<<1,x);push(id<<1|1,x);
24     tr[id].a=0;
25 }
26 void update(int x,ll v,bool assign,int id,int l,int r){
27     if(l==r){if(assign)tr[id].s=v;else tr[id].s+=v;tr[id].a=0;return;}
28     int mid=l+((ll)r-l)/2;pushdown(id);
29     if(x<=mid)update(x,v,assign,id<<1,l,mid);
30     else update(x,v,assign,id<<1|1,mid+1,r);
31     pushup(id);
32 }
33 void modify(int ql,int qr,ll x,int id,int l,int r){
34     if(ql<=l&&r<=qr){push(id,x);return;}
35     int mid=l+((ll)r-l)/2;pushdown(id);
36     if(ql<=mid)modify(ql,qr,x,id<<1,l,mid);
37     if(mid<qr)modify(ql,qr,x,id<<1|1,mid+1,r);
38     pushup(id);
39 }
40 ll query(int ql,int qr,int id,int l,int r){
41     if(ql<=l&&r<=qr)return tr[id].s;
42     int mid=l+((ll)r-l)/2;pushdown(id);
43     if(qr<=mid)return query(ql,qr,id<<1,l,mid);
44     if(mid<ql)return query(ql,qr,id<<1|1,mid+1,r);
45     ll x=query(ql,qr,id<<1,l,mid),y=query(ql,qr,id<<1|1,mid+1,r);
46     return min(x,y);
47 }
48
49 public:
50 void set(){vl=1;vr=0;tr.clear();}
51 void set(int l,int r){
52     ll n=(ll)r-l+1;assert(1<=n&&n<=INT_MAX/4);
53     vl=1;vr=r;tr.assign(4*n,node());
54 }
55 void setN(int n){set(1,n);}
56 void upd(int x,ll v){assert(vl<=x&&x<=vr);update(x,v,true,1,vl,vr);}
57 void add(int x,ll v){assert(vl<=x&&x<=vr);update(x,v,false,1,vl,vr);}
58 void add(int l,int r,ll v){
59     assert(vl<=l&&(ll)l<=1ll+r&&r<=vr);
60     if(l<=r)modify(l,r,v,1,vl,vr);
61 }
62 ll ask(int l,int r){assert(vl<=l&&l<=r&&r<=vr);return query(l,r,1,vl,vr);}
63 ll ask(int x){return ask(x,x);}
64
65 };
66 // end for ds/Segment Tree/MinAdd.cpp
67 //////////////////////////////////////

```

2.8 ds/Segment Tree(with lazytag).cpp

通用线段树（带懒标记）

什么时候用

默认维护区间和，下标 1..n，初值全零。递归部分只处理 info，按题目改写信息与标记的运算。

调用顺序与接口

- `segt t; t.setN(n)` 初始化。
- `upd(i,v)` 单点赋值；`ask(l,r)` 区间和，`ask(i)` 单点值。
- `add(i,v)` 单点加，`add(l,r,v)` 区间加。

最小示例（放入 main）：

```
1 segt t; t.setN(3); t.upd(2,5);
2 t.add(1,2,2); assert(t.ask(1,3)==9);
```

使用边界

info + info 按左右顺序合并；upd 在叶子整体替换 info，单点 add 使用 info(delta,0)，通过 info += info 更新叶子。初值在 build 的 info(0,1) 处修改。参数转换、提取答案只改公开接口。USE_COMPARE 默认开启；自定义信息不支持减法与比较时，删除文件顶部的 define 行，对应运算和二分函数就不编译。默认二分要求所有点值非负、目标为正且不超过总和。set/setN 清空重建。两版同名 segt，只选一份。lazt += lazt 的顺序是旧标记后接新标记；info += lazt 负责作用标记，区间长度等元数据由 info 保存。不能仅因支持单点加，就直接认为相同统计能支持区间加。

完整代码

```
1 //////////////////////////////////////
2 //
3 // template for Segment Tree
4 // version 1.1 (Last Update Jun 27th, 2026)
5 //
6 // usage:
7 //   segt st; st.setN(n); // or set(l,r), 1-indexed
8 //   upd/add point; add(l,r,v) range; ask; find_first/last
9 //
10 //////////////////////////////////////
11 // remove this line when info has no subtraction / comparison
12 #define USE_COMPARE
13 struct segt{
14 private:
15 //////////////////////////////////////
16 // basic operation
17 struct lazt{
18   // modify here
19   ll a;
20   bool empty()const{return a==0;}
21   void set(){a=0;}
22   lazt():a(0){}
23   lazt(ll _a):a(_a){}
24   inline lazt& operator +=(const lazt &y){
25     a+=y.a;
26     return *this;
27   }
28   inline lazt operator +(const lazt &y)const{
29     return lazt(*this)+=y;
30   }
31 }
```

```

31 };
32 struct info{
33     // modify here
34     ll s,len;
35     info():s(0),len(0){}
36     info(ll _s,ll _len):s(_s),len(_len){}
37     inline info& operator +=(const info &y){
38         s+=y.s;len+=y.len;
39         return *this;
40     }
41     inline info operator +(const info &y)const{
42         return info(*this)+=y;
43     }
44     inline info& operator +=(const lazt &y){
45         s+=len*y.a;
46         return *this;
47     }
48     inline info operator +(const lazt &y)const{
49         return info(*this)+=y;
50     }
51 #ifdef USE_COMPARE
52     // for find_first or find_last
53     inline info& operator -=(const info &y){
54         s-=y.s;len-=y.len;
55         return *this;
56     }
57     inline info operator -(const info &y)const{
58         return info(*this)-=y;
59     }
60     inline bool operator <= (const info &a)const{
61         return s<=a.s;
62     }
63 #endif
64 };
65 ///////////////////////////////////////////////////
66 // basic segment tree template, private operations
67 int vl=1,vr=0;
68 vector<info> val;
69 vector<lazt> laz;
70 inline void push(int id,const lazt &x){
71     laz[id]+=x;val[id]+=x;
72 }
73 inline void pushdown(int id){
74     if(!laz[id].empty()) push(id<<1,laz[id]),push(id<<1|1,laz[id]),laz[id].set();
75 }
76 inline void pushup(int id){
77     val[id]=val[id<<1]+val[id<<1|1];
78 }
79 void build(int id,int l,int r){
80     laz[id]=lazt();
81     if(l==r){
82         val[id]=info(0,1); // please modify this init value
83         return;
84     }
85     int mid=l+((ll)r-l)/2;

```

```

86  build(id<<1,l,mid);build(id<<1|1,mid+1,r);
87  pushup(id);
88  }
89  void update(int x,const info &v,int id,int l,int r){
90      if(l==r){
91          assert(x==l);
92          val[id]=v;laz[id].set();return;
93      }
94      int mid=l+((ll)r-l)/2;
95      pushdown(id);
96      if(x<=mid) update(x,v,id<<1,l,mid);
97      else update(x,v,id<<1|1,mid+1,r);
98      pushup(id);
99  }
100 void modify(int x,const info &v,int id,int l,int r){
101     if(l==r){
102         assert(x==l);
103         val[id]+=v;return;
104     }
105     int mid=l+((ll)r-l)/2;
106     pushdown(id);
107     if(x<=mid) modify(x,v,id<<1,l,mid);
108     else modify(x,v,id<<1|1,mid+1,r);
109     pushup(id);
110 }
111 void modify(int ql,int qr,const laz &v,int id,int l,int r){
112     if(ql<=l&&r<=qr){push(id,v);return;}
113     int mid=l+((ll)r-l)/2;
114     pushdown(id);
115     if(ql<=mid) modify(ql,qr,v,id<<1,l,mid);
116     if(mid<qr) modify(ql,qr,v,id<<1|1,mid+1,r);
117     pushup(id);
118 }
119 info query(int ql,int qr,int id,int l,int r){
120     if(ql<=l&&r<=qr) return val[id];
121     int mid=l+((ll)r-l)/2;
122     pushdown(id);
123     if(qr<=mid) return query(ql,qr,id<<1,l,mid);
124     if(mid<ql) return query(ql,qr,id<<1|1,mid+1,r);
125     return query(ql,qr,id<<1,l,mid)+query(ql,qr,id<<1|1,mid+1,r);
126 }
127 #ifdef USE_COMPARE
128 int find_first(info x,int id,int l,int r){
129     // if no compare between infos, delete this
130     if(l==r) return l;
131     int mid=l+((ll)r-l)/2;
132     pushdown(id);
133     return x<=val[id<<1]?find_first(x,id<<1,l,mid):find_first(x-val[id<<1],id<<1|1,mid+1,r);
134 }
135 int find_last(info x,int id,int l,int r){
136     // if no compare between infos, delete this
137     if(l==r) return l;
138     int mid=l+((ll)r-l)/2;
139     pushdown(id);
140     return x<=val[id<<1|1]?find_last(x,id<<1|1,mid+1,r):find_last(x-val[id<<1|1],id<<1,l,mid);

```

```

141 }
142 #endif
143 //////////////////////////////////////////////////
144 // public operations, remember to setN()/set()
145 // convert arguments to info / lazt, and extract the answer here
146 public:
147 void set(){vl=1;vr=0;val.clear();laz.clear();}
148 void set(int _l,int _r){
149     ll n=(ll)_r-_l+1;assert(1<=n&&n<=INT_MAX/4);
150     vl=_l;vr=_r;val.assign(4*n,info());
151     laz.assign(4*n,lazt());
152     build(1,vl,vr);
153 }
154 void setN(int _n){
155     set(1,_n);
156 }
157 void upd(int x,ll a){
158     assert(vl<=x&&x<=vr);
159     update(x,info(a,1),1,vl,vr);
160 }
161 void add(int x,ll a){
162     assert(vl<=x&&x<=vr);
163     modify(x,info(a,0),1,vl,vr);
164 }
165 void add(int ql,int qr,ll a){
166     assert(vl<=ql&&(ll)ql<=1ll+qr&&qr<=vr);
167     if(ql<=qr) modify(ql,qr,lazt(a),1,vl,vr);
168 }
169 ll ask(int ql,int qr){
170     assert(vl<=ql&&ql<=qr&&qr<=vr);
171     return query(ql,qr,1,vl,vr).s;
172 }
173 ll ask(int x){
174     assert(vl<=x&&x<=vr);
175     // query a[x]
176     return query(x,x,1,vl,vr).s;
177 }
178 #ifdef USE_COMPARE
179 int find_first(ll x){
180     assert(!val.empty()&&x>0&&info(x,0)<=val[1]);
181     return find_first(info(x,0),1,vl,vr);
182 }
183 int find_last(ll x){
184     assert(!val.empty()&&x>0&&info(x,0)<=val[1]);
185     return find_last(info(x,0),1,vl,vr);
186 }
187 #endif
188 };
189
190 #undef USE_COMPARE
191 // end for ds/Segment Tree(with lazytag).cpp
192 //////////////////////////////////////////////////

```

2.9 ds/Segment Tree(without lazytag).cpp

通用线段树（不带懒标记）

什么时候用

默认维护区间和，下标 1..n，初值全零。递归部分只处理 info，按题目改写信息与标记的运算。

调用顺序与接口

- segt t; t.setN(n) 初始化。
- upd(i,v) 单点赋值；ask(l,r) 区间和，ask(i) 单点值。
- add(i,v) 单点加，不支持区间加。

最小示例（放入 main）：

```
1 segt t; t.setN(3); t.upd(2,5);
2 t.add(2,2); assert(t.ask(1,3)==7);
```

使用边界

info + info 按左右顺序合并；upd 在叶子整体替换 info，单点 add 使用 info(delta,0)，通过 info += info 更新叶子。初值在 build 的 info(0,1) 处修改。参数转换、提取答案只改公开接口。USE_COMPARE 默认开启；自定义信息不支持减法与比较时，删除文件顶部的 define 行，对应运算和二分函数就不编译。默认二分要求所有点值非负、目标为正且不超过总和。set/setN 清空重建。两版同名 segt，只选一份。

改写示例：单点加、区间最大子段和（非空）删除顶部的 #define USE_COMPARE，把 info 替换为下面这段：区间 ask 的返回字段从 s 改为 mx，其他部分不改。len=0 的参数表示单点增量，len=1 表示叶子；合并的左右区间均非空。

```
1 struct info{
2     ll s,pre,suf,mx,len;
3     info(ll x=0,ll n=0):s(x),pre(x),suf(x),mx(x),len(n){}
4     inline info& operator +=(const info &y){
5         if(!y.len){ // point add: y is info(delta,0)
6             s+=y.s;pre=suf=mx=s;return *this;
7         }
8         mx=max({mx,y.mx,suf+y.pre});
9         pre=max(pre,s+y.pre);suf=max(y.suf,y.s+suf);
10        s+=y.s;len+=y.len;return *this;
11    }
12    inline info operator +(const info &y)const{return info(*this)+=y;}
13 };
```

完整代码

```
1 //////////////////////////////////////
2 //
3 // template for Segment Tree
4 // version 1.1 (Last Update Jun 27th, 2026)
5 //
6 // usage:
7 //     segt st; st.setN(n); // or set(l,r), 1-indexed
8 //     upd/add point; ask(l,r); find_first/last
9 //
10 //////////////////////////////////////
11 // remove this line when info has no subtraction / comparison
12 #define USE_COMPARE
13 struct segt{
14 private:
15     //////////////////////////////////////
```

```

16 // basic operation
17 struct info{
18     // modify here
19     ll s,len;
20     info():s(0),len(0){}
21     info(ll _s,ll _len):s(_s),len(_len){}
22     inline info& operator +=(const info &y){
23         s+=y.s;len+=y.len;
24         return *this;
25     }
26     inline info operator +(const info &y)const{
27         return info(*this)+=y;
28     }
29 #ifdef USE_COMPARE
30     // for find_first or find_last
31     inline info& operator -=(const info &y){
32         s-=y.s;len-=y.len;
33         return *this;
34     }
35     inline info operator -(const info &y)const{
36         return info(*this)-=y;
37     }
38     inline bool operator <= (const info &a)const{
39         return s<=a.s;
40     }
41 #endif
42 };
43 ///////////////////////////////////////////////////
44 // basic segment tree template, private operations
45 int vl=1,vr=0;
46 vector<info> val;
47 inline void pushup(int id){
48     val[id]=val[id<<1]+val[id<<1|1];
49 }
50 void build(int id,int l,int r){
51     if(l==r){
52         val[id]=info(0,1); // please modify this init value
53         return;
54     }
55     int mid=l+((ll)r-l)/2;
56     build(id<<1,l,mid);build(id<<1|1,mid+1,r);
57     pushup(id);
58 }
59 void update(int x,const info &v,int id,int l,int r){
60     if(l==r){
61         assert(x==l);
62         val[id]=v;return;
63     }
64     int mid=l+((ll)r-l)/2;
65     if(x<=mid) update(x,v,id<<1,l,mid);
66     else update(x,v,id<<1|1,mid+1,r);
67     pushup(id);
68 }
69 void modify(int x,const info &v,int id,int l,int r){
70     if(l==r){

```

```

71     assert(x==l);
72     val[id]+=v;return;
73 }
74 int mid=l+((ll)r-l)/2;
75 if(x<=mid) modify(x,v,id<<1,l,mid);
76 else modify(x,v,id<<1|1,mid+1,r);
77 pushup(id);
78 }
79 info query(int ql,int qr,int id,int l,int r){
80     if(ql<=l&&r<=qr) return val[id];
81     int mid=l+((ll)r-l)/2;
82     if(qr<=mid) return query(ql,qr,id<<1,l,mid);
83     if(mid<ql) return query(ql,qr,id<<1|1,mid+1,r);
84     return query(ql,qr,id<<1,l,mid)+query(ql,qr,id<<1|1,mid+1,r);
85 }
86 #ifdef USE_COMPARE
87 int find_first(info x,int id,int l,int r){
88     // if no compare between infos, delete this
89     if(l==r) return l;
90     int mid=l+((ll)r-l)/2;
91     return x<=val[id<<1]?find_first(x,id<<1,l,mid):find_first(x-val[id<<1],id<<1|1,mid+1,r);
92 }
93 int find_last(info x,int id,int l,int r){
94     // if no compare between infos, delete this
95     if(l==r) return l;
96     int mid=l+((ll)r-l)/2;
97     return x<=val[id<<1|1]?find_last(x,id<<1|1,mid+1,r):find_last(x-val[id<<1|1],id<<1,l,mid);
98 }
99 #endif
100 //////////////////////////////////////
101 // public operations, remember to setN()/set()
102 // convert arguments to info / lazt, and extract the answer here
103 public:
104 void set(){vl=1;vr=0;val.clear();}
105 void set(int _l,int _r){
106     ll n=(ll)_r-_l+1;assert(1<=n&&n<=INT_MAX/4);
107     vl=_l;vr=_r;val.assign(4*n,info());
108     build(1,vl,vr);
109 }
110 void setN(int _n){
111     set(1,_n);
112 }
113 void upd(int x,ll a){
114     assert(vl<=x&&x<=vr);
115     update(x,info(a,1),1,vl,vr);
116 }
117 void add(int x,ll a){
118     assert(vl<=x&&x<=vr);
119     modify(x,info(a,0),1,vl,vr);
120 }
121 ll ask(int ql,int qr){
122     assert(vl<=ql&&ql<=qr&&qr<=vr);
123     return query(ql,qr,1,vl,vr).s;
124 }
125 ll ask(int x){

```

```

126     assert(vl<=x&&x<=vr);
127     // query a[x]
128     return query(x,x,1,vl,vr).s;
129 }
130 #ifdef USE_COMPARE
131 int find_first(ll x){
132     assert(!val.empty()&&x>0&&info(x,0)<=val[1]);
133     return find_first(info(x,0),1,vl,vr);
134 }
135 int find_last(ll x){
136     assert(!val.empty()&&x>0&&info(x,0)<=val[1]);
137     return find_last(info(x,0),1,vl,vr);
138 }
139 #endif
140 };
141
142 #undef USE_COMPARE
143 // end for ds/Segment Tree(without lazytag).cpp
144 ///////////////////////////////////

```

2.10 ds/zkw.cpp

非递归线段树框架

什么时候用

单点赋值和闭区间聚合，当前 info 维护区间和与最大前缀和（允许空前缀）。单次 $O(\log n)$ 。

调用顺序与接口

- `zkw_segt t; t.setN(n)` 下标为 $0..n-1$ 。
- `upd(i,a,b)` 设置该叶 info 的 $s=a$ 、 $mx=b$ 。
- 合并两段时 $s=L.s+R.s$ ， $mx=\max(L.mx,L.s+R.mx)$ ；ask 返回最终 mx 。

最小示例（放入 main）：

```

1 zkw_segt t; t.setN(3);
2 t.upd(0,-4,0); t.upd(1,9,9); t.upd(2,-2,0);
3 assert(t.ask(0,2)==5);

```

使用边界

普通数组的叶子设为 $s=a[i]$ 、 $mx=\max(0,a[i])$ 。若 s 全为零且 mx 非负，也可用于区间最大值。其他统计应重写 info 的合并和单位元；合并必须满足结合律。下标从 0 开始。

完整代码

```

1 //
2 // template for ZKW segment tree
3 // version 1.0 (Last Update Jun 13th, 2026)
4 //
5 // usage:
6 //     zkw_segt z; z.setN(n); z.upd(i, s, mx); z.ask(l, r);
7 //     0-indexed; leaf mx=max(0ll,s); ask returns maximum prefix sum
8 //
9 struct zkw_segt{
10 // information

```

```

11 struct info{
12     // sum and maximum prefix sum, including the empty prefix
13     ll s,mx;
14     info():s(0),mx(0){}
15     info(ll _s,ll _mx):s(_s),mx(_mx){}
16     inline info& operator +=(const info &y){
17         ll os=s;
18         s+=y.s;
19         mx=max(mx,os+y.mx);
20         return *this;
21     }
22     inline info operator +(const info &y)const{
23         return info(*this)+=y;
24     }
25 };
26 // basic segment tree template, private operations
27 private:
28 int n=0,B=0;vector<info> a;
29 void modify(int x,info v){
30     x+=B+1;
31     a[x]=v;
32     while(x>1) x>>=1,a[x]=a[x<<1]+a[x<<1|1];
33 }
34 info query(int l,int r){
35     l+=B,r+=B+2;
36     info ql,qr;
37     while(l^r^1){
38         if(~l&1) ql=ql+a[l^1];
39         if(r&1) qr=a[r^1]+qr;
40         l>>=1,r>>=1;
41     }
42     return ql+qr;
43 }
44 // basic segment tree template, public operations
45 // please transfer (int,int) to info here
46 public:
47 void setN(int _n){
48     assert(1<=_n&&_n<(1<<29));n=_n;B=1;
49     while(B<n+2)B*=2;
50     a.assign(2*B,info());
51 }
52 void upd(int x,ll a,ll b){
53     assert(0<=x&&x<n);
54     modify(x,info(a,b));
55 }
56 ll ask(int l,int r){
57     assert(0<=l&&l<=r&&r<n);
58     return query(l,r).mx;
59 }
60 };

```

2.11 ds/Dynamic Segment Tree.cpp

大值域稀疏单点统计

什么时候用

在整数键 $-10^9..10^9$ 上做单点加、区间和；节点带键交换以压缩空间。单次 $O(\log V)$ 。

调用顺序与接口

- `dynamic_segt t; t.set()` 清空；`set(m)` 可预留节点。
- `add(x,v)` 给键 `x` 的权值加 `v`；`ask(l,r)` 求键区间权值和。

最小示例（放入 main）：

```
1 dynamic_segt t; t.set();
2 t.add(-100,3); t.add(200,4);
3 assert(t.ask(-100,0)==3);
```

使用边界

空键权值为零。与主席树不同：不保留历史版本，也不支持复用旧根。文件有全局对象 `T`。

完整代码

```
1 //
2 // template for Dynamic Segment Tree
3 // this code accept single-point modify, range query, cost memory O(N)
4 // version 1.0 (Last Update Jun 16th, 2026)
5 //
6 // usage:
7 //   dynamic_segt T; T.set(); T.add(pos, val); T.ask(l, r);
8 //   sparse segt on value domain [L,R]
9 //
10 struct dynamic_segt{
11 private:
12   static const int L=-1e9,R=1e9;
13   int tot=0;
14   struct info{
15     ll s;
16     info():s(0){}
17     info(ll _s):s(_s){}
18     inline info& operator +=(const info &x){s+=x.s;return *this;}
19     inline info operator +(const info &x) const{return info(*this)+=x;}
20   };
21   struct node{
22     int p=0,ls=0,rs=0;
23     info self,val;
24   };
25   vector<node> tr=vector<node>(1);
26   int rt=0;
27   int new_node(int p,info x){
28     assert(tr.size()<INT_MAX);tr.push_back({p,0,0,x,x});
29     return ++tot;
30   }
31   inline void pushup(int id){
32     tr[id].val=tr[tr[id].ls].val+tr[id].self+tr[tr[id].rs].val;
33   }
34   int add(int p,info v,int id,int l,int r){
35     if(!id)return new_node(p,v);
```

```

36  if(tr[id].p==p){tr[id].self+=v;pushup(id);return id;}
37  int mid=l+((ll)r-l)/2;
38  if(p<=mid){
39      if(p>tr[id].p)swap(tr[id].p,p),swap(tr[id].self,v);
40      int x=add(p,v,tr[id].ls,l,mid);tr[id].ls=x;
41  }else{
42      if(p<tr[id].p)swap(tr[id].p,p),swap(tr[id].self,v);
43      int x=add(p,v,tr[id].rs,mid+1,r);tr[id].rs=x;
44  }
45  pushup(id);return id;
46 }
47 void query(int ql,int qr,int id,int l,int r,info &X){
48     if(max(ql,l)>min(r,qr)||!id) return;
49     if(ql<=l&&r<=qr){X+=tr[id].val;return;}
50     int mid=l+((ll)r-l)/2;
51     query(ql,qr,tr[id].ls,l,mid,X);
52     if(ql<=tr[id].p&&tr[id].p<=qr) X+=tr[id].self;
53     query(ql,qr,tr[id].rs,mid+1,r,X);
54 }
55 public:
56 void set(int m=0){assert(m>=0);tot=rt=0;tr.assign(1,node());tr.reserve((size_t)m+1);}
57 void add(int p,ll s){
58     assert(L<=p&&p<=R);
59     rt=add(p,info(s),rt,L,R);
60 }
61 ll ask(int l,int r){
62     assert(L<=l&&l<=r&&r<=R);
63     info X;
64     query(l,r,rt,L,R,X);
65     return X.s;
66 }
67 }T;

```

2.12 ds/Segment Tree Merge.cpp

可合并的多棵线段树

什么时候用

维护多棵互不共享节点的树，值域 $0..n$ 。当前权值设计用于统计所有键 $y \geq x$ 的 $(y-x)^*$ 权值。

调用顺序与接口

- `segt_merge t; t.setN(n)` 初始化池，外部根变量设为 0。
- `add(rt,y,w)` 给键 y 加权 w ，根按引用更新。
- `ask(rt,x)` 返回 $\sum_{y \geq x} (y-x)^* w[y]$ 。
- `merge(a,b)` 把 b 合入 a 并把 b 置零。

最小示例（放入 `main`）：

```

1 segt_merge t; t.setN(10); int a=0,b=0;
2 t.add(a,5,2); t.add(b,8,1); t.merge(a,b);
3 assert(b==0 && t.ask(a,3)==9);

```

使用边界

这是破坏性合并，不是可持久化。合并后不能再使用旧根副本。单点操作 $O(\log V)$ ，合并成本取决于重叠节点数；若想查询普通权值和，要改 info/query 的对外接口。

完整代码

```

1 //
2 // template for Segment Tree Merge
3 // version 1.0 (Last Update Jun 25th, 2026)
4 //
5 // usage:
6 //   segt_merge sm; sm.setN(n); int rt=0;
7 //   sm.add(rt,i,v); sm.ask(rt,x); sm.merge(a,b);
8 //
9 struct segt_merge{
10 private:
11 struct info{
12     // modify here
13     ll s,s1;
14     info():s(0),s1(0){}
15     info(ll _s,ll _s1):s(_s),s1(_s1){}
16     inline info& operator +=(const info &y){
17         s+=y.s;s1+=y.s1;
18         return *this;
19     }
20     inline info operator +(const info &y)const{
21         return info(*this)+=y;
22     }
23 };
24 vector<info> val=vector<info>(1);
25 vector<int> ls{0},rs{0};int n=0,tot=0;
26 inline void pushup(int id){val[id]=val[ls[id]]+val[rs[id]];}
27 int add(int x,info v,int id,int l,int r){
28     if(!id){
29         assert(tot<INT_MAX-1);id=++tot;
30         val.emplace_back();ls.push_back(0);rs.push_back(0);
31     }
32     if(l==r){val[id]+=v;return id;}
33     int mid=l+(r-l)/2;
34     if(x<=mid){int t=add(x,v,ls[id],l,mid);ls[id]=t;}
35     else {int t=add(x,v,rs[id],mid+1,r);rs[id]=t;}
36     pushup(id);return id;
37 }
38 info query(int ql,int qr,int id,int l,int r){
39     if(!id||max(l,ql)>min(r,qr)) return info();
40     if(ql<=l&&r<=qr) return val[id];
41     int mid=l+(r-l)/2;
42     return query(ql,qr,ls[id],l,mid)+query(ql,qr,rs[id],mid+1,r);
43 }
44 void merge(int &u,int &v,int l,int r){
45     if(!u||!v){u|=v;return;}
46     if(l==r){val[u]+=val[v];return;}
47     int mid=l+(r-l)/2;
48     merge(ls[u],ls[v],l,mid);
49     merge(rs[u],rs[v],mid+1,r);
50     pushup(u);
51 }

```

```

52 public:
53 void setN(int _n){
54     assert(_n>=0);n=_n;tot=0;
55     val.assign(1,info());ls.assign(1,0);rs.assign(1,0);
56 }
57 void add(int &id,int x,int o){
58     assert(0<=x&&x<=n);
59     assert(0<=id&&id<=tot);
60     id=add(x,info(o,1ll*x*o),id,0,n);
61 }
62 ll ask(int &id,int x){
63     assert(0<=id&&id<=tot&&0<=x&&x<=n);
64     auto it=query(x,n,id,0,n);
65     return it.s1-it.s*x;
66 }
67 void merge(int &x,int &y){
68     assert(0<=x&&x<=tot&&0<=y&&y<=tot&&(&x!=&y)&&(!x||x!=y));
69     merge(x,y,0,n);
70     y=0;
71 }
72 };

```

2.13 ds/lichao.cpp

线段李超树

什么时候用

插入只在一段整数横坐标上有效的直线，查询某点的最小值。

调用顺序与接口

- lichao t; t.set() 清空。
- add(line{k,b},l,r) 插入 $y=kx+b$ ；省略 l,r 表示整个默认值域。
- ask(x) 返回最小纵坐标，没有覆盖返回 2e18。

最小示例（放入 main）：

```

1 lichao t; t.set();
2 t.add({2,3},0,10); t.add({-1,8},0,10);
3 assert(t.ask(2)==6);

```

使用边界

横坐标及斜率限制在 $[-10^9, 10^9]$ ，有效端点的纵坐标需在 $[-2e18, 2e18]$ 。整条线插入 $O(\log V)$ ，线段插入 $O(\log^2 V)$ ，查询 $O(\log V)$ 。

完整代码

```

1 //
2 // template for lichao segment tree
3 // version 1.2 (Last Update Jun 13th, 2026)
4 //
5 // usage:
6 //   lichao lc; lc.set(); lc.add({k,b}); lc.add({k,b}, l, r);
7 //   ll y = lc.ask(x); // min y=kx+b, INF if empty
8 //

```

```

9 struct line{
10     ll k,b;
11     ll f(ll x){return k*x+b;}
12     // please, check whether use __int128
13 };
14 struct lichao{
15     // you may add y=kb+x where x \in [l,r]
16     // you can query x, answer the **min** y (if no such, return inf = 2e18)
17     static const ll inf=2e18,V=1e9;
18     vector<int> ls{0},rs{0},p{0};int tot=0,cnt=0,rt=0;
19     vector<line> L{{0,inf}};
20     // if you don't use range add, this can up to n
21     inline ll gv(const int &id,const ll &v){return L[id].f(v);}
22     void set(){
23         ls.assign(1,0);rs.assign(1,0);p.assign(1,0);
24         L.assign(1,{0,inf});tot=cnt=rt=0;
25     }
26     int new_node(){
27         assert(tot<INT_MAX-1);ls.push_back(0);rs.push_back(0);p.push_back(0);
28         return ++tot;
29     }
30     int add(int x,int id,int l,int r){
31         if(!id)id=new_node();
32         if(l==r){
33             if(gv(x,l)<gv(p[id],l))p[id]=x;
34             return id;
35         }
36         int mid=l+((ll)r-l)/2;
37         if(gv(x,mid)<gv(p[id],mid))swap(x,p[id]);
38         if(gv(x,l)<gv(p[id],l)){int t=add(x,ls[id],l,mid);ls[id]=t;}
39         if(gv(x,r)<gv(p[id],r)){int t=add(x,rs[id],mid+1,r);rs[id]=t;}
40         return id;
41     }
42     int upd(int x,int ql,int qr,int id,int l,int r){
43         if(max(ql,l)>min(r,qr))return id;
44         if(!id)id=new_node();
45         if(ql<=l&&r<=qr)return add(x,id,l,r);
46         int mid=l+((ll)r-l)/2;
47         int a=upd(x,ql,qr,ls[id],l,mid),b=upd(x,ql,qr,rs[id],mid+1,r);
48         ls[id]=a;rs[id]=b;return id;
49     }
50     ll ask(int x,int id,int l,int r){
51         if(!id) return inf;
52         ll ans=gv(p[id],x);
53         if(l==r) return ans;
54         int mid=l+((ll)r-l)/2;
55         if(x<=mid) ans=min(ans,ask(x,ls[id],l,mid));
56         else ans=min(ans,ask(x,rs[id],mid+1,r));
57         return ans;
58     }
59
60
61 void add(line x,int l=-V,int r=V){
62     assert(-V<=l&&l<=r&&r<=V);
63     assert(-V<=x.k&&x.k<=V);

```

```

64  assert((__int128)x.k*l+x.b>=-inf&&(__int128)x.k*l+x.b<=inf);
65  assert((__int128)x.k*r+x.b>=-inf&&(__int128)x.k*r+x.b<=inf);
66  assert(cnt<INT_MAX-1);L.push_back(x);++cnt;
67  rt=upd(cnt,l,r,rt,-V,V);
68  }
69  ll ask(int x){
70  assert(-V<=x&&x<=V);
71  return ask(x,rt,-V,V);
72  }
73  };

```

2.14 ds/ODT.cpp

有序区间集合

什么时候用

维护若干不相交的常值闭区间。适合区间覆盖以及依赖当前分段的统计。

调用顺序与接口

- ODT t; t.set() 清空; add(l,r,c) 只插入一个区间, 不会自动清除旧区间。
- 赋值应先 erase(l,r) 再 add(l,r,c); erase 返回被覆盖的旧段。
- extract(l,r) 返回当前区间片段, 每段字段为 l,r,c。

最小示例 (放入 main):

```

1  ODT t; t.set(); t.add(1,5,0);
2  t.erase(2,4); t.add(2,4,7);
3  auto a=t.extract(2,3);
4  assert(a.size()==1 && a[0].c==7);

```

使用边界

extract 会切开边界, 可能增加内部段数; 没有自动合并相邻同色段。一次操作可能遍历很多段, 不保证对任意输入都是对数复杂度。

完整代码

```

1  //
2  // template for ODT
3  // Chtholly is very cute!
4  // version 1.0 (Last Update Jun 16th, 2026)
5  //
6  // usage:
7  //   ODT odt; odt.set(); odt.erase(l,r); odt.add(l,r,c);
8  //   odt.extract(l,r); // assign color on [l,r]
9  //
10 struct seg{
11     int l,r,c;
12     seg():l(0),r(0),c(0){}
13     seg(int _l,int _r,int _c):l(_l),r(_r),c(_c){}
14     inline bool operator < (const seg &x) const {return l<x.l;}
15 };
16 struct ODT{
17 private:
18     std::set<seg> S;

```

```

19 void cut(int y){
20     auto it=S.upper_bound(seg(y,0,0));
21     if(it==S.begin()) return;
22     auto [l,r,c]=*(--it);
23     if(l<y&& y<=r) S.erase(it),S.insert(seg(l,y-1,c)),S.insert(seg(y,r,c));
24 }
25 public:
26 void set(){S.clear();}
27 vector<seg> erase(int l,int r){
28     // erase segments from [l,r]
29     assert(l<=r);
30     cut(l);if(r<INT_MAX)cut(r+1);
31     auto it=S.lower_bound(seg(l,0,0));
32     vector<seg> I;
33     while(1){
34         if(it==S.end()||it->l>r) break;
35         I.push_back(*it);
36         it=S.erase(it);
37     }
38     return I;
39 }
40 vector<seg> extract(int l,int r){
41     // extract segments from [l,r]
42     // before you use this function, pay attention is the TC correct?
43     assert(l<=r);
44     cut(l);if(r<INT_MAX)cut(r+1);
45     auto it=S.lower_bound(seg(l,0,0));
46     vector<seg> I;
47     while(1){
48         if(it==S.end()||it->l>r) break;
49         I.push_back(*it);
50         it++;
51     }
52     return I;
53 }
54 void add(int l,int r,int c){
55     assert(l<=r);
56     S.insert(seg(l,r,c));
57 }
58 };
59 // !!!!!!!!!!! before add segments, erase the range first!!!!!!!!!!!!

```

2.15 ds/FastSet.cpp

有界整数集合

什么时候用

用分层位集维护集合成员及前驱后继。修改和前驱后继 $O(\log_{64} V)$ ，成员判断 $O(1)$ 。

调用顺序与接口

- FastSet s; s.setN(v) 允许存储 0..v，包含 v。
- insert/erase 幂等，s[x] 判断是否存在。
- prev(x) 为最大 $\leq x$ 的元素，next(x) 为最小 $\geq x$ 的元素；没有则返回 -1。

- front/back 返回最小/最大元素; to_vector() 扫描整个值域输出元素。

最小示例（放入 main）：

```
1 FastSet s; s.setN(100);
2 s.insert(3); s.insert(100);
3 assert(s.next(3)==3 && s.prev(99)==3);
4 assert(s.next(101)==-1);
```

使用边界

值域太大时不适合使用，空间跟 V 相关而非当前元素个数。需要 src 的位运算和最值辅助函数。

完整代码

```
1 //
2 // template for fast set, insert/erase a non-negative integer in O(log_w(V))
3 // query a integer exist in O(1)
4 // version 1.2 (Last Update Jun 13th, 2026)
5 //
6 // usage:
7 //   FastSet fs; fs.setN(n); fs.insert(x); fs.erase(x);
8 //   fs[x]; fs.prev/next; needs chkmin/chkmax/first_bit/last_bit
9 //
10 struct FastSet{
11     vector<vector<ull>>> a;int V=0,B=0;
12     void setN(int _V){
13         assert(0<=_V&&_V<INT_MAX);V=_V+1;a.clear();
14         for(int n=V;;){
15             n=(n-1)/64+1;a.emplace_back(n,0);
16             if(n==1)break;
17         }
18         B=(int)a.size()-1;
19     }
20     inline bool operator [](int x)const{assert(0<=x&&x<V);return a[0][x>>6]>>(x&63)&1;}
21     vector<int> to_vector()const{
22         vector<int> I;
23         if(a.empty())return I;
24         for(int i=0;i<(int)a[0].size();i++){
25             for(ull x=a[0][i];x&=x-1)I.push_back(i*64+first_bit(x));
26         }
27         return I;
28     }
29     void insert(int x){
30         assert(0<=x&&x<V);
31         if((*this)[x]) return;
32         for(int i=0;i<=B;i++){
33             a[i][x>>6]|=(1ull<<(x&63));
34             x>>=6;
35         }
36     }
37     void erase(int x){
38         assert(0<=x&&x<V);
39         if(!(*this)[x]) return;
40         for(int i=0;i<=B;i++){
41             a[i][x>>6]&=~(1ull<<(x&63));
42             if(a[i][x>>6]) break;
43             x>>=6;
44         }
45     }
```

```

44 }
45 int prev(int x){
46     // find the first element <= x, if no, return -1
47     if(x<0||a.empty())return -1;
48     chkmin(x,V-1);
49     for(int h=0;h<=B;h++){
50         if(x<0||(x>>6)>=(int)a[h].size()) break;
51         ull f=a[h][x>>6]<<(63-(x&63));
52         if(!f){
53             x=(x>>6)-1;
54             continue;
55         }
56         x-=63-last_bit(f);
57         for(int g=h-1;g>=0;g--){
58             x<<=6;
59             x+=last_bit(a[g][x>>6]);
60         }
61         return x;
62     }
63     return -1;
64 }
65 int next(int x){
66     // find the first element >=x, if no, return -1
67     if(x>=V||a.empty())return -1;
68     chkmax(x,0);
69     for(int h=0;h<=B;h++){
70         if(x<0||(x>>6)>=(int)a[h].size()) break;
71         ull f=a[h][x>>6]>>(x&63);
72         if(!f){
73             x=(x>>6)+1;
74             continue;
75         }
76         x+=first_bit(f);
77         for(int g=h-1;g>=0;g--){
78             x<<=6;
79             x+=first_bit(a[g][x>>6]);
80         }
81         return x;
82     }
83     return -1;
84 }
85 inline int front(){return next(0);}
86 inline int back(){return prev(V-1);}
87 };

```

2.16 ds/FastODT.cpp

位集加速的区间覆盖

什么时候用

借助 FastSet 存储区间右端点。下标 $1..n$ ，适合值域固定的区间覆盖。

调用顺序与接口

- 先粘贴 FastSet，再粘贴本文件。
- `Fast_ODT t; t.setN(n)` 分配空间；先 `insert(1,n,c0)` 明确初始颜色。
- `insert(l,r,c)` 覆盖区间并返回旧的 $\{L,R,col\}$ 段；`extract(l,r)` 获取当前片段。

额外依赖（按此顺序粘贴）：`ds/FastSet.cpp`。

最小示例（放入 main）：

```
1 Fast_ODT t; t.setN(5); t.insert(1,5,0);
2 t.insert(2,4,7); auto a=t.extract(2,3);
3 assert(a.size()==1 && a[0][2]==7);
```

使用边界

`extract` 会分裂边界。不保证任意操作序列的高效性，成本仍取决于访问的段数。

完整代码

```
1 //
2 // template for Fast ODT, require Fast Set
3 // Chtholly is very cute!
4 // version 1.0 (Last Update Jun 16th, 2026)
5 //
6 // usage:
7 //   Fast_ODT fot; fot.setN(n); fot.insert(l,r,c); fot.extract(l,r);
8 //   requires FastSet.cpp
9 //
10 struct Fast_ODT{
11 private:
12   vector<int> a;int n=0;
13   FastSet S;
14   void cut(int y){
15     if(S[y]) return;
16     a[y]=a[S.next(y)];
17     S.insert(y);
18   }
19 public:
20   void setN(int _n){assert(1<=_n&&_n<INT_MAX);n=_n;a.assign(n+1,0);S.setN(n);S.insert(n);a[n]=-1;}
21
22   vector<array<int,3>> extract(int l,int r){
23     // extract segments from [l,r]
24     // before you use this function, pay attention is the TC correct?
25     assert(1<=l&&l<=r&&r<=n);
26     cut(l-1);cut(r);
27     vector<array<int,3>> I;
28     for(int i=l,j;i<=r;){
29       j=S.next(i);
30       I.push_back({i,j,a[j]});
31       i=j+1;
32     }
33   }
```

```

33     return I;
34 }
35 vector<array<int,3>> insert(int l,int r,int c){
36     // erase segments from [l,r]
37     // return vector for segs, [l,r,c]
38     assert(1<=l&&l<=r&&r<=n);
39     cut(l-1);cut(r);
40     vector<array<int,3>> I;
41     for(int i=l,j;i<=r;){
42         j=S.next(i);
43         S.erase(j);
44         I.push_back({i,j,a[j]});
45         i=j+1;
46     }
47     a[r]=c;S.insert(r);
48     return I;
49 }
50 };

```

2.17 ds/FHQ.cpp

按键或按序列位置分裂的 Treap

什么时候用

随机平衡二叉树。外部维护根编号 rt，空树为 0，普通 split/merge 期望 $O(\log n)$ 。

调用顺序与接口

- FHQ t; t.set() 清空; newnode(x) 建一个 key=x 的节点。
- split_by_key(rt,k,a,b) 分为 key≤k 与 key>k; split_by_size(rt,k,a,b) 分出前 k 个。
- rt=merge(a,b) 要求 a 所有元素应在 b 前面; 按键维护时要求 $\max(a) \leq \min(b)$ 。
- merge_it(a,b) 可合并键域交错的两树。push(rt,v) 给整个非空子树的 key 加 v。

最小示例 (放入 main):

```

1 FHQ t; t.set(); int rt=0;
2 for(int v:{2,5,8})rt=t.merge(rt,t.newnode(v));
3 int a,b; t.split_by_key(rt,5,a,b);
4 assert(t.siz[a]==2 && t.siz[b]==1);
5 rt=t.merge(a,b); assert(t.siz[rt]==3);

```

使用边界

setN(n) 只是创建 n 个孤立节点，不会把它们连成树。按键分裂依赖排序不变量；区间加可能破坏该不变量。没有现成的区间翻转或持久化。全局 rnd 可能与别的模板重名。

完整代码

```

1 //
2 // template for FHQ Treap
3 //
4 // usage:
5 //   FHQ t; t.setN(n); split_by_size / split_by_key / merge / merge_it
6 //   t.push(id, x) for lazy add on key[]
7 //
8 mt19937_64 rnd(time(0));

```

```

9 struct FHQ{
10 int tot=0;
11 vector<int> ls{0},rs{0},key{0},siz{0},tag{0};
12 vector<ll> val{0};
13 inline void push(const int &id,const int &x){
14     assert(1<=id&&id<=tot);
15     key[id]+=x;tag[id]+=x;
16 }
17 inline void pushup(const int &id){
18     siz[id]=siz[ls[id]]+siz[rs[id]]+1;
19 }
20 inline void pushdown(const int &id){
21     if(tag[id]){
22         if(ls[id]) push(ls[id],tag[id]);
23         if(rs[id]) push(rs[id],tag[id]);
24         tag[id]=0;
25     }
26 }
27 void set(){setN(0);}
28 void setN(int _n){
29     assert(0<=_n&&_n<INT_MAX);tot=_n;
30     ls.assign(tot+1,0);rs.assign(tot+1,0);tag.assign(tot+1,0);
31     key.assign(tot+1,0);siz.assign(tot+1,1);siz[0]=0;val.resize(tot+1);
32     for(int i=1;i<=tot;i++)val[i]=rnd();
33 }
34 inline int newnode(int x){
35     assert(tot<INT_MAX-1);++tot;
36     ls.push_back(0);rs.push_back(0);tag.push_back(0);siz.push_back(1);
37     key.push_back(x);val.push_back(rnd());return tot;
38 }
39 int leftmost(int x){
40     while(ls[x]) pushdown(x),x=ls[x];
41     return x;
42 }
43 int rightmost(int x){
44     while(rs[x]) pushdown(x),x=rs[x];
45     return x;
46 }
47 int merge(int p,int q){
48     if(!p||!q) return p|q;
49     if(val[p]>val[q]) {pushdown(p);rs[p]=merge(rs[p],q);pushup(p);return p;}
50     else {pushdown(q);ls[q]=merge(p,ls[q]);pushup(q);return q;}
51 }
52 void split_by_key(int rt,int k,int &p,int &q){
53     if(!rt){p=q=0;return;}
54     pushdown(rt);
55     if(k<key[rt]) q=rt,split_by_key(ls[rt],k,p,ls[rt]);
56     else p=rt,split_by_key(rs[rt],k,rs[rt],q);
57     pushup(rt);
58 }
59 void split_by_size(int rt,int k,int &p,int &q){
60     assert(0<=k&&k<=siz[rt]);
61     if(k==0){p=0;q=rt;return;}
62     if(k==siz[rt]){p=rt;q=0;return;}
63     pushdown(rt);

```

```

64  if(k<=siz[ls[rt]]) q=rt,split_by_size(ls[rt],k,p,ls[rt]),pushup(rt);
65  else p=rt,split_by_size(rs[rt],k-1-siz[ls[rt]],rs[rt],q),pushup(rt);
66  }
67  int merge_it(int p,int q){
68  // ~O(logn*logV) to merge two set (maybe intersect)
69  if(!p||!q) return p|q;
70  if(key[rightmost(p)]<=key[leftmost(q)]) return merge(p,q);
71  if(key[rightmost(q)]<=key[leftmost(p)]) return merge(q,p);
72  if(val[p]<val[q]) swap(p,q);
73  int r,s;
74  pushdown(p);
75  split_by_key(q,key[p],r,s);
76  ls[p]=merge_it(ls[p],r);
77  rs[p]=merge_it(rs[p],s);
78  pushup(p);
79  return p;
80  }
81  void alldown(int id){
82  if(!id) return;
83  pushdown(id);
84  alldown(ls[id]);
85  alldown(rs[id]);
86  }
87  };

```

2.18 ds/pq2stack.cpp

把优先队列删除转为栈撤销

什么时候用

适合只支持栈式加入/撤销，却需要删除最大权元素的离线或可撤销结构。模板只安排顺序，实际状态由调用方维护。

调用顺序与接口

- `pq2stack<T> q; q.set()` 初始化；`push(x,w)` 登记元素及删除优先级。
- 每次查询前先 `prep()`，按返回顺序把元素加入外部状态。
- `pop()` 删除最大权元素，返回外部栈需要撤销的次数；之后再次 `prep` 会给出重新加入的元素。

最小示例（放入 `main`）：

```

1 pq2stack<int> q; q.set(); vector<int> st;
2 q.push(10,1); q.push(20,2);
3 for(int x:q.prep())st.push_back(x);
4 int k=q.pop(); while(k--)st.pop_back();
5 for(int x:q.prep())st.push_back(x);
6 assert(st.size()==1 && st[0]==10);

```

使用边界

不能把 `pop` 的返回值当成被删元素。每次重加入都需要重新产生撤销记录；同权元素删除顺序不固定。单次可能重排大量元素，收益依赖整段操作序列。

完整代码

```

1 //
2 // template for pq2stack (a.k.a. fuckGold14526 trick)
3 // version 1.0 (Last Update Jun 28th, 2026)
4 //
5 // usage:
6 //   pq2stack<T> pq; pq.push(x,w); pq.prep(); pq.pop();
7 //   pop removes max weight; needs chkmin; call prep() before each query
8 //
9 template<typename T>
10 struct pq2stack{
11 private:
12 struct info{
13     T e;int w;bool tag;
14     inline bool operator < (const info &x)const{return w<x.w;}
15 };
16 int n=0,r=0,d=0;
17 vector<info> I;
18 public:
19 void set(){n=r=d=0;I.clear();}
20 vector<T> prep(){
21     vector<T> ans;
22     while(d<I.size()) ans.push_back(I[d++].e);
23     return ans;
24 }
25 void push(T x,int w){
26     I.push_back({x,w,0});
27 }
28 void imerge(int i,int j,int k){
29     if(i==j||j==k) return;
30     static vector<info> y;
31     y.resize(k-i);
32     merge(I.begin()+i,I.begin()+j,I.begin()+j,I.begin()+k,y.begin());
33     for(int t=i;t<k;t++) I[t]=y[t-i];
34 }
35 int pop(){
36     // pq2stack core
37     assert(!I.empty());
38     int tar=(r?r&-r:inf<int>),mn=I.size()-1,hd=mn;
39     for(int i=I.size()-1;i>=0;i--){
40         if(I[i].tag){
41             I[i].tag=false;
42             tar--;
43         }
44         if(I[i]<I[mn]) mn=i;
45         else hd=i;
46         if(tar==0) break;
47     }
48     sort(I.begin()+n,I.end());
49     int cut=I.size();
50     for(int i=I.size()-1;i>hd;i--) if(I[i]<I[i-1]){
51         imerge(i,cut,I.size());
52         cut=i;
53     }
54     imerge(hd,cut,I.size());

```

```

55 int ans=max(0,d-hd);
56 int mx=(int)I.size()-1;
57 for(int i=n;i<(int)I.size();i++) if(I[i].w>I[mx].w) mx=i;
58 if(mx!=(int)I.size()-1) swap(I[mx],I.back());
59 I.pop_back();
60 chkmin(d,hd);
61 n=I.size();
62 if(r==0){
63     for(int i=0;i<n;i++) I[i].tag=true;
64     r=n;
65 }else{
66     for(int i=n-(r&-r)+1;i<n;i++) I[i].tag=true;
67     r--;
68 }
69 return ans;
70 }
71 };
72 // !!!!!!!!!!!!!!!!!!!!! before each query, use pq.prep() !!!!!!!!!!!!!!!!!!!!!

```

2.19 ds/xihe_tree.cpp

析合树：返回完整结构

什么时候用

对 $1..n$ 的排列构建析合树。构建时间 $O(n \log n)$ ，包含四毛子 RMQ、辅助线段树和返回结构在内的空间均为 $O(n)$ 。

调用顺序与接口

先粘贴 `basic/ST table (linear).cpp`。 `xihe_tree tr; auto t=tr.build(a)` 接受不带哨兵的 `vector`；也可先 `setN(n)`，填写 `tr.a[1..n]`，再调用 `build()`。结果独立持有数据：`rt` 为根，`cnt` 为结点数，`son[u]` 为按位置排序的儿子；`typ[u]` 为 LEAF/INC/DEC/PRIME；`[L[u],R[u]]` 为下标闭区间，`[mn[u],mx[u]]` 为值域闭区间；`id[i]` 为第 `i` 项对应的叶子编号。

最小示例（放入 main）：

```
1 xihe_tree tr; auto t=tr.build(vector<int>{3,1,2});
2 int u=t.rt;
3 assert(t.L[u]==1 && t.R[u]==3);
4 assert(t.mn[u]==1 && t.mx[u]==3);
5 assert(t.typ[u]==xihe_tree::DEC && t.son[u].size()==2);
6 assert(t.mn[t.id[2]]==1);
```

使用边界

输入必须恰为 1..n 的排列，不接受重复值；必要时在外部离散化，但需确认题目是否只关心排名连续。返回的结点编号、下标区间、id 均从 1 开始，0 不使用。n=0 时 rt=cnt=0。INC/DEC 是儿子值域顺序递增/递减的合点，PRIME 是析点。不要假定父编号大于儿子编号；在返回结果上自行遍历、DP，构建器不含 solve。再次 build 或修改构建器不会改变已返回结果。

完整代码

```
1 //////////////////////////////////////
2 //
3 // template for Divide-Combine Tree
4 //
5 // usage:
6 //     Paste basic/ST table (linear).cpp first.
```

```

7 // xihe_tree tr; auto t=tr.build(vector<int>{3,1,2});
8 // Or setN(n), fill a[1..n], then auto t=tr.build().
9 // Input must be a permutation of 1..n. Node / interval indices are 1-based.
10 // t.rt, t.son, t.typ, t.L/R, t.mn/mx, t.id; no problem-specific solver.
11 // Build O(n log n), space O(n). Children are ordered by position.
12 //
13 //////////////////////////////////////
14 struct xihe_tree{
15     enum Type{LEAF,INC,DEC,PRIME};
16     struct result{
17         int n=0,rt=0,cnt=0;
18         vector<vector<int>> son;
19         vector<Type> typ;
20         vector<int> L,R,mn,mx,id;
21     };
22     int n=0;vector<int> a;
23     void set(){n=0;a.clear();}
24     void setN(int _n){assert(0<=_n&&_n<=INT_MAX/4);n=_n;a.assign(n+1,0);}
25     private:
26     struct segt{
27         int n;vector<int> val,laz;
28         segt(int _n):n(_n),val(4*n),laz(4*n){}
29         void push(int id,int x){val[id]+=x;laz[id]+=x;}
30         void pushdown(int id){if(laz[id]){push(id<<1,laz[id]);push(id<<1|1,laz[id]);laz[id]=0;}}
31         void add(int ql,int qr,int x,int id,int l,int r){
32             if(ql<=l&&r<=qr){push(id,x);return;}
33             int mid=(l+r)>>1;pushdown(id);
34             if(ql<=mid)add(ql,qr,x,id<<1,l,mid);
35             if(mid<qr)add(ql,qr,x,id<<1|1,mid+1,r);
36             val[id]=min(val[id<<1],val[id<<1|1]);
37         }
38         void add(int l,int r,int x){add(l,r,x,1,1,n);}
39         int first(){
40             int id=1,l=1,r=n;assert(val[1]<=0);
41             while(l<r){
42                 int mid=(l+r)>>1;pushdown(id);
43                 if(val[id<<1]<=0){id<<=1;r=mid;}
44                 else{id=id<<1|1;l=mid+1;}
45             }
46             return l;
47         }
48     };
49     public:
50     result build()const{
51         assert(a.size()==(size_t)n+1||!n);
52         result res;res.n=n;
53         auto &son=res.son;auto &typ=res.typ;auto &L=res.L;auto &R=res.R;auto &id=res.id;
54         int &cnt=res.cnt;
55         son.resize(2*n+1);typ.resize(2*n+1,LEAF);L.resize(2*n+1);R.resize(2*n+1);id.resize(n+1);
56         if(!n){res.mn.resize(1);res.mx.resize(1);return res;}
57         vector<char> vis(n+1);
58         for(int i=1;i<=n;i++){assert(1<=a[i]&&a[i]<=n&&!vis[a[i]]);vis[a[i]]=true;}
59         vector<int> v(a.begin()+1,a.end());
60         Linear_RMQ<int> mn;Linear_RMQ<int,true> mx;mn.build(v);mx.build(v);
61         auto judge=[&](int l,int r){return mx.ask(l-1,r-1)-mn.ask(l-1,r-1)==r-l;};

```

```

62  vector<int> M(2*n+1),st1(n+1),st2(n+1),st(n+1);
63  int tp1=0,tp2=0,tp=0;segt T(n);
64  for(int i=1;i<=n;i++){
65      while(tp1&&a[i]<=a[st1[tp1]]) T.add(st1[tp1-1]+1,st1[tp1],a[st1[tp1]]),tp1--;
66      while(tp2&&a[i]>=a[st2[tp2]]) T.add(st2[tp2-1]+1,st2[tp2],-a[st2[tp2]]),tp2--;
67      T.add(st1[tp1]+1,i,-a[i]);
68      T.add(st2[tp2]+1,i,a[i]);
69      st1[++tp1]=st2[++tp2]=i;
70      id[i]=++cnt;L[cnt]=R[cnt]=M[cnt]=i;typ[cnt]=LEAF;
71      int le=T.first(),now=cnt;
72      while(tp&&L[st[tp]]>=le){
73          if((typ[st[tp]]==INC|typ[st[tp]]==DEC)&&judge(M[st[tp]],i)){
74              // 当儿子
75              R[st[tp]]=i;M[st[tp]]=L[now];son[st[tp]].push_back(now);now=st[tp--];
76          }else if(judge(L[st[tp]],i)){
77              // 新合点
78              ++cnt;
79              if(a[L[st[tp]]]<a[i]) typ[cnt]=INC;
80              else typ[cnt]=DEC;
81              L[cnt]=L[st[tp]];R[cnt]=i;M[cnt]=L[now];
82              son[cnt].push_back(st[tp]);
83              son[cnt].push_back(now);
84              now=cnt;tp--;
85          }else{
86              // 新析点
87              ++cnt;typ[cnt]=PRIME;
88              son[cnt].push_back(now);
89              do son[cnt].push_back(st[tp--]);
90              while(tp&&!judge(L[st[tp]],i));
91              L[cnt]=(tp?L[st[tp]]:L[now]);R[cnt]=i;
92              if(tp) son[cnt].push_back(st[tp--]);
93              reverse(son[cnt].begin(),son[cnt].end());
94              now=cnt;
95          }
96      }
97      st[++tp]=now;
98      T.add(1,i,-1);
99  }
100  assert(tp==1);res.rt=st[1];
101  son.resize(cnt+1);typ.resize(cnt+1);L.resize(cnt+1);R.resize(cnt+1);
102  res.mn.resize(cnt+1);res.mx.resize(cnt+1);
103  for(int i=1;i<=cnt;i++){res.mn[i]=mn.ask(L[i]-1,R[i]-1);res.mx[i]=mx.ask(L[i]-1,R[i]-1);}
104  return res;
105 }
106 result build(const vector<int> &v){
107     assert(v.size()<=(size_t)INT_MAX/4);setN(v.size());copy(v.begin(),v.end(),a.begin()+1);return build();
108 }
109 };
110 // end for ds/xihe_tree.cpp
111 //////////////////////////////////

```

3 图论

3.1 graph/flow.cpp

Dinic 最大流

什么时候用

有向网络最大流，容量是非负 ll 。一般图最坏 $O(V^2 E)$ ，具体模型常有更好表现。

调用顺序与接口

- `Flow_Graph g; g.setN(n); g.setST(s,t)` 初始化，点编号 $1..n$ ，源汇必须不同。
- `id=g.add_edge(u,v,cap)` 只添加一条有向边，自动生成反向残量边。
- `max_flow()` 返回累计最大流值；`edge_flow(id)` 查看返回的正向边上的流量。

最小示例（放入 main）：

```
1 Flow_Graph g; g.setN(3); g.setST(1,3);
2 int id=g.add_edge(1,2,5); g.add_edge(2,3,3);
3 assert(g.max_flow()==3 && g.edge_flow(id)==3);
4 assert(g.max_flow()==3);
```

使用边界

重复求流返回总流量，不是新增流量；添加边后继续增广。`setN` 清空图，`new_node` 保留旧图添加点。无向容量边需根据建模显式加边。流量总和应在 ll 范围内。

完整代码

```
1 //
2 // template for max flow (dinic)
3 // last update Jun 23rd, 2026
4 //
5 // usage:
6 //   Flow_Graph g; g.setN(n); g.setST(S,T); g.add_edge(u,v,cap);
7 //   ll f = g.max_flow(); g.edge_flow(id);
8 //   setN clears the graph; new_node keeps existing edges
9 //
10 struct Flow_Graph{
11 private:
12 static const ll inf=2.02e18;
13 struct edge{int v,nxt;ll w;};
14 vector<edge> e=vector<edge>(2);bool is_flowед=false;
15 int n=0,m=1,S=0,T=0;ll ans=0;
16 vector<int> lst{0},level{0},cur{0};
17 void add(int u,int v,ll w){
18     assert(m<INT_MAX-1);e.push_back({v,lst[u],w});lst[u]=++m;
19 }
20 bool BFS(){
21     for(int i=0;i<=n;i++) level[i]=-1;
22     queue<int> q;q.push(S);level[S]=0;
23     while(!q.empty()){
24         int u=q.front();q.pop();
25         if(u==T) return true;
26         for(int id=lst[u];id=id=e[id].nxt) if(e[id].w&&level[e[id].v]==-1){
27             level[e[id].v]=level[u]+1;
28             q.push(e[id].v);
29         }
```

```

30     }
31     return false;
32 }
33 ll DFS(int u,ll flow){
34     if(u==S) return flow;
35     ll now=0;
36     for(int &i=cur[u];i;i=e[i].nxt) if(level[e[i].v]==level[u]-1&&e[i^1].w){
37         ll fo=DFS(e[i].v,min(e[i^1].w,flow-now));
38         e[i^1].w-=fo;e[i].w+=fo;now+=fo;
39         if(now==flow) break;
40     }
41     return now;
42 }
43 public:
44 void set(){
45     n=S=T=0;m=1;ans=0;is_flowd=false;e.assign(2,{0,0,0});
46     lst.assign(1,0);level.assign(1,0);cur.assign(1,0);
47 }
48 void setN(int _n){
49     assert(0<=_n&&_n<INT_MAX);set();n=_n;
50     lst.assign(n+1,0);level.assign(n+1,0);cur.assign(n+1,0);
51 }
52 void setST(int _S,int _T){
53     assert(1<=min(_S,_T)&&max(_S,_T)<INT_MAX&&_S!=_T);
54     if(S!=_S||T!=_T){
55         for(int i=2;i<=m;i+=2)e[i].w+=e[i^1].w,e[i^1].w=0;
56         ans=0;
57     }
58     S=_S;T=_T;n=max({n,S,T});
59     lst.resize(n+1);level.resize(n+1);cur.resize(n+1);is_flowd=false;
60 }
61 int new_node(){
62     assert(n<INT_MAX-1);++n;lst.push_back(0);level.push_back(0);cur.push_back(0);
63     is_flowd=false;return n;
64 }
65 ll edge_flow(int id){
66     assert(2<=id&&id<=m&&id%2==0);
67     return e[id^1].w;
68 }
69 int add_edge(int u,int v,ll w){
70     // return forward edge id
71     assert(1<=min(u,v)&&max(u,v)<=n&&0<=w);
72     assert(m&1);
73     is_flowd=false;add(u,v,w);add(v,u,0);return m-1;
74 }
75 ll max_flow(){
76     assert(1<=min(S,T)&&max(S,T)<=n&&S!=T);
77     if(is_flowd) return ans;
78     is_flowd=true;
79     while(BFS()){
80         for(int i=1;i<=n;i++) cur[i]=lst[i];
81         ll f=DFS(T,inf);assert(ans<=LLONG_MAX-f);ans+=f;
82     }
83     return ans;
84 }

```

```
85 };
```

3.2 graph/mcmf.cpp

最小费用最大流

什么时候用

先求最大可行流量，再在最大流中最小化费用。有负费用边时先用 SPFA 求势能；之后 Dijkstra 求最短路，在零约化费用子图上用 Dinic 批量增广，适合大量路径费用相同的网络。DFS 使用显式数组，不依赖递归栈。

调用顺序与接口

- `Cost_Flow_Graph g; g.setN(n); g.setST(s,t)` 初始化。
- `add_edge(u,v,cap,cost)` 添加有向边并返回正向边编号。
- `min_cost_flow()` 返回 {总流量, 总费用}; `edge_flow(id)` 查询流量。

最小示例（放入 main）:

```
1 Cost_Flow_Graph g; g.setN(3); g.setST(1,3);
2 int id=g.add_edge(1,2,2,-1); g.add_edge(2,3,2,3);
3 auto [f,c]=g.min_cost_flow();
4 assert(f==2 && c==4 && g.edge_flow(id)==2);
```

使用边界

允许负费用边，但整个输入图不能有负费用环。没有指定流量参数；要限制流量，增加一个容量为上限的新源边。更改图后重新从原始容量计算；不改图则返回缓存。容量、总流量及计算过程中的总费用须能用 `ll` 表示，距离与势能使用 `__int128`。相同最短路费用的增广会合并，但费用层数很多时仍可能较慢。

完整代码

```
1 //
2 // template for min cost max flow (potential + dijkstra + blocking flow)
3 //
4 // usage:
5 //   Cost_Flow_Graph g; g.setN(n); g.setST(S,T); g.add_edge(u,v,cap,cost);
6 //   auto [flow,cost]=g.min_cost_flow(); g.edge_flow(id);
7 //   negative costs allowed, input graph must have no negative cost cycle
8 //   setN clears the graph; editing the graph makes the next solve start over
9 //   flow and cost must fit ll; distances use __int128
10 //
11 struct Cost_Flow_Graph{
12 private:
13 static constexpr __int128 inf=(__int128)1<<120;
14 struct edge{int v,nxt;ll w,c;};
15 vector<edge> e=vector<edge>(2);bool is_flow=0;
16 int n=0,m=1,S=0,T=0;ll ans=0,cost=0;
17 vector<int> lst{0},pre{0},dep,cur,q;
18 vector<ll> lim,rem;
19 vector<__int128> h{0},dis{0};
20 void add(int u,int v,ll w,ll c){
21   assert(m<INT_MAX-1);e.push_back({v,lst[u],w,c});lst[u]=++m;
22 }
23 void SPFA(){
24   fill(h.begin(),h.end(),0);
25   bool neg=false;
```

```

26   for(int i=2;i<=m;i++)if(e[i].w&&e[i].c<0){neg=true;break;}
27   if(!neg)return;
28   vector<int> cnt(n+1);vector<char> inq(n+1,true);
29   queue<int> q;for(int i=1;i<=n;i++)q.push(i);
30   while(!q.empty()){
31       int u=q.front();q.pop();inq[u]=false;
32       for(int id=lst[u];id;id=e[id].nxt)if(e[id].w){
33           int v=e[id].v;
34           if(h[v]<=h[u]+e[id].c)continue;
35           h[v]=h[u]+e[id].c;cnt[v]=cnt[u]+1;
36           assert(cnt[v]<n);
37           if(!inq[v])inq[v]=true,q.push(v);
38       }
39   }
40 }
41 bool Dijkstra(){
42     fill(dis.begin(),dis.end(),inf);dis[S]=0;
43     priority_queue<pair<__int128,int>,vector<pair<__int128,int>>,greater<pair<__int128,int>>> q;
44     q.push({0,S});
45     while(!q.empty()){
46         auto [d,u]=q.top();q.pop();
47         if(d!=dis[u])continue;
48         if(u==T)break;
49         for(int id=lst[u];id;id=e[id].nxt)if(e[id].w){
50             int v=e[id].v;__int128 w=e[id].c+h[u]-h[v];
51             assert(w>=0);
52             if(dis[v]<=d+w)continue;
53             dis[v]=d+w;q.push({dis[v],v});
54         }
55     }
56     if(dis[T]==inf)return false;
57     for(int i=1;i<=n;i++)h[i]+=min(dis[i],dis[T]);
58     return true;
59 }
60 bool BFS(){
61     fill(dep.begin(),dep.end(),-1);dep[S]=0;
62     int l=0,r=0;q[r++]=S;
63     while(l<r){
64         int u=q[l++];
65         for(int id=lst[u];id;id=e[id].nxt){
66             int v=e[id].v;
67             if(!e[id].w||dep[v]!=-1||h[v]!=h[u]+e[id].c)continue;
68             dep[v]=dep[u]+1;q[r++]=v;
69         }
70     }
71     return dep[T]!=-1;
72 }
73 ll DFS(){
74     // iterative DFS: rem[u] is the part of lim[u] not sent yet
75     int u=S;lim[u]=rem[u]=LLONG_MAX;
76     while(true){
77         if(u==T)rem[u]=0;
78         int &id=cur[u];
79         while(rem[u]&&id){
80             int v=e[id].v;

```

```

81         if(e[id].w&&dep[v]==dep[u]+1&&h[v]==h[u]+e[id].c)break;
82         id=e[id].nxt;
83     }
84     if(!rem[u]||!id){
85         ll f=lim[u]-rem[u];
86         if(u==S)return f;
87         if(rem[u])dep[u]--;
88         int x=pre[u];u=e[x^1].v;
89         e[x].w-=f;e[x^1].w+=f;rem[u]-=f;
90         if(rem[u])cur[u]=e[x].nxt;
91     }else{
92         int v=e[id].v;pre[v]=id;
93         lim[v]=rem[v]=min(rem[u],e[id].w);u=v;
94     }
95 }
96 }
97 public:
98 void set(){
99     n=S=T=0;m=1;ans=cost=0;is_flow=false;e.assign(2,{0,0,0,0});
100    dep.clear();cur.clear();q.clear();lim.clear();rem.clear();
101    lst.assign(1,0);pre.assign(1,0);h.assign(1,0);dis.assign(1,0);
102 }
103 void setN(int _n){
104     assert(0<=_n&&_n<INT_MAX);set();n=_n;
105     lst.assign(n+1,0);pre.assign(n+1,0);h.assign(n+1,0);dis.assign(n+1,0);
106 }
107 void setST(int _S,int _T){
108     assert(1<=min(_S,_T)&&max(_S,_T)<INT_MAX&&_S!=_T);
109     S=_S;T=_T;n=max({n,S,T});
110     lst.resize(n+1);pre.resize(n+1);h.resize(n+1);dis.resize(n+1);is_flow=false;
111 }
112 int new_node(){
113     assert(n<INT_MAX-1);++n;lst.push_back(0);pre.push_back(0);h.push_back(0);dis.push_back(0);
114     is_flow=false;return n;
115 }
116 ll edge_flow(int id){
117     assert(2<=id&&id<=m&&id%2==0);
118     return e[id^1].w;
119 }
120 int add_edge(int u,int v,ll w,ll c){
121     // return forward edge id
122     assert(1<=min(u,v)&&max(u,v)<=n&&w>=0&&c!=LLONG_MIN);
123     assert(m&1);
124     is_flow=false;add(u,v,w,c);add(v,u,0,-c);return m-1;
125 }
126 pair<ll,ll> min_cost_flow(){
127     assert(1<=min(S,T)&&max(S,T)<=n&&S!=T);
128     if(is_flow)return {ans,cost};
129     for(int i=2;i<=m;i+=2)e[i].w+=e[i^1].w,e[i^1].w=0;
130     ans=cost=0;SPFA();
131     dep.resize(n+1);cur.resize(n+1);q.resize(n+1);
132     lim.resize(n+1);rem.resize(n+1);
133     while(Dijkstra())while(BFS()){
134         cur=lst;ll f=DFS();assert(f>0);
135         __int128 c=h[T]-h[S];

```

```

136     assert(ans<=LLONG_MAX-f);
137     assert(c>=((__int128)LLONG_MIN-cost)/f&&c<=((__int128)LLONG_MAX-cost)/f);
138     ans+=f;cost=(ll)(cost+c*f);
139 }
140 is_flowd=true;return {ans,cost};
141 }
142 };

```

3.3 graph/sssp.cpp

非负权单源最短路

什么时候用

堆优化 Dijkstra, 有向图, 复杂度 $O((n+m) \log n)$ 。

调用顺序与接口

- graph_SSSP g; g.setN(n); g.setS(s) 设置图和源点。
- add_edge(u,v,w) 要求 $w \geq 0$; 无向图要加两个方向。
- SSSP() 返回长度 $n+1$ 的距离数组, 0 号不用, 不可达为 10^{18} 。

最小示例 (放入 main):

```

1 graph_SSSP g; g.setN(3); g.setS(1);
2 g.add_edge(1,2,4); g.add_edge(2,3,6);
3 auto d=g.SSSP(); assert(d[3]==10);

```

使用边界

有限最短路必须小于不可达哨兵 10^{18} , 边权相加不能溢出。不支持负权; 不会返回具体路径。

完整代码

```

1 //
2 // template for Single-source shortest path
3 // last update Jun 23rd, 2026
4 //
5 // usage:
6 //   graph_SSSP G; G.setN(n); G.setS(s); G.add_edge(u,v,w);
7 //   auto d = G.SSSP(); // unreachable 1e18, non-negative weights
8 //
9 struct graph_SSSP{
10 private:
11 int n=0,s=0;
12 vector<vector<pair<int,ll>>> edg;
13 public:
14 void setN(int N){assert(0<=N&&N<INT_MAX);n=N;s=0;edg.assign(n+1,{});}
15 void setS(int S){s=S;assert(1<=s&&s<=n);}
16 void add_edge(int u,int v,ll w){
17     assert(1<=min(u,v)&&max(u,v)<=n&&0<=w);
18     edg[u].push_back(make_pair(v,w));
19 }
20 vector<ll> SSSP(){
21     // return a vector of length n+1 --- ans[i] shortest path between s and i
22     // if no path, ans[i] will be 1e18
23     assert(1<=s&&s<=n);
24     priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>> q;

```

```

25  vector<ll> d(n+1,1e18);
26  d[s]=0;q.push(make_pair(0,s));
27  while(!q.empty()){
28      auto [dd,u]=q.top();q.pop();
29      if(dd!=d[u]) continue;
30      for(auto [v,w]:edg[u] if(w<d[v]-d[u])){
31          d[v]=d[u]+w;
32          q.push(make_pair(d[v],v));
33      }
34  }
35  return d;
36 }
37 };

```

3.4 graph/2-sat.cpp

布尔约束求解

什么时候用

求满足所有二元或子句的一组赋值，时间与空间 $O(n+m)$ 。

调用顺序与接口

- `two_sat s; s.setN(n)` 初始化 n 个布尔变量，下标 $1..n$ 。
- `add(x,a,y,b)` 表示 (变量 x 等于 a) 或 (变量 y 等于 b)， a,b 为 $0/1$ 。
- `solve()` 无解返回空 vector，否则 `ans[1..n]` 为可行赋值。

最小示例（放入 main）：

```

1  two_sat s; s.setN(2);
2  s.add(1,1,1,1); // force x1=true
3  s.add(1,0,2,1); // x1 implies x2
4  auto ans=s.solve();
5  assert(!ans.empty() && ans[1] && ans[2]);

```

使用边界

只有“或”子句接口；例如强制 $x=a$ 可重复同一文字。setN 清空约束。不要把 add 当作直接添加两条蕴含边。

完整代码

```

1  //
2  // template for 2-SAT
3  //
4  // usage:
5  //   two_sat sat; sat.set(); sat.setN(n); sat.add(x1,o1,x2,o2);
6  //   auto ans = sat.solve(); // empty if unsat; var i: 2i-1=T, 2i=F
7  //   setN clears old clauses
8  //
9  struct two_sat{
10 int tot=0,scnt=0,top=0,n=0;
11 vector<int> low,dfn,stk,bel;
12 vector<vector<int>> edg;vector<char> inq;
13 void set(){setN(0);}
14 void setN(int _n){
15     // 1-index, 2i-1 true 2i false
16     assert(0<=_n&&_n<(INT_MAX-2)/2);n=_n;

```

```

17 low.assign(2*n+2,0);dfn.assign(2*n+2,0);stk.assign(2*n+2,0);
18 bel.assign(2*n+2,0);inq.assign(2*n+2,0);edg.assign(2*n+2,{});
19 tot=scnt=top=0;
20 }
21 void tarjan(int u){
22     vector<pair<int,int>> q{{u,0}};
23     low[u]=dfn[u]=++tot;stk[++top]=u;inq[u]=true;
24     while(!q.empty()){
25         auto &[x,i]=q.back();
26         if(i<(int)edg[x].size()){
27             int v=edg[x][i++];
28             if(!dfn[v]){
29                 low[v]=dfn[v]=++tot;stk[++top]=v;inq[v]=true;q.push_back({v,0});
30             }else if(inq[v])low[x]=min(low[x],dfn[v]);
31         }else{
32             int v=x;q.pop_back();
33             if(!q.empty()){int p=q.back().first;low[p]=min(low[p],low[v]);}
34             if(low[v]==dfn[v]){
35                 scnt++;int y;
36                 do{y=stk[top--];inq[y]=false;bel[y]=scnt;}while(y!=v);
37             }
38         }
39     }
40 }
41 vector<int> solve(){
42     // if impossible return empty
43     // else return a vector with length n+1, the a[i] - any valid answer
44     tot=scnt=top=0;
45     for(int i=1;i<=2*n;i++) low[i]=dfn[i]=bel[i]=inq[i]=0;
46     for(int i=1;i<=2*n;i++) if(!dfn[i]) tarjan(i);
47     for(int i=1;i<=n;i++) if(bel[2*i-1]==bel[2*i]) return {};
48     vector<int> ans(n+1);
49     for(int i=1;i<=n;i++) ans[i]=bel[2*i-1]<bel[2*i];
50     return ans;
51 }
52 void add(int x1,int o1,int x2,int o2){
53     // x1 = o1 or x2 = o2
54     assert(1<=min(x1,x2)&&max(x1,x2)<=n&&0<=min(o1,o2)&&max(o1,o2)<=1);
55     edg[2*x1-(!o1)].push_back(2*x2-o2);
56     edg[2*x2-(!o2)].push_back(2*x1-o1);
57 }
58 };

```

3.5 graph/block-cut-tree.cpp

点双连通分量与圆方树

什么时候用

无向图的点双分解，支持非连通图和重边，不支持自环。构建时间 $O(n+m)$ 。

调用顺序与接口

- block_cut g; g.setN(n) 后 add_edge, 再 build。
- 原点仍为 1..n, 方点为 n+1..scc, tr 是圆方森林邻接表。

- 第 k 个方点 $n+k$ 的原图边存在 $ec[k]$ 中。ff、dep 是森林上的父亲和深度。

最小示例（放入 main）：

```
1 block_cut g; g.setN(3);
2 g.add_edge(1,2); g.add_edge(2,3); g.build();
3 assert(g.scc==5 && g.tr[2].size()==2);
```

使用边界

scc 是最终最大节点编号，不是分量个数，方点数应为 $scc-n$ 。割点可以属于多个点双，不能像 SCC 一样给每个原点分配唯一块号。递归深度可能达到 $O(n)$ 。

完整代码

```
1 //
2 // template for Block Cut Tree
3 // use add_edge to add (u,v), tr[n+i] for vertex in i-th component
4 // ec[i] for edge in i-th component
5 // version 1.0 (Last Update Jun 17th, 2026)
6 //
7 // usage:
8 //   block_cut bc; bc.setN(n); bc.add_edge(u,v); bc.build();
9 //   vertex u -> tree u; BCC i -> tree n+i; bc.ec[i] edges
10 //
11 struct block_cut{
12     int n=0,tot=0,scc=0,top=0;
13     vector<char> inq;
14     vector<int> low,dfn,stk,dep,ff;
15     vector<vector<int>> edg,tr;
16     vector<vector<array<int,2>>> ec;
17     void setN(int _n){
18         assert(0<=_n&&_n<(INT_MAX-2)/2);n=_n;
19         low.assign(n+2,0);dfn.assign(n+2,0);stk.assign(n+2,0);inq.assign(n+2,0);
20         edg.assign(n+2,{});tr.assign(2*n+2,{});ec.assign(n+2,{});
21         dep.assign(2*n+2,0);ff.assign(2*n+2,0);tot=scc=top=0;
22     }
23     void add_edge(int u,int v){
24         assert(1<=min(u,v)&&max(u,v)<=n&&u!=v);
25         edg[u].push_back(v);
26         edg[v].push_back(u);
27     }
28     void tarjan(int u,int fa=0){
29         // frame: vertex, parent, next edge, skipped parent edge
30         vector<array<int,4>> q{{u,fa,0,0}};
31         low[u]=dfn[u]=++tot;stk[++top]=u;inq[u]=true;
32         while(!q.empty()){
33             auto &[x,p,i,skip]=q.back();
34             if(i<(int)edg[x].size()){
35                 int v=edg[x][i++];
36                 if(v==p&&!skip){skip=1;continue;}
37                 if(!dfn[v]){
38                     low[v]=dfn[v]=++tot;stk[++top]=v;inq[v]=true;q.push_back({v,x,0,0});
39                 }else if(inq[v])low[x]=min(low[x],dfn[v]);
40             }else{
41                 int v=x,parent=p;q.pop_back();
42                 if(q.empty())continue;
43                 low[parent]=min(low[parent],low[v]);
```

```

44     if(low[v]>=dfn[parent]){
45         ++scc;tr[scc].push_back(parent);tr[parent].push_back(scc);int y;
46         do{y=stk[top--];inq[y]=false;tr[y].push_back(scc);tr[scc].push_back(y);}while(y!=v);
47     }
48 }
49 }
50 }
51 void dfs(int u,int fa){
52     vector<int> q{u};ff[u]=fa;dep[u]=dep[fa]+1;
53     for(int i=0;i<(int)q.size();i++){
54         int x=q[i];
55         for(int v:tr[x])if(v!=ff[x]){ff[v]=x;dep[v]=dep[x]+1;q.push_back(v);}
56     }
57 }
58 void build(){
59     fill(dfn.begin(),dfn.end(),0);fill(inq.begin(),inq.end(),0);
60     fill(dep.begin(),dep.end(),0);fill(ff.begin(),ff.end(),0);
61     for(auto &v:tr)v.clear();for(auto &v:ec)v.clear();
62     tot=top=0;scc=n;
63     for(int i=1;i<=n;i++)if(!dfn[i]){
64         tarjan(i);dfs(i,0);inq[i]=false;top=0;
65     }
66     for(int i=1;i<=n;i++)for(int j:edg[i])if(i<j){
67         int x=ff[i]==ff[j]?ff[i]:(dep[i]>dep[j]?ff[i]:ff[j]);
68         assert(n<x&&x<=scc);ec[x-n].push_back({i,j});
69     }
70 }
71 };

```

3.6 graph/Edge BCC.cpp

边双连通分量及桥森林

什么时候用

无向图删除桥后缩点，支持重边和非连通图。时间 $O(n+m)$ 。

调用顺序与接口

- `bi_edge g; g.setN(n)` 后加边。
- `auto e=g.build()` 返回连接不同分量的边表；每端是分量编号。
- `bel[u]` 表示点所属分量，分量编号 `1..scc`。

最小示例（放入 main）：

```

1 bi_edge g; g.setN(3);
2 g.add_edge(1,2); g.add_edge(1,2); g.add_edge(2,3);
3 auto e=g.build();
4 assert(g.bel[1]==g.bel[2] && g.bel[2]!=g.bel[3]);
5 assert(e.size()==1);

```

使用边界

桥森林的顶点是分量，不是原图点。与圆方树分别处理边双、点双，不能互换。

完整代码

```

1 //
2 // template for Edge BCC (edge-biconnected components)
3 //
4 // usage:
5 //   bi_edge b; b.setN(n); b.add_edge(u,v); auto E = b.build();
6 //   bel[u] is EBCC id; E lists edges between different blocks
7 //
8 struct bi_edge{
9   int n=0,top=0,tot=0,scnt=0;
10  vector<vector<int>> edg;
11  vector<int> low,dfn,stk,bel;
12  void setN(int _n){
13    assert(0<=_n&&_n<INT_MAX);n=_n;edg.assign(n+1,{});
14    low.assign(n+1,0);dfn.assign(n+1,0);stk.assign(n+1,0);bel.assign(n+1,0);
15    top=tot=scnt=0;
16  }
17  void add_edge(int u,int v){
18    assert(1<=min(u,v)&&max(u,v)<=n);
19    edg[u].push_back(v);
20    edg[v].push_back(u);
21  }
22  vector<array<int,2>> build(){
23    fill(dfn.begin(),dfn.end(),0);top=tot=scnt=0;
24    for(int u=1;u<=n;u++)if(!dfn[u]){
25      vector<array<int,4>> q{{u,0,0,0}};
26      low[u]=dfn[u]=++tot;stk[++top]=u;
27      while(!q.empty()){
28        auto &[x,p,i,skip]=q.back();
29        if(i<(int)edg[x].size()){
30          int v=edg[x][i++];
31          if(v==p&&!skip){skip=1;continue;}
32          if(!dfn[v]){low[v]=dfn[v]=++tot;stk[++top]=v;q.push_back({v,x,0,0});}
33          else low[x]=min(low[x],dfn[v]);
34        }else{
35          int v=x,parent=p;q.pop_back();
36          if(parent)low[parent]=min(low[parent],low[v]);
37          if(low[v]==dfn[v]){
38            scnt++;int y;do{y=stk[top--];bel[y]=scnt;}while(y!=v);
39          }
40        }
41      }
42    }
43    vector<array<int,2>> E;
44    for(int i=1;i<=n;i++) for(int j:edg[i]) if(bel[i]<bel[j])
45      E.push_back({bel[i],bel[j]});
46    return E;
47  }
48 };

```

3.7 graph/blossom.cpp

无权一般图最大匹配

什么时候用

带花树，允许奇环，返回匹配边数，最坏 $O(n^3)$ 。

调用顺序与接口

- blossom g; g.setN(n) 后 add_edge 添加无向边。
- solve() 返回最大匹配的边数；match[u] 为匹配点，未匹配为 0。
- solve()*2==n 判定存在完全匹配。

最小示例（放入 main）：

```
1 blossom g; g.setN(4);
2 g.add_edge(1,2); g.add_edge(2,3);
3 g.add_edge(3,1); g.add_edge(3,4);
4 assert(g.solve()==2);
5 assert(g.match[g.match[1]]==1);
```

使用边界

这是无权匹配，不会最小化或最大化边权。点数为奇数时不可能完全匹配。重复 solve 会重新求解。

完整代码

```
1 //
2 // template for General Graph Matching (blossom),  $O(n^3)$ 
3 //
4 // usage:
5 //   blossom g; g.setN(n); g.add_edge(u,v); int cnt=g.solve();
6 //   cnt*2==n if perfect; match[u]=0 if unmatched; 1-index
7 //
8 struct blossom{
9   int n=0;
10  vector<vector<int>> edg;
11  vector<int> match,pre,be1,q;
12  vector<char> inq,vis,flower;
13  void setN(int _n){
14    assert(0<=_n&&_n<INT_MAX);n=_n;
15    edg.assign(n+1,{});match.assign(n+1,0);
16    pre.resize(n+1);be1.resize(n+1);q.resize(n+1);
17    inq.resize(n+1);vis.resize(n+1);flower.resize(n+1);
18  }
19  void add_edge(int u,int v){
20    assert(1<=min(u,v)&&max(u,v)<=n);
21    if(u==v)return;
22    edg[u].push_back(v);edg[v].push_back(u);
23  }
24  int lca(int u,int v){
25    fill(vis.begin(),vis.end(),false);
26    while(1){
27      u=be1[u];vis[u]=true;
28      if(!match[u])break;
29      u=pre[match[u]];
30    }
31    while(!vis[be1[v]])v=pre[match[be1[v]]];
```

```

32     return bel[v];
33 }
34 void shrink(int u,int v,int w){
35     while(bel[u]!=w){
36         flower[bel[u]]=flower[bel[match[u]]]=true;
37         pre[u]=v;v=match[u];u=pre[v];
38     }
39 }
40 bool BFS(int s){
41     fill(inq.begin(),inq.end(),false);
42     fill(pre.begin(),pre.end(),0);
43     iota(bel.begin(),bel.end(),0);
44     int h=0,t=0;q[t++]=s;inq[s]=true;
45     while(h<t){
46         int u=q[h++];
47         for(int v:edg[u]){
48             if(bel[u]==bel[v]||match[u]==v)continue;
49             if(v==s||(match[v]&&pre[match[v]])){
50                 int w=lca(u,v);
51                 fill(flower.begin(),flower.end(),false);
52                 shrink(u,v,w);shrink(v,u,w);
53                 for(int i=1;i<=n;i++)if(flower[bel[i]]){
54                     bel[i]=w;
55                     if(!inq[i])inq[i]=true,q[t++]=i;
56                 }
57             }else if(!pre[v]){
58                 pre[v]=u;
59                 if(!match[v]){
60                     while(v){
61                         int x=pre[v],y=match[x];
62                         match[v]=x;match[x]=v;v=y;
63                     }
64                     return true;
65                 }
66                 int x=match[v];inq[x]=true;q[t++]=x;
67             }
68         }
69     }
70     return false;
71 }
72 int solve(){
73     fill(match.begin(),match.end(),0);
74     int ans=0;
75     for(int i=1;i<=n;i++)if(!match[i])ans+=BFS(i);
76     return ans;
77 }
78 };

```

3.8 graph/bipartite-matching.cpp

二分图最大匹配

什么时候用

给定左右两部，多次加边后求最大基数匹配，返回匹配的点对。先贪心匹配，再用多源 BFS 记录起点与前驱，遇到未匹配右点就立即增广；整个过程不递归。

调用顺序与接口

- `Bipartite_Matching g; g.setN(n,m)`，左部下标 $1..n$ ，右部下标 $1..m$ ；两部编号互不影响。
- `g.add_edge(u,v)` 加入左点 u 到右点 v 的边；允许重边。
- `auto a=g.solve()` 返回 `vector<pair<int,int>>`，每个二元组 (u,v) 是一对匹配的左点和右点，按左点递增；`a.size()` 就是最大匹配数。
- 求解后的 `matchL[u]`、`matchR[v]` 是对应另一部的点，未匹配为 0。`set()` 清空，`setN` 清空并设置两部大小。

最小示例（放入 main）：

```
1 Bipartite_Matching g; g.setN(3,3);
2 g.add_edge(1,1); g.add_edge(2,1);
3 g.add_edge(2,2); g.add_edge(3,2);
4 auto a=g.solve(); assert(a.size()==2);
5 g.add_edge(3,3); a=g.solve(); assert(a.size()==3);
6 for(auto [u,v]:a){
7     assert(g.matchL[u]==v && g.matchR[v]==u);
8 }
```

使用边界

邻接表在 `solve` 内按左点压成连续存储，保持各点的加边顺序。`rt[u]` 记录搜索树的未匹配起点，`pre[u]` 记录增广前驱；某起点完成增广后，跳过该搜索树剩余队列项。一轮允许处理不同长度的增广路。设总点数 $V=n+m$ 、边数 E ，空间 $O(V+E)$ ，每轮 $O(n+E)$ ；保守最坏时间界为 $O((V+E)(\min(n,m)+1))$ ，不沿用分层 HK 的复杂度保证。空部、空边集允许，返回空匹配。每次 `solve` 都重新计算；可继续加边再调用。

完整代码

```
1 //////////////////////////////////////
2 //
3 // template for Bipartite Matching
4 // usage:
5 //   Bipartite_Matching g; g.setN(n,m); g.add_edge(u,v);
6 //   auto a=g.solve(); // pairs {u,v}, ordered by the left endpoint
7 //   Left 1..n, right 1..m; a.size() is the maximum matching size.
8 //   matchL / matchR are partners, 0 if unmatched; parallel edges are allowed.
9 //   Greedy initialization + multi-source BFS augmentation; solve rebuilds.
10 //   O(n+m+E) space; O(n+E) per BFS round.
11 //
12 //////////////////////////////////////
13 struct Bipartite_Matching{
14     struct edge{int u,v;};
15     int n=0,m=0;
16     vector<edge> e;
17     vector<int> matchL,matchR;
18     void set(){n=m=0;e.clear();matchL.clear();matchR.clear();}
19     void setN(int _n,int _m){
20         assert(0<=_n&&_n<INT_MAX-1&&0<=_m&&_m<INT_MAX);set();n=_n;m=_m;
21         matchL.assign(n+1,0);matchR.assign(m+1,0);
```

```

22 }
23 void add_edge(int u,int v){
24     assert(1<=u&&u<=n&&1<=v&&v<=m&&e.size()<INT_MAX);
25     e.push_back({u,v});
26 }
27 vector<pair<int,int>> solve(){
28     matchL.assign(n+1,0);matchR.assign(m+1,0);
29     vector<int> head(n+2),to(e.size()),rt(n+1),pre(n+1),q;
30     for(auto [u,v]:e)head[u+1]++;
31     partial_sum(head.begin(),head.end(),head.begin());
32     copy(head.begin(),head.begin()+n+1,rt.begin());
33     for(auto [u,v]:e)to[rt[u]++]=v;
34     q.reserve(n);
35     for(int u=1;u<=n;u++)for(int i=head[u];i<head[u+1];i++)if(!matchR[to[i]]){
36         matchL[u]=to[i];matchR[to[i]]=u;break;
37     }
38     while(true){
39         fill(rt.begin(),rt.end(),0);q.clear();bool found=false;
40         for(int u=1;u<=n;u++)if(!matchL[u]&&head[u]!=head[u+1]){
41             rt[u]=u;pre[u]=0;q.push_back(u);
42         }
43         for(int h=0;h<(int)q.size();h++){
44             int u=q[h];if(matchL[rt[u]])continue;
45             for(int i=head[u];i<head[u+1];i++){
46                 int v=to[i],w=matchR[v];
47                 if(!w){
48                     found=true;
49                     for(int x=u;v;x=pre[x]){
50                         int t=matchL[x];matchL[x]=v;matchR[v]=x;v=t;
51                     }
52                     break;
53                 }
54                 if(!rt[w])rt[w]=rt[u],pre[w]=u,q.push_back(w);
55             }
56         }
57         if(!found)break;
58     }
59     vector<pair<int,int>> ans;ans.reserve(min(n,m));
60     for(int u=1;u<=n;u++)if(matchL[u])ans.emplace_back(u,matchL[u]);
61     return ans;
62 }
63 };
64 // end for graph/bipartite-matching.cpp
65 //////////////////////////////////////

```

3.9 graph/regular-bipartite.cpp

正则二分图的完美匹配分解

什么时候用

两部各 n 个点，每个点度数均为 d ，将全部 nd 条边恰好分成 d 组完美匹配。支持多重边，同一对点之间的边按重数计入分解。

调用顺序与接口

- `Regular_Bipartite g; g.setN(n); g.add_edge(u,v)` 加入左点 u 到右点 v 的边，两部下标均为 $1..n$ 。
- `auto a=g.solve()` 返回 `vector<vector<int>>`，共 d 组。每组长度 $n+1$ ，`a[c][u]` 直接是第 c 组中左点 u 匹配的右点；`a[c][0]=0` 占位。
- 每组的 `a[c][1..n]` 都是右部点的一个排列。所有组中 (u,v) 出现的总次数等于该边的输入重数。组编号为 $0..d-1$ ，组间没有固定的字典序要求。
- d 从输入度数推得，`solve` 用 `assert` 检查正则条件； $d=0$ 返回空分解。 $n=0$ 也返回空分解。

最小示例（放入 main）：

```
1 Regular_Bipartite g; g.setN(2);
2 g.add_edge(1,1); g.add_edge(1,2);
3 g.add_edge(2,1); g.add_edge(2,2);
4 auto a=g.solve(); assert(a.size()==2);
5 for(auto &match:a){
6     assert(match.size()==3 && match[0]==0);
7     assert(1<=match[1] && match[1]<=2);
8     assert(1<=match[2] && match[2]<=2);
9     assert(match[1]!=match[2]);
10 }
```

使用边界

确定性算法，时间 $O(n + nd \log(nd + 1))$ ，包括结果在内空间 $O(n + nd)$ 。偶数度用欧拉分边；奇数度通过整数重数缩减找完美匹配，再借用已分解的匹配，把另一分支凑成二的幂次度数。无随机失败概率。非正则图不属于此接口，不要关闭断言后直接输入；重复 `solve` 不修改原边集。

完整代码

```
1 //////////////////////////////////////
2 //
3 // template for Regular Bipartite Matching
4 // usage:
5 //   Regular_Bipartite g; g.setN(n); g.add_edge(u,v);
6 //   auto a=g.solve(); // d perfect matchings, a[c][u] is the right partner
7 //   Each row has n+1 entries, a[c][0]=0; c is 0-indexed, u is 1-indexed.
8 //   Both sides are 1..n; every vertex must have the same degree d.
9 //   Parallel edges are allowed and counted with multiplicity.
10 //   Deterministic  $O(n+nd*\log(nd+1))$  time,  $O(n+nd)$  space.
11 //
12 //////////////////////////////////////
13 struct Regular_Bipartite{
14     struct edge{int u,v;};
15     int n=0;
16     vector<edge> e=vector<edge>(1);
17     void set(){n=0;e.assign(1,{});}
18     void setN(int _n){assert(0<=_n&&_n<INT_MAX/4);set();n=_n;}
19     void add_edge(int u,int v){
```

```

20  assert(1<=min(u,v)&&max(u,v)<=n&&e.size()+n<INT_MAX/2);
21  e.push_back({u,v});
22  }
23  private:
24  // Split an even-degree bipartite graph into two equal-degree subgraphs.
25  // Negative id -u denotes a temporary edge (u,u).
26  vector<char> split(const vector<int> &a)const{
27  assert(a.size()<INT_MAX/2);int m=a.size();
28  vector<int> head(2*n+1,-1),nxt(2*m);
29  vector<char> vis(m),col(m);
30  for(int i=0;i<m;i++){
31  int id=a[i],u=id>0?e[id].u:-id,v=(id>0?e[id].v:-id)+n;
32  nxt[i*2]=head[u];head[u]=i*2;
33  nxt[i*2+1]=head[v];head[v]=i*2+1;
34  }
35  for(int s=1;s<=2*n;s++){
36  int u=s,c=0;
37  while(head[u]!=-1){
38  int x=head[u];head[u]=nxt[x];int i=x/2;
39  if(vis[i])continue;
40  vis[i]=true;col[i]=c;c^=1;
41  int id=a[i];u=x&1?(id>0?e[id].u:-id):(id>0?e[id].v:-id)+n;
42  }
43  assert(u==s&&!c);
44  }
45  return col;
46  }
47  vector<int> perfect(const vector<int> &ids,int d)const{
48  ll q=1;while(q<(ll)ids.size())q*=2;
49  vector<pair<int,ll>> a; a.reserve(ids.size()+n);
50  for(int id:ids)a.emplace_back(id,q/d);
51  if(q%d)for(int u=1;u<=n;u++)a.emplace_back(-u,q%d);
52  while(q>1){
53  vector<int> odd;
54  for(auto [id,w]:a)if(w&1)odd.push_back(id);
55  auto col=split(odd);int c0=0,c1=0;
56  for(int i=0;i<(int)odd.size();i++)if(odd[i]<0)(col[i]?c1:c0)++;
57  int keep=c1<c0,j=0,k=0;
58  for(auto [id,w]:a){
59  ll v=w/2;if(w&1)v+=col[k++]==keep;
60  if(v)a[j++]={id,v};
61  }
62  a.resize(j);q/=2;
63  }
64  vector<int> ans(n);
65  for(auto [id,w]:a){assert(id>0&&w==1);ans[e[id].u-1]=id;}
66  assert((int)a.size()==n);return ans;
67  }
68  void divide(vector<int> a,int d,vector<vector<int>> &ans)const{
69  if(!d)return;
70  if(d==1){
71  vector<int> b(n);for(int id:a)b[e[id].u-1]=id;
72  ans.push_back(move(b));return;
73  }
74  if(d&1){

```

```

75     ans.push_back(perfect(a,d));const auto &b=ans.back();
76     a.erase(remove_if(a.begin(),a.end(),[&](int id){return b[e[id].u-1]==id;}),a.end());
77     d--;
78 }
79 auto col=split(a);vector<int> l,r;l.reserve(a.size()/2);r.reserve(a.size()/2);
80 for(int i=0;i<(int)a.size();i++){col[i]?r:l}.push_back(a[i]);
81 divide(move(l),d/2,ans);
82 // Borrow some completed matchings, so the other degree is a power of two.
83 int k=1;while(k<=((d-1)/2))k*=2;
84 for(int i=0;i<k-d/2;i++){
85     r.insert(r.end(),ans.back().begin(),ans.back().end());ans.pop_back();
86 }
87 divide(move(r),k,ans);
88 }
89 public:
90 vector<vector<int>> solve()const{
91     if(!n)return {};
92     vector<int> l(n+1),r(n+1),ids(e.size()-1);
93     for(int id=1;id<(int)e.size();id++)l[e[id].u]++,r[e[id].v]++,ids[id-1]=id;
94     int d=l[1];for(int u=1;u<=n;u++)assert(l[u]==d&&r[u]==d);
95     vector<vector<int>> ans;ans.reserve(d);divide(move(ids),d,ans);
96     for(auto &a:ans){
97         vector<int> b(n+1);for(int u=1;u<=n;u++)b[u]=e[a[u-1]].v;
98         a=move(b);
99     }
100     return ans;
101 }
102 };
103 // end for graph/regular-bipartite.cpp
104 //////////////////////////////////////

```

3.10 graph/directed-mst.cpp

指定根的最小树形图

什么时候用

在带权有向图中，以 S 为根，选取 $n-1$ 条边，使 S 能沿选边到达每个点，且总权最小。每个非根点恰有一条入边。

调用顺序与接口

- `Directed_MST g; g.setN(n); id=g.add_edge(u,v,w)` 加入 u 指向 v 的边， w 为权，返回从 1 开始的边编号。
- `auto a=g.solve(S)` 返回 `result`。`a.ok=false` 表示不存在以 S 为根的生成树形图，此时 `id` 为空。
- 有解时 `a.cost` 为总权，`a.id[v]` 为 v 的入边编号，`a.id[S]=0`；遍历所有非根点即可得到实际边集，父亲是 `g.e[a.id[v]].u`。
- 点和 `id` 数组均为 1 下标。 $n=1$ 时有解、总权为 0；要求 $1 \leq S \leq n$ 。允许负权、重边、自环；同端点只保留最小权，平权保留较早加入的边。

最小示例（放入 `main`）：

```

1 Directed_MST g; g.setN(4);
2 g.add_edge(1,2,5); g.add_edge(1,3,8);
3 g.add_edge(2,3,-2); g.add_edge(3,2,-1);
4 g.add_edge(3,4,1);
5 auto a=g.solve(1);
6 assert(a.ok && a.cost==4 && a.id[1]==0);

```

```

7 assert(a.id[2]==1 && a.id[3]==3 && a.id[4]==5);
8 assert(!g.solve(4).ok);

```

使用边界

设 n 为点数、 m 为输入边数，时间 $O((n+m)\log(n+1))$ ，空间 $O(n+m)$ ；任意显式图至少需要 $O(m)$ 读边，只有稀疏图才能写成 n 乘多对数。使用左偏堆、可回退并查集缩环并恢复选边。中间权值和 cost 都是 `__int128`；输出总权时先抄 `basic/int128_IO.cpp`，不要直接强转 `ll`。solve 不修改原图，可更换根重新求解。

完整代码

```

1 ///////////////////////////////////////////////////
2 //
3 // template for Directed MST (Chu-Liu / Edmonds)
4 // usage:
5 //   Directed_MST g; g.setN(n); g.add_edge(u,v,w); auto a=g.solve(S);
6 //   a.ok; a.cost; a.id[v] is the incoming edge of v, a.id[S]=0.
7 //   Vertices / edge ids are 1-indexed; edges point away from the root S.
8 //   Negative weights / parallel edges / loops are allowed. No solution: ok=false.
9 //    $O((n+m)\log(n+1))$  time,  $O(n+m)$  space; cost uses __int128.
10 //
11 ///////////////////////////////////////////////////
12 struct Directed_MST{
13     struct edge{int u,v;ll w;};
14     struct result{bool ok=false;__int128 cost=0;vector<int> id;};
15     int n=0;
16     vector<edge> e=vector<edge>(1);
17     void set(){n=0;e.assign(1,{})};
18     void setN(int _n){assert(0<=_n&&_n<INT_MAX);set();n=_n;}
19     int add_edge(int u,int v,ll w){
20         assert(1<=min(u,v)&&max(u,v)<=n&&e.size()<INT_MAX);
21         int id=e.size();e.push_back({u,v,w});return id;
22     }
23 private:
24     struct heap{
25         struct node{int ls=0,rs=0,dis=0,id=0;__int128 w=0,lazy=0;};
26         vector<node> a=vector<node>(1);
27         void add(int u,__int128 w){if(u)a[u].w+=w,a[u].lazy+=w;}
28         void down(int u){add(a[u].ls,a[u].lazy);add(a[u].rs,a[u].lazy);a[u].lazy=0;}
29         int merge(int u,int v){
30             if(!u||!v)return u|v;
31             if(a[u].w>a[v].w||(a[u].w==a[v].w&&a[u].id>a[v].id))swap(u,v);
32             down(u);a[u].rs=merge(a[u].rs,v);
33             if(a[a[u].ls].dis<a[a[u].rs].dis)swap(a[u].ls,a[u].rs);
34             a[u].dis=a[a[u].rs].dis+1;return u;
35         }
36         int push(int u,int id,ll w){
37             a.push_back({0,0,1,id,w,0});return merge(u,(int)a.size()-1);
38         }
39         int pop(int u){down(u);return merge(a[u].ls,a[u].rs);}
40     };
41     struct DSU{
42         vector<int> fa,siz;
43         vector<pair<int,int>> stk;
44         DSU(int n):fa(n+1),siz(n+1,1){iota(fa.begin(),fa.end(),0);}
45         int find(int u)const{while(fa[u]!=u)u=fa[u];return u;}
46         bool join(int u,int v){

```

```

47     u=find(u);v=find(v);if(u==v)return false;
48     if(siz[u]<siz[v])swap(u,v);
49     stk.emplace_back(u,v);fa[v]=u;siz[u]+=siz[v];return true;
50 }
51 void rollback(int t){
52     while((int)stk.size()>t){auto [u,v]=stk.back();stk.pop_back();fa[v]=v;siz[u]-=siz[v];}
53 }
54 };
55 public:
56 result solve(int S)const{
57     assert(1<=S&&S<=n);DSU dsu(n);heap h;
58     vector<int> rt(n+1),seen(n+1),path(n),q(n),in(n+1);
59     // Keep one cheapest edge per ordered pair, also bounding heap size by n*n.
60     {
61         vector<vector<int>> edg(n+1);vector<int> vis(n+1),best(n+1),a;
62         for(int id=1;id<(int)e.size();id++)if(e[id].u!=e[id].v&&e[id].v!=S)edg[e[id].v].push_back(id);
63         for(int v=1;v<=n;v++){
64             a.clear();
65             for(int id:edg[v]){
66                 int u=e[id].u;
67                 if(vis[u]!=v)vis[u]=v,best[u]=id,a.push_back(u);
68                 else if(e[id].w<e[best[u]].w)best[u]=id;
69             }
70             for(int u:a)rt[v]=h.push(rt[v],best[u],e[best[u]].w);
71         }
72     }
73     struct cycle{int u,t;vector<int> e;};
74     vector<cycle> cyc;__int128 cost=0;seen[S]=-1;
75     for(int s=1;s<=n;s++){
76         int u=dsu.find(s),top=0;
77         while(!seen[u]){
78             while(rt[u]&&dsu.find(e[h.a[rt[u]].id].u)==u)rt[u]=h.pop(rt[u]);
79             if(!rt[u])return {};
80             int id=h.a[rt[u]].id;__int128 w=h.a[rt[u]].w;
81             cost+=w;h.add(rt[u],-w);rt[u]=h.pop(rt[u]);
82             q[top]=id;path[top++]=u;seen[u]=s;u=dsu.find(e[id].u);
83             if(seen[u]==s){
84                 int end=top,t=dsu.stk.size(),r=0,v;
85                 do{v=path[--top];r=h.merge(r,rt[v]);}while(dsu.join(u,v));
86                 u=dsu.find(u);rt[u]=r;seen[u]=0;
87                 cyc.push_back({u,t,vector<int>(q.begin()+top,q.begin()+end)});
88             }
89         }
90         for(int i=0;i<top;i++)in[dsu.find(e[q[i]].v)]=q[i];
91     }
92     for(int i=(int)cyc.size()-1;i>=0;i--){
93         auto &c=cyc[i];int id=in[c.u];assert(id);
94         dsu.rollback(c.t);
95         for(int x:c.e)in[dsu.find(e[x].v)]=x;
96         in[dsu.find(e[id].v)]=id;
97     }
98     return {true,cost,move(in)};
99 }
100 };
101 // end for graph/directed-mst.cpp

```

102 //////////////////////////////////

3.11 graph/chordal.cpp

弦图判定与完美消除序列

什么时候用

判断无向图中是否不存在长度至少为 4 的无弦诱导环。如果是弦图，返回一个完美消除序列。

调用顺序与接口

- Chordal g; g.setN(n), 点下标 1..n; g.add_edge(u,v) 加无向边。自环忽略，重边在求解时去重。
- bool ok=g.solve()。成功时 peo[0..n-1] 按从前到后给出消除顺序，rk[u] 是 u 在序列中的 1 下标位置。
- 依次消除 peo 中的点时，每个点尚未消除的邻居构成团。不要把 MCS 的选点顺序直接作为消除顺序，两者相反。
- 失败时清空 peo、rk。空图和非连通图均支持；可以继续加边再求解。

最小示例（放入 main）：

```
1 Chordal g; g.setN(4);
2 g.add_edge(1,2); g.add_edge(2,3);
3 g.add_edge(3,4); g.add_edge(4,1);
4 assert(!g.solve() && g.peo.empty());
5 g.add_edge(1,3); assert(g.solve());
6 assert(g.peo.size()==4);
7 for(int i=0;i<4;i++)assert(g.rk[g.peo[i]]==i+1);
```

使用边界

时间、空间均为 $O(n+m)$, m 为输入边数。MCS 用桶维护已选邻居数；验证时把结点按最靠前的后继邻居分组，统一检查后继邻居的包含关系，避免对每个邻域两两枚举。返回 false 时只给出判定，不构造无弦环证据。

完整代码

```
1 //////////////////////////////////
2 //
3 // template for Chordal Graph Recognition (MCS)
4 // usage:
5 //   Chordal g; g.setN(n); g.add_edge(u,v); bool ok=g.solve();
6 //   If ok, peo[0..n-1] is the elimination order, rk[u] is 1-indexed.
7 //   Undirected, vertices 1..n; loops are ignored, parallel edges are merged.
8 //   O(n+m) time / space; a failed solve clears peo and rk.
9 //
10 //////////////////////////////////
11 struct Chordal{
12     int n=0;
13     vector<vector<int>> edg;
14     vector<int> peo,rk;
15     void set(){n=0;edg.clear();peo.clear();rk.clear();}
16     void setN(int _n){assert(0<=_n&&_n<INT_MAX);set();n=_n;edg.resize(n+1);}
17     void add_edge(int u,int v){
18         assert(1<=min(u,v)&&max(u,v)<=n);
19         if(u!=v)edg[u].push_back(v),edg[v].push_back(u);
20     }
21     bool solve(){
22         vector<int> vis(n+1),cnt(n+1);vector<vector<int>> bucket(n+1),son(n+1);
23         for(int u=1;u<=n;u++){
24             int k=0;
```

```

25     for(int v:edg[u])if(vis[v]!=u)vis[v]=u,edg[u][k++]=v;
26     edg[u].resize(k);bucket[0].push_back(u);
27 }
28 peo.resize(n);rk.assign(n+1,0);int mx=0;
29 for(int i=n;i>=1;i--){
30     while(true){
31         while(bucket[mx].empty())mx--;
32         int u=bucket[mx].back();bucket[mx].pop_back();
33         if(rk[u]||cnt[u]!=mx)continue;
34         peo[i-1]=u;rk[u]=i;
35         for(int v:edg[u])if(!rk[v]){
36             bucket[++cnt[v]].push_back(v);mx=max(mx,cnt[v]);
37         }
38         break;
39     }
40 }
41 for(int u=1;u<=n;u++){
42     int p=0;
43     for(int v:edg[u])if(rk[v]>rk[u]&&(!p||rk[v]<rk[p]))p=v;
44     if(p)son[p].push_back(u);
45 }
46 fill(vis.begin(),vis.end(),0);
47 for(int p=1;p<=n;p++){
48     for(int v:edg[p])vis[v]=p;
49     for(int u:son[p])for(int v:edg[u])if(v!=p&&rk[v]>rk[u]&&vis[v]!=p){
50         peo.clear();rk.clear();return false;
51     }
52 }
53 return true;
54 }
55 };
56 // end for graph/chordal.cpp
57 ///////////////////////////////////

```

4 树上算法

4.1 trees/HLD.cpp

LCA、祖先和路径分段

什么时候用

对一棵连通无向树预处理，支持 $O(1)$ LCA 和 $O(\log n)$ 路径分段。

调用顺序与接口

- `Tree t; t.setN(n)` 后加入 $n-1$ 条边；可设置 `t.rt`，再 `build`。
- `lca(u,v)`、`dist(u,v)` 返回最近公共祖先及边数距离；`kth_anc(u,k)` 的 $k=0$ 返回自身。
- `in[u]` 是 DFS 编号，`dfn[i]` 反查原点；子树为 `[in[u],out[u]]`。
- `index_of_path(u,v)` 返回覆盖点路径的一组 DFS 序闭区间。

最小示例（放入 main）：

```
1 Tree t; t.setN(3);
2 t.add_edge(1,2); t.add_edge(1,3); t.build();
3 assert(t.lca(2,3)==1 && t.dist(2,3)==2);
4 int cnt=0;
5 for(auto [l,r]:t.index_of_path(2,3))cnt+=r-l+1;
6 assert(cnt==3);
```

使用边界

配线段树时，把点 u 的值放在位置 `in[u]`，不要直接用 u 。路径段没有保证从 u 到 v 的方向顺序，只能直接用于可交换聚合。按边统计时要另行排除 LCA。与 `DC.cpp` 都定义 `Tree`。

完整代码

```
1 //
2 // template for Heavy-Light Decomposition, prepared by Otomachi Una
3 // version 1.0 (Last Update Jun 13th, 2026)
4 //
5 // usage:
6 //   Tree tr; tr.setN(n); tr.add_edge(u,v); tr.build();
7 //   tr.lca(u,v); tr.index_of_path(u,v);
8 //
9 struct Tree{
10 int n=0,rt=1,tot=0;
11 vector<vector<int>> edg,st;
12 vector<int> in,out,siz,ff,dep,dfn,hson,top,ed;
13 void setN(int _n){
14     assert(1<=_n&&_n<INT_MAX-1);n=_n;tot=0;rt=1;
15     edg.assign(n+1,{});st.resize(__lg(n)+1);
16     for(int i=0;i<(int)st.size();i++)st[i].assign(n-(1<<i)+2,0);
17     in.assign(n+1,0);
18     out.assign(n+1,0);
19     siz.assign(n+1,0);
20     ff.assign(n+1,0);
21     dep.assign(n+1,0);
22     dfn.assign(n+1,0);
23     hson.assign(n+1,0);
24     top.assign(n+1,0);
25     ed.assign(n+1,0);
```

```

26 }
27 void setRt(int _rt){assert(1<=_rt&&_rt<=n);rt=_rt;}
28 void add_edge(int u,int v){
29     assert(1<=min(u,v)&&max(u,v)<=n&&u!=v);
30     edg[u].push_back(v);
31     edg[v].push_back(u);
32 }
33 void HLD(int u,int fa){
34     vector<int> q{u};assert(!siz[u]);siz[u]=1;ff[u]=fa;
35     for(int i=0;i<(int)q.size();i++){
36         int x=q[i];
37         for(int v:edg[x])if(v!=ff[x]){
38             assert(!siz[v]);siz[v]=1;ff[v]=x;dep[v]=dep[x]+1;q.push_back(v);
39         }
40     }
41     for(int i=(int)q.size()-1;i>0;i--){
42         int x=q[i],p=ff[x];siz[p]+=siz[x];
43         if(siz[x]>siz[hson[p]])hson[p]=x;
44     }
45 }
46 void HLD1(int u,int fa){
47     int start=tot;vector<int> q{u};ff[u]=fa;
48     while(!q.empty()){
49         int x=q.back();q.pop_back();in[x]=++tot;dfn[tot]=x;st[0][tot]=ff[x];
50         for(int i=(int)edg[x].size()-1;i>=0;i--){
51             int v=edg[x][i];if(v==ff[x]||v==hson[x])continue;
52             top[v]=v;q.push_back(v);
53         }
54         if(hson[x]){top[hson[x]]=top[x];q.push_back(hson[x]);}
55         out[x]=in[x]+siz[x]-1;
56     }
57     for(int i=tot;i>start;i--){int x=dfn[i];ed[x]=hson[x]?ed[hson[x]]:x;}
58 }
59 inline int higher(int x,int y){return dep[x]<dep[y]?x:y;}
60 void build(){
61     // build all basic information in O(nlogn) (without lca O(n))
62     assert(1<=rt&&rt<=n);tot=0;
63     fill(hson.begin(),hson.end(),0);fill(siz.begin(),siz.end(),0);
64     dep[rt]=1;top[rt]=rt;HLD(rt,0);HLD1(rt,0);assert(tot==n);
65     for(int i=1;i<(int)st.size();i++) for(int j=1;j<=n-(1<<i)+1;j++){
66         st[i][j]=higher(st[i-1][j],st[i-1][j+(1<<(i-1))]);
67     }
68     inline int lca(int x,int y){
69         // find lca ancestor in O(1)
70         assert(1<=min(x,y)&&max(x,y)<=n);
71         if(x==y) return x;
72         x=in[x],y=in[y];
73         if(x>y) swap(x,y);
74         x++;int k=_lg(y-x+1);
75         return higher(st[k][x],st[k][y-(1<<k)+1]);
76     }
77     inline int dist(int u,int v){
78         return dep[u]+dep[v]-2*dep[lca(u,v)];
79     }
80     inline int kth_anc(int u,int k){

```

```

81 // find k-th ancestor in O(logn)
82 assert(1<=u&&u<=n&&0<=k&&k<dep[u]);
83 while(1){
84     if(dep[u]-dep[top[u]]>=k) return dfn[in[u]-k];
85     k-=dep[u]-dep[top[u]]+1;
86     u=ff[top[u]];
87     if(!u) return rt;
88 }
89 }
90 inline vector<array<int,2>> index_of_path(int x,int y){
91     // return all in ids in path (x,y)
92     assert(1<=min(x,y)&&max(x,y)<=n);
93     vector<array<int,2>> id;
94     while(top[x]!=top[y]){
95         if(dep[top[x]]<dep[top[y]]) swap(x,y);
96         id.push_back({in[top[x]],in[x]});
97         x=ff[top[x]];
98     }
99     if(dep[x]>dep[y]) swap(x,y);
100    id.push_back({in[x],in[y]});
101    return id;
102 }
103 void output(){
104     cerr<<"*****\n";
105     cerr<<"structure:"<<endl;
106     cerr<<"root = "<<rt<<endl;
107     for(int i=1;i<=n;i++) for(int j:edg[i]) if(i<j)
108         cerr<<i<<' '<<j<<' \n';
109     cerr<<"*****\n";
110     cerr<<"dfn-order: ";
111     for(int i=1;i<=n;i++) cerr<<dfn[i]<<" \n"[i==n];
112     cerr<<"top: ";
113     for(int i=1;i<=n;i++) cerr<<top[i]<<" \n"[i==n];
114 }
115 };

```

4.2 trees/DC.cpp

点分治框架

什么时候用

提供寻找重心、递归处理剩余连通块的框架。与 HLD 查询共用 Tree，题目需要的统计尚未实现。

调用顺序与接口

- Tree t; t.setN(n) 后加入树边，设置 rt。
- 需要 LCA 时先 HLD_build; DC_build 从整棵树开始点分治。
- 在 get 中收集子树信息，在 DC 中实现合并、统计和清空逻辑。

粘贴本文件之前：

最小示例（放入 main）：

```
1 Tree t; t.setN(3);
```

```

2 t.add_edge(1,2); t.add_edge(2,3);
3 t.HLD_build(); assert(t.dist(1,3)==2);
4 t.DC_build(); // traversal only, no problem answer

```

使用边界

本例只演示初始化和遍历，不会输出路径统计答案。不能原样当作“距离为 k 的点对计数”使用。

完整代码

```

1 //
2 // template for Centroid Decomposition, prepared by Otomachi Una
3 // version 1.0 (Last Update Jun 13th, 2026)
4 //
5 // usage:
6 //   Tree tr; tr.setN(n); tr.add_edge(u,v); tr.HLD_build(); tr.DC_build();
7 //   fill logic inside DC();
8 //
9 struct Tree{
10 int n=0,rt=1,tot=0;
11 vector<vector<int>> edg,st;
12 vector<int> in,out,siz,ff,dep,dfn,hson,top,ed,msiz,csiz;
13 vector<char> vis;int tsiz=0,cen=0;
14 void setN(int _n){
15     assert(1<=_n&&_n<INT_MAX-1);n=_n;tot=0;rt=1;
16     edg.assign(n+1,{});st.resize(_lg(n)+1);
17     for(int i=0;i<(int)st.size();i++)st[i].assign(n-(1<<i)+2,0);
18     in.assign(n+1,0);
19     out.assign(n+1,0);
20     siz.assign(n+1,0);
21     ff.assign(n+1,0);
22     dep.assign(n+1,0);
23     dfn.assign(n+1,0);
24     hson.assign(n+1,0);
25     top.assign(n+1,0);
26     ed.assign(n+1,0);
27     msiz.assign(n+1,0);
28     csiz.assign(n+1,0);
29     vis.assign(n+1,0);tsiz=cen=0;
30 }
31 void add_edge(int u,int v){
32     assert(1<=min(u,v)&&max(u,v)<=n&&u!=v);
33     edg[u].push_back(v);
34     edg[v].push_back(u);
35 }
36 void HLD(int u,int fa){
37     vector<int> q{u};assert(!siz[u]);siz[u]=1;ff[u]=fa;
38     for(int i=0;i<(int)q.size();i++){
39         int x=q[i];
40         for(int v:edg[x])if(v!=ff[x]){
41             assert(!siz[v]);siz[v]=1;ff[v]=x;dep[v]=dep[x]+1;q.push_back(v);
42         }
43     }
44     for(int i=(int)q.size()-1;i>0;i--){
45         int x=q[i],p=ff[x];siz[p]+=siz[x];
46         if(siz[x]>siz[hson[p]])hson[p]=x;
47     }

```

```

48 }
49 void HLD1(int u,int fa){
50     int start=tot;vector<int> q{u};ff[u]=fa;
51     while(!q.empty()){
52         int x=q.back();q.pop_back();in[x]=++tot;dfn[tot]=x;st[0][tot]=ff[x];
53         for(int i=(int)edg[x].size()-1;i>=0;i--){
54             int v=edg[x][i];if(v==ff[x]||v==hson[x])continue;
55             top[v]=v;q.push_back(v);
56         }
57         if(hson[x]){top[hson[x]]=top[x];q.push_back(hson[x]);}
58         out[x]=in[x]+siz[x]-1;
59     }
60     for(int i=tot;i>start;i--){int x=dfn[i];ed[x]=hson[x]?ed[hson[x]]:x;}
61 }
62 inline int higher(int x,int y){return dep[x]<dep[y]?x:y;}
63 void HLD_build(){
64     // build all basic information in O(nlogn) (without lca O(n))
65     assert(1<=rt&&rt<=n);tot=0;
66     fill(hson.begin(),hson.end(),0);fill(siz.begin(),siz.end(),0);
67     dep[rt]=1;top[rt]=rt;HLD(rt,0);HLD1(rt,0);assert(tot==n);
68     for(int i=1;i<(int)st.size();i++) for(int j=1;j<=n-(1<<i)+1;j++)
69         st[i][j]=higher(st[i-1][j],st[i-1][j+(1<<(i-1))]);
70 }
71 inline int lca(int x,int y){
72     // find lca ancestor in O(1)
73     assert(1<=min(x,y)&&max(x,y)<=n);
74     if(x==y) return x;
75     x=in[x],y=in[y];
76     if(x>y) swap(x,y);
77     x++;int k=_lg(y-x+1);
78     return higher(st[k][x],st[k][y-(1<<k)+1]);
79 }
80 inline int dist(int u,int v){
81     return dep[u]+dep[v]-2*dep[lca(u,v)];
82 }
83 inline int kth_anc(int u,int k){
84     // find k-th ancestor in O(logn)
85     assert(1<=u&&u<=n&&0<=k&&k<dep[u]);
86     while(1){
87         if(dep[u]-dep[top[u]]>=k) return dfn[in[u]-k];
88         k-=dep[u]-dep[top[u]]+1;
89         u=ff[top[u]];
90         if(!u) return rt;
91     }
92 }
93 void get_size(int u,int fa){
94     vector<array<int,2>> q{{u,fa}};
95     for(int i=0;i<(int)q.size();i++){
96         auto [x,p]=q[i];csiz[x]=1;msiz[x]=0;
97         for(int v:edg[x])if(v!=p&&!vis[v])q.push_back({v,x});
98     }
99     for(int i=(int)q.size()-1;i>=0;i--){
100         auto [x,p]=q[i];msiz[x]=max(msiz[x],tsiz-csiz[x]);
101         if(msiz[x]<msiz[cen])cen=x;
102         if(i){csiz[p]+=csiz[x];msiz[p]=max(msiz[p],csiz[x]);}

```

```

103     }
104 }
105 void get(int u,int fa){
106     vector<array<int,2>> q{{u,fa}};
107     for(int i=0;i<(int)q.size();i++){
108         auto [x,p]=q[i];
109         // collect information at x here
110         for(int v:edg[x])if(v!=p&&!vis[v])q.push_back({v,x});
111     }
112 }
113 void DC(int u){
114     vis[u]=true;
115     for(int v:edg[u]) if(!vis[v]){
116         // get some info?
117         get(v,0);
118     }
119     // solve whole
120     for(int v:edg[u]) if(!vis[v]){
121         tsiz=csiz[v];cen=0;
122         get_size(v,0);get_size(cen,0);DC(cen);
123     }
124     // cerr<<"finish"<<endl;
125 }
126 void DC_build(){
127     assert(1<=rt&&rt<=n);fill(vis.begin(),vis.end(),0);
128     tsiz=n;msiz[cen=0]=1e9;get_size(rt,0);get_size(cen,0);DC(cen);
129 }
130 void output(){
131     cerr<<"*****\n";
132     cerr<<"structure:"<<endl;
133     cerr<<"root = "<<rt<<endl;
134     for(int i=1;i<=n;i++) for(int j:edg[i]) if(i<j)
135         cerr<<i<<' '<<j<<' \n';
136     cerr<<"*****\n";
137     cerr<<"dfn-order: ";
138     for(int i=1;i<=n;i++) cerr<<dfn[i]<<" \n"[i==n];
139     cerr<<"top: ";
140     for(int i=1;i<=n;i++) cerr<<top[i]<<" \n"[i==n];
141 }
142 // modify from here
143 void solve(){
144     // do not forget setN
145     // use add_edge(u,v) to add a edge
146     // do not forget HLD_build()
147     int N;cin>>N;setN(N);
148     for(int i=1,f;i<=n;i++){
149         cin>>f;
150         if(f) add_edge(i,f);
151         else rt=i;
152     }
153     HLD_build();
154     DC_build();
155 }
156 };

```

4.3 trees/cartesian.cpp

小根堆笛卡尔树

什么时候用

$O(n)$ 构建，同时满足下标的中序顺序与值的小根堆序，空间 $O(n)$ 。

调用顺序与接口

`auto [ls,rs]=cartesian(a)` 接受从 0 开始的 vector，返回两个同长度 vector，分别为左右儿子下标。无儿子用 -1。相等值保留较早下标在上方。根是原数组中第一个最小值的位置。

最小示例（放入 main）：

```
1 vector<int> a{3,1,2,1};
2 auto [ls,rs]=cartesian(a);
3 int rt=min_element(a.begin(),a.end())-a.begin();
4 assert(rt==1 && ls[rt]==0 && rs[rt]==3 && ls[3]==2);
```

使用边界

空输入返回两个空 vector，此时没有根。构造无递归，单调数组形成长链也可直接构建；后续遍历要自行避免递归爆栈。类型只需支持小于比较。返回值不含原数组或额外哨兵。

完整代码

```
1 //////////////////////////////////////
2 //
3 // template for Cartesian Tree (min heap)
4 //
5 // usage:
6 //   auto [ls,rs]=cartesian(a); // 0-indexed, absent child = -1
7 //    $O(n)$ ; equal values keep the earlier index above the later one.
8 //   The root is the first minimum; empty input returns two empty vectors.
9 //
10 //////////////////////////////////////
11 template<typename T> pair<vector<int>,vector<int>> cartesian(const vector<T> &a){
12     assert(a.size()<(size_t)INT_MAX);int n=a.size();
13     vector<int> ls(n,-1),rs(n,-1),st;st.reserve(n);
14     for(int i=0;i<n;i++){
15         int last=-1;
16         while(!st.empty()&&a[i]<a[st.back()]){last=st.back();st.pop_back();}
17         ls[i]=last;
18         if(!st.empty())rs[st.back()]=i;
19         st.push_back(i);
20     }
21     return {move(ls),move(rs)};
22 }
23 // end for trees/cartesian.cpp
24 //////////////////////////////////////
```

5 数论

5.1 number theory/basic.cpp

模逆元和扩展 CRT

什么时候用

提供 gcd、lcm、逆元与不要求模数互素的同余方程合并。

调用顺序与接口

- `inv(a,m)` 要求 $m>0$ 且 $\gcd(a,m)=1$ 。
- `COE(r,m)` 表示 $x=r \pmod m$, $a+b$ 或 $a+=b$ 合并两个方程。
- 结果 `empty()` 为 `true` 表示无解; 否则 `r` 是标准余数、`M` 是合并模数, `accept(x)` 判断满足性。

最小示例 (放入 main):

```
1 COE a(2,3),b(3,5); auto c=a+b;
2 assert(!c.empty() && c.r==8 && c.M==15);
3 assert(c.accept(23) && inv(3,11)==4);
4 assert((COE(0,2)+COE(1,2)).empty());
```

使用边界

合并后的模数须能放入 `ll`, 内部乘法虽提升到 `__int128`, 但结果不是任意精度。`inv` 函数会与部分模数模板的全局 `inv` 数组重名, 需要按实际组合改名。

完整代码

```
1 //
2 // template for inv, CRT, etc.
3 // version 1.0 (Last Update Jun 23rd, 2026)
4 //
5 // usage:
6 //     inv(x,M); COE a(r,M); COE c=a+b;
7 //     c.empty() if no solution
8 //
9 using ull=unsigned long long;
10 inline ull gcd(ull x,ull y){
11     // Stein's Algorithm
12     if(!x||!y) return x|y;
13     int c=__builtin_ctzll(x|y);
14     x>>=__builtin_ctzll(x),y>>=__builtin_ctzll(y);
15     while(x!=y){
16         if(x<y)swap(x,y);
17         x-=y;
18         x>>=__builtin_ctzll(x);
19     }
20     return y<<c;
21 }
22 inline ull lcm(ull x,ull y){
23     if(!x||!y)return 0;
24     x/=gcd(x,y);assert(x<=ULLONG_MAX/y);return x*y;
25 }
26 inline ll inv(ll x,ll M){
27     // return x^{-1} mod M
28     assert(M>0);x%=M;if(x<0)x+=M;
29     ull a=M,b=x,__int128 y=0,z=1;
```

```

30 while(b){
31     const ull q=a/b,c=a-q*b;
32     a=b,b=c;
33     const __int128 w=y-static_cast<__int128>(q)*z;
34     y=z,z=w;
35 }
36 assert(a==1U);y%=M;
37 return (y>=0?y:M+y);
38 }
39 // Congruence equation, make, merge, etc.
40 struct COE{
41 ll r,M;
42 COE():r(0),M(1){}
43 COE(ll _r,ll _M):r(_r),M(_M){assert(M>0);r%=M;if(r<0)r+=M;}
44 inline bool empty()const{return M==1;}
45 inline bool accept(ll x)const{if(empty())return false;x%=M;return (x<0?x+M:x)==r;}
46 inline COE& operator +=(const COE &x){
47     if(empty()||x.empty()){M=-1;return *this;}
48     ll g=gcd(M,x.M);
49     if(r%g!=x.r%g){M=-1;return *this;}
50     assert(M/g<=LLONG_MAX/x.M);
51     ll i=inv(M/g,x.M/g),new_M=M/g*x.M;
52     r=((__int128)r+static_cast<__int128>(x.r-r)/g*i%(x.M/g)*M)%new_M;
53     M=new_M;
54     if(r<0) r+=M;
55     return *this;
56 }
57 inline COE operator +(const COE &x)const{
58     return COE(*this)+=x;
59 }
60 };

```

5.2 number theory/linear sieve.cpp

线性筛素数

什么时候用

$O(n)$ 时间和空间, 适合需要枚举范围内全部素数。

调用顺序与接口

- `init(n)` 重建范围 $0..n$, $n \geq 1$ 。
- `is_prime[x]` 判断 x 是否为素数; `prime[1..m]` 为升序素数, `prime[0]` 是占位。

最小示例 (放入 `main`):

```

1 init(20);
2 assert(m==8 && prime[1]==2 && prime[m]==19);
3 assert(is_prime[17] && !is_prime[1]);

```

使用边界

不是任意大整数判素接口, 不能访问 n 以外。全局 `init`、`m`、`is_prime` 名字较通用, 与其他模板组合时应检查冲突。

完整代码

```

1 //
2 // template for Linear Sieve
3 //
4 // usage:
5 //   init(n); is_prime[x]; prime[i] // i-th prime, 1-index
6 //
7 vector<char> is_prime;
8 vector<int> prime;
9 int m=0;
10 void init(int n){
11     assert(1<=n&&n<INT_MAX);
12     is_prime.assign(n+1,true);prime.assign(1,0);m=0;
13     is_prime[0]=is_prime[1]=false;
14     for(int i=2;i<=n;i++){
15         if(is_prime[i])prime.push_back(i),++m;
16         for(int j=1;j<=m&&prime[j]<=n/i;j++){
17             int x=i*prime[j];is_prime[x]=false;
18             if(i%prime[j]==0)break;
19         }
20     }
21 }
```

5.3 number theory/pollard-rho.cpp

大整数判素与质因数分解

什么时候用

Miller-Rabin 配合 Pollard-Rho, 适用于范围内不适合筛到上界的整数。随机算法的运行时间依赖输入。

调用顺序与接口

- `is_prime(n)` 判断素数。
- `factor(n)` 返回升序质因子, 含重数; `n=1` 返回空。
- `pollard_rho(n)` 提供一个因子, 素数时返回自身。

最小示例 (放入 `main`):

```

1 auto f=factor(360);
2 assert((f==vector<ull>{2,2,2,3,3,5}));
3 assert(is_prime(1000000007ULL));
```

使用边界

`factor` 的输入要求 $1 \leq n < 2^{62}$, 不能传 0。不是完整 `uint64` 范围版本。判素函数与线性筛的 `is_prime` 数组同名, 不能原样同贴。

完整代码

```

1 //
2 // template for Pollard-Rho, prepared by Otomachi Una
3 // version 1.0 (Last Update Jun 17th, 2026)
4 //
5 // usage:
6 //   is_prime(n); auto f = factor(n);
7 //
```

```

8 using ull=unsigned long long;
9 using u128=__uint128_t;
10 inline ull rho_gcd(ull x,ull y){
11     // Stein's Algorithm
12     if(!x||!y) return x|y;
13     int c=__builtin_ctzll(x|y);
14     x>>=__builtin_ctzll(x),y>>=__builtin_ctzll(y);
15     while(x!=y){
16         if(x<y)swap(x,y);
17         x-=y;
18         x>>=__builtin_ctzll(x);
19     }
20     return y<<c;
21 }
22 bool is_prime(ull n){
23     if(n<2)return false;
24     for(auto y:{2,3,5}){
25         if(n==y)return true;
26         if(n%y==0)return false;
27     }
28     assert(n<(1ull<<62)); // use Montgomery
29     ull n2=n*2,r=n&3;
30     for(int _=0;_<5;_++)r*=2-n*r;
31     r=-r;
32     ull t=-n/n,e=-u128(n)%n;
33     auto redc=[&](u128 x)->ull{return (x+u128((ull)(x)*r)*n)>>64;};
34     auto mul=[&](ull x,ull y)->ull{return redc(u128(x)*y);};
35     auto de=[&](ull x)->ull{x=redc(x);return x<n?x:x-n;};
36     auto en=[&](ull x)->ull{return mul(x,e);};
37     auto pow=[&](ull a,ull b){
38         ull res=t,base=en(a);
39         while(b){
40             if(b&1)res=mul(res,base);
41             base=mul(base,base);
42             b>>=1;
43         }
44         return res;
45     };
46     ull d=n-1;int z=__builtin_ctzll(d);d>>=z;
47     auto miller_rabin=[&](ull b){
48         if(b==0)return true;
49         ull y=pow(b,d);
50         if(de(y)==1)return true;
51         for(int i=0;i<z;i++){
52             if(de(y)==n-1)return true;
53             y=mul(y,y);
54         }
55         return false;
56     };
57     if(n<4759123141ull){
58         for(auto b:{2,7,61})if(!miller_rabin(b%n))return false;
59     }else{
60         for(auto b:{2,325,9375,28178,450775,9780504,1795265022})if(!miller_rabin(b%n))return false;
61     }
62     return true;

```

```

63 }
64 ull pollard_rho(ull p){
65     assert(p>=2);
66     if(p%2==0)return 2;
67     if(is_prime(p))return p;
68     assert(p<(1ull<<62)); // use Montgomery
69     ull n=p,n2=n*2,r=n&3;
70     for(int _=0;_<5;_++)r*=2-n*r;
71     r=-r;
72     ull t=-n%n;
73     auto redc=[&](u128 x)->ull{return (x+u128((ull)(x)*r)*n)>>64;};
74     auto mul=[&](ull x,ull y)->ull{return redc(u128(x)*y);};
75     auto dif=[&](ull x,ull y)->ull{x+=n2-y;return x<n2?x:x-n2;};
76     auto de=[&](ull x)->ull{x=redc(x);return x<n?x:x-n;};
77     mt19937_64 rnd;
78     while(1){
79         ull c=rnd()%(p-1)+1,y=rnd()%(p-1)+1;
80         auto f=[&](ull x)->ull{return redc(u128(x)*x+c);};
81         for(ull s=1;;s<=1){
82             ull x=y;
83             const ull m=min(1ull<<max(int(__lg(p))/3-8,0),s);
84             for(ull i=0;i<s/m;i++){
85                 ull w=t,z=y;
86                 for(ull j=0;j<m;j++){
87                     y=f(y);
88                     w=mul(w,dif(y,x));
89                 }
90                 ull g=rho_gcd(de(w),n);
91                 if(g>1){
92                     if(g<n)return g;
93                     for(ull j=0;j<m;j++){
94                         z=f(z);
95                         if((g=rho_gcd(de(dif(z,x)),n))!=1){
96                             if(g<n)return g;
97                             else goto fail;
98                         }
99                     }
100                 }
101             }
102         }
103         fail;;
104     }
105 }
106 vector<ull> factor(ull x){
107     vector<ull> ans;
108     auto dfs=[&](auto self,ull x){
109         if(x==1)return;
110         ull y=pollard_rho(x);
111         if(x==y){ans.push_back(x);return;}
112         self(self,y);self(self,x/y);
113     };dfs(dfs,x);
114     sort(ans.begin(),ans.end());
115     return ans;
116 }

```

5.4 number theory/floor sum.cpp

类欧几里得整除求和

什么时候用

计算闭区间 $i=l..r$ 上 $\text{floor}((a*i+b)/c)$ 的和, 时间 $O(\log c)$, 返回 `__int128`。

调用顺序与接口

- `floor_sum(l,r,a,b,c)` 参数都是 `ll`。
- `l,r,a,b` 均非负, `c>0`; `l>r` 返回 0。

最小示例 (放入 `main`):

```
1 auto ans=floor_sum(1,4,3,1,2);
2 assert(ans==16); // 2+3+5+6
```

使用边界

不能直接处理负系数或负下标。输出可配 `int128_IO`; 不要先在 `ll` 中计算 `a*l`。答案和内部累计值须在有符号 128 位范围内。

完整代码

```
1 //
2 // template for Floor Sum, O(log c)
3 //
4 // usage:
5 //   floor_sum(l,r,a,b,c); // sum floor((a*i+b)/c), l<=i<=r
6 //   0<=l,r,a,b; c>0; return __int128, answer must fit
7 //
8 __int128 floor_sum(ll l,ll r,ll a,ll b,ll c){
9   assert(l>=0&&r>=0&&a>=0&&b>=0&&c>0);
10  if(l>r)return 0;
11  __int128 n=((__int128)r-l+1,A=a,B=((__int128)a*l+b,C=c,ans=0;
12  while(1){
13    ans+=n*(n-1)/2*(A/C)+n*(B/C);
14    A%=C;B%=C;
15    __int128 y=A*n+B;
16    if(y<C)break;
17    n=y/C;B=y%C;swap(A,C);
18  }
19  return ans;
20 }
```

5.5 number theory/du jiao sieve.cpp

μ 、 φ 前缀和

什么时候用

预筛小范围并缓存整除分块递归, 分别查询 $\sum \mu(i)$ 、 $\sum \varphi(i)$, $i=1..n$ 。

调用顺序与接口

- `du_jiao d`; `d.init_mu(B)` 后调用 `sum_mu(n)`, 返回 `ll`。
- `init_phi(B)` 后调用 `sum_phi(n)`, 返回 `__int128`。
- `B>=1`, `n>=0`; 两个初始化和缓存相互独立, 可只使用一个。

最小示例（放入 main）：

```

1 du_jiao d; d.init_mu(3);
2 assert(d.sum_mu(10)==-1);
3 d.init_phi(3); assert(d.sum_phi(10)==32);
4 assert(d.sum_mu(0)==0 && d.sum_phi(0)==0);

```

使用边界

必须先初始化对应函数。B 是预处理上界而非查询上限，常从 $n^{2/3}$ 附近结合内存选择；预筛 $O(B)$ ，递归性能随 B 和查询集改变。重复初始化会清空该项缓存。

完整代码

```

1 //
2 // template for Du Jiao Sieve
3 //
4 // usage:
5 //   du_jiao d; d.init_mu(B); ll mu=d.sum_mu(n);
6 //   d.init_phi(B); __int128 phi=d.sum_phi(n);
7 //   sum over [1,n], n>=0; init only what is needed
8 //   B ~ n^(2/3); O(B+n/sqrt(B)) time, O(B+n/B) space
9 //
10 struct du_jiao{
11     vector<int> mu;
12     vector<ll> phi;
13     unordered_map<ll,ll> mem_mu;
14     unordered_map<ll,__int128> mem_phi;
15     void init_mu(int m){
16         assert(m>=1&&m<INT_MAX);
17         vector<int> prime;vector<bool> vis(m+1,false);
18         mu.assign(m+1,0);mem_mu.clear();mu[1]=1;
19         for(int i=2;i<=m;i++){
20             if(!vis[i])prime.push_back(i),mu[i]=-1;
21             for(int p:prime){
22                 if(p>m/i)break;
23                 int x=i*p;vis[x]=true;
24                 if(i%p==0)break;
25                 mu[x]=-mu[i];
26             }
27         }
28         for(int i=1;i<=m;i++)mu[i]+=mu[i-1];
29     }
30     void init_phi(int m){
31         assert(m>=1&&m<INT_MAX);
32         vector<int> prime;vector<bool> vis(m+1,false);
33         phi.assign(m+1,0);mem_phi.clear();phi[1]=1;
34         for(int i=2;i<=m;i++){
35             if(!vis[i])prime.push_back(i),phi[i]=i-1;
36             for(int p:prime){
37                 if(p>m/i)break;
38                 int x=i*p;vis[x]=true;
39                 if(i%p==0){phi[x]=phi[i]*p;break;}
40                 phi[x]=phi[i]*(p-1);
41             }
42         }
43         for(int i=1;i<=m;i++)phi[i]+=phi[i-1];
44     }

```

```
45 ll sum_mu(ll n){
46     assert(n>=0&&!mu.empty());
47     if(n<(ll)mu.size())return mu[n];
48     auto it=mem_mu.find(n);
49     if(it!=mem_mu.end())return it->second;
50     __int128 ans=1;
51     for(ll l=2,r;l<=n;){
52         r=n/(n/l);ans-=(__int128)(r-l+1)*sum_mu(n/l);
53         if(r==n)break;
54         l=r+1;
55     }
56     return mem_mu[n]=(ll)ans;
57 }
58 __int128 sum_phi(ll n){
59     assert(n>=0&&!phi.empty());
60     if(n<(ll)phi.size())return phi[n];
61     auto it=mem_phi.find(n);
62     if(it!=mem_phi.end())return it->second;
63     __int128 ans=(__int128)n*(n+(__int128)1)/2;
64     for(ll l=2,r;l<=n;){
65         r=n/(n/l);ans-=(__int128)(r-l+1)*sum_phi(n/l);
66         if(r==n)break;
67         l=r+1;
68     }
69     return mem_phi[n]=ans;
70 }
71 };
```

6 多项式

6.1 polynomial/poly-fast.cpp

998244353 下的常用多项式

什么时候用

适用于固定模数下的大规模卷积与形式幂级数。卷积 $O(n \log n)$ ，BM 和普通插值为二次复杂度。

调用顺序与接口

- `poly f={a0,a1,...}` 表示 $a_0+a_1x+\dots$ ，下标就是次数。
- `f*g` 是卷积；加减和数乘也支持。`Inv(f)` 返回模 x^n 的倒数， $n=f.size()$ ，`f[0]` 必须非零。
- `Ln(f)` 要求 `f[0]=1`；`Exp(f)` 要求 `f[0]=0`；返回长度与输入相同。
- `diff/integ` 求导/积分，积分常数取零；`value(f,x)` 是单点求值。
- `BM(seq)` 求线性递推的分母多项式，`FSPE(F,G,k)` 求 F/G 的第 k 项，`RSPE(seq,k)` 从已有递推样本推第 k 项， k 从 0 开始。

最小示例（放入 `main`）：

```
1 poly f{1,1,0,0};
2 auto g=Inv(f); auto h=f*g; h.resize(4);
3 assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
4 auto e=Exp(Ln(f));
5 for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
6 assert(FSPE({0,1},{1,-1,-1},10).x==55);
```

使用边界

精度由输入 vector 长度决定，想求前 n 项就先 `resize(n)`，不要删掉需要保留的高位零。`operator/` 是相关运算 $(f/g)[i]=\sum_j f[i+j]*g[j]$ ，不是多项式除法。形式幂级数相除要 `f*Inv(g)` 后截断。各多项式文件均内置模数类型和全局表，只选一份，不要再粘贴 `ModInt`。NTT 长度不超过 2^{23} ，`Inv/Ln/Exp` 的中间卷积也计入这个限制。`lagrange(x,y)` 要求取值点互异，允许零点。

考场版：本节已换成普通迭代 NTT；fft 输出按自然顺序排列，`invFft` 接受同样顺序。不能与原版变换内核混搭。

完整代码

```
1 //
2 // template for ModInt
3 // version 3.0 (Last Update Jun 15th, 2026)
4 //
5 // usage:
6 //   init(n); mint a,b; C(n,k); binom(n,k);
7 //
8 template <unsigned _M> struct ModInt{
9     static constexpr unsigned MOD=_M;
10    static_assert(1<MOD&&MOD<=INT_MAX);
11    unsigned x;
12    constexpr ModInt():x(0){}
13    constexpr ModInt(unsigned y):x(y%MOD){}
14    constexpr ModInt(int y):x((y%=static_cast<int>(MOD))<0?y+MOD:y){}
15    constexpr ModInt(unsigned long long y):x(y%MOD){}
16    constexpr ModInt(long long y):x((y%=static_cast<long long>(MOD))<0?y+MOD:y){}
17    ModInt& operator +=(const ModInt &a){x=((x+=a.x)>=MOD)?x-MOD:x;return *this;}
18    ModInt& operator -=(const ModInt &a){x=((x-=a.x)>=MOD)?x+MOD:x;return *this;}
19    ModInt& operator *=(const ModInt &a){x=static_cast<unsigned long long>(x)*a.x%MOD;return *this;}
```

```

20 ModInt& operator /=(const ModInt &a){return (*this)*=a.inv();}
21 bool operator ==(const ModInt &a)const{return x==a.x;}
22 bool operator !=(const ModInt &a)const{return x!=a.x;}
23 explicit operator bool() const { return x != 0; }
24 bool operator!() const { return x == 0; }
25 ModInt inv() const {
26     unsigned a=MOD,b=x;ll y=0,z=1;
27     while(b){
28         const unsigned q=a/b,c=a-q*b;
29         a=b,b=c;
30         const ll w=y-(ll)q*z;
31         y=z,z=w;
32     }
33     assert(a==1);return ModInt(y);
34 }
35 ModInt pow(long long k) const {
36     unsigned long long e=k;
37     ModInt a=*this,b=1;
38     if(k<0){a=a.inv();e=0-e;}
39     for(;e>=1){if(e&1)b*=a;a*=a;}
40     return b;
41 }
42 ModInt& operator++() {*this+=1;return *this;}
43 ModInt& operator--() {*this-=1;return *this;}
44 ModInt operator +(const ModInt &a)const{return *this;}
45 ModInt operator -(const ModInt &a)const{ModInt a;a.x=(x?MOD-x:0u);return a;}
46 ModInt operator +(const ModInt &a)const{return ModInt(*this)+a;}
47 ModInt operator -(const ModInt &a)const{return ModInt(*this)-a;}
48 ModInt operator *(const ModInt &a)const{return ModInt(*this)*a;}
49 ModInt operator /(const ModInt &a)const{return ModInt(*this)/a;}
50
51 template<typename T> ModInt friend operator +(T a,const ModInt &b){return ModInt(a)+b;}
52 template<typename T> ModInt friend operator -(T a,const ModInt &b){return ModInt(a)-b;}
53 template<typename T> ModInt friend operator *(T a,const ModInt &b){return ModInt(a)*b;}
54 template<typename T> ModInt friend operator /(T a,const ModInt &b){return ModInt(a)/b;}
55 friend istream& operator >> (istream &i,ModInt &x){ll y;if(i>>y)x=ModInt(y);return i;}
56 friend ostream& operator << (ostream &o,const ModInt &x){o<<x.x;return o;}
57 };
58
59 const int MOD=998244353;
60 using mint = ModInt<MOD>;
61 vector<mint> fac{1},ifac{1},inv{0,1};
62 void init(int n=0){
63     assert(0<=n&&n<MOD);
64     int m=fac.size();if(n<m)return;
65     n=max(n,(int)min(2ll*m,(ll)MOD-1));
66     fac.resize(n+1);ifac.resize(n+1);inv.resize(max(n+1,2));
67     for(int i=m;i<=n;i++){
68         fac[i]=fac[i-1]*i;
69         if(i>1)inv[i]=-(MOD/i)*inv[MOD%i];
70         ifac[i]=ifac[i-1]*inv[i];
71     }
72 }
73 inline mint C(int x,int y){
74     // choose x from y

```

```

75     if(x<0||y<x)return 0;
76     init(y);return fac[y]*ifac[x]*ifac[y-x];
77 }
78 inline mint binom(int y,int x){return C(x,y);}
79
80
81 //
82 // template for polynomial
83 // version 7.0 (Last Update Jun 15th, 2026)
84 // NTT core replaced with the faster version
85 //
86 // usage:
87 //     init(n);
88 //     poly h=f*g; Inv, Ln, Exp, BM, FSPE, value;
89 //
90
91 using ll=long long;
92 using poly = vector<mint>;
93
94 constexpr unsigned MO = 998244353U;
95 constexpr unsigned MO2 = 2U * MO;
96 constexpr int FFT_MAX=23;
97 void ntt(mint *a,int n,bool inverse){
98     assert(1<=n&&n<=(1<<FFT_MAX)&&!(n&(n-1)));
99     for(int i=1,j=0;i<n;i++){
100         int k=n>>1;
101         for(;j&k;k>>=1)j^=k;
102         j^=k;if(i<j)swap(a[i],a[j]);
103     }
104     for(int len=2;len<=n;len<<=1){
105         mint step=mint(3).pow((MOD-1)/len);
106         if(inverse)step=step.inv();
107         for(int i=0;i<n;i+=len){
108             mint cur=1;
109             for(int j=0;j<len/2;j++,cur*=step){
110                 mint x=a[i+j],y=a[i+j+len/2]*cur;
111                 a[i+j]=x+y;a[i+j+len/2]=x-y;
112             }
113         }
114     }
115     if(inverse){mint v=mint(n).inv();for(int i=0;i<n;i++)a[i]*=v;}
116 }
117 void fft(mint *a,int n){ntt(a,n,false);}
118 void invFft(mint *a,int n){ntt(a,n,true);}
119
120 void fft(poly &as) {
121     fft(as.data(), as.size());
122 }
123 void invFft(poly &as) {
124     invFft(as.data(), as.size());
125 }
126
127 // -----
128
129 poly operator *(poly f, poly g) {

```

```

130  if(f.empty()||g.empty())return {};
131  assert(f.size()+g.size()-1<=(1<<FFT_MAX));
132  int n = f.size() + g.size();
133  int l = 0; while ((1 << l) < n-1) ++l;
134  f.resize(1 << l); g.resize(1 << l);
135  fft(f); fft(g);
136  for (int i = 0; i < (1 << l); i++) f[i] *= g[i];
137  invFft(f);
138  f.resize(n - 1);
139  return f;
140 }
141
142 poly operator /(poly f, poly g) {
143     if(f.empty()||g.empty())return poly(f.size());
144     int m = g.size();
145     reverse(g.begin(), g.end());
146     g = f * g;
147     for (int i = 0; i < (int)f.size(); i++) f[i] = g[i + m - 1];
148     return f;
149 }
150
151 void operator +=(poly &f, const poly &g) {
152     if (f.size() < g.size()) f.resize(g.size());
153     for (int i = 0; i < (int)g.size(); i++) f[i] += g[i];
154 }
155
156 poly operator +(poly f, const poly &g) { f += g; return f; }
157
158 void operator -=(poly &f, const poly &g) {
159     if (f.size() < g.size()) f.resize(g.size());
160     for (int i = 0; i < (int)g.size(); i++) f[i] -= g[i];
161 }
162
163 poly operator -(poly f, const poly &g) { f -= g; return f; }
164
165 poly operator *(poly f, mint x) {
166     for (mint &i : f) i *= x;
167     return f;
168 }
169
170 poly operator *(mint x, poly f) {
171     for (mint &i : f) i *= x;
172     return f;
173 }
174
175 mint value(const poly &f,mint x){
176     mint ans=0;
177     for(int i=f.size()-1;i>=0;i--) ans=ans*x+f[i];
178     return ans;
179 }
180
181 poly Inv(poly f) {
182     assert(!f.empty()&&f[0]);
183     int n = f.size();
184     int l = 0; while ((1 << l) < n) ++l;

```

```

185     f.resize(1 << 1);
186     poly g{f[0].inv()}, _f;
187     for (int i = 1; i <= 1; i++) {
188         _f = poly(begin(f), begin(f) + (1 << i));
189         g.resize(1 << (i+1)); _f.resize(1 << (i+1));
190         fft(g); fft(_f);
191         for (int j = 0; j < (1 << (i+1)); j++)
192             g[j] = mint(2) * g[j] - g[j] * g[j] * _f[j];
193         invFft(g);
194         fill(begin(g) + (1 << i), end(g), 0);
195     }
196     g.resize(n);
197     return g;
198 }
199
200 poly integ(poly f) {
201     int n = f.size(); ::init(n); f.resize(n + 1);
202     for (int i = n; i >= 1; i--) f[i] = f[i - 1] * ::inv[i];
203     f[0] = 0;
204     return f;
205 }
206
207 poly diff(poly f) {
208     if(f.empty()) return {};
209     int n = f.size();
210     for (int i = 0; i < n - 1; i++)
211         f[i] = f[i + 1] * (i + 1);
212     f.pop_back();
213     return f;
214 }
215
216 poly Ln(poly f) {
217     assert(!f.empty() && f[0].x==1);
218     poly f_ = diff(f), _f = Inv(f);
219     int n = f_.size(), m = _f.size();
220     int l = 0; while ((1 << l) < n + m) ++l;
221     f_.resize(1 << l); _f.resize(1 << l);
222     fft(f_); fft(_f);
223     for (int i = 0; i < (1 << l); i++) f_[i] *= _f[i];
224     invFft(f_);
225     f_ = integ(f_);
226     f_.resize(f.size());
227     return f_;
228 }
229
230 poly Exp(poly f) {
231     assert(!f.empty() && !f[0]);
232     poly g{1}, _f, _g;
233     int n = f.size();
234     int l = 0; while ((1 << l) < n) ++l;
235     f.resize(1 << l);
236     for (int i = 1; i <= l; i++) {
237         _f = poly(begin(f), begin(f) + (1 << i));
238         _g = Ln(g);
239         g.resize(1 << (i+1)); _f.resize(1 << (i+1)); _g.resize(1 << (i+1));

```

```

240     fft(g); fft(_f); fft(_g);
241     for (int j = 0; j < (1 << (i+1)); j++)
242         g[j] *= mint(1) - _g[j] + _f[j];
243     invFft(g);
244     fill(begin(g) + (1 << i), end(g), 0);
245 }
246 g.resize(n);
247 return g;
248 }
249
250 poly BM(poly a) {
251     poly C{1}, B{1};
252     int L = 0, m = 1;
253     mint b = 1;
254     for (int n = 0; n < (int)a.size(); n++) {
255         mint d = 0;
256         for (int i = 0; i <= L; i++) d += C[i] * a[n - i];
257         if (!d) {
258             m++;
259         } else {
260             poly T = C;
261             mint coef = d * b.inv();
262             poly xmB(m, 0);
263             for (mint val : B) xmB.push_back(val);
264             if (C.size() < xmB.size()) C.resize(xmB.size(), 0);
265             for (int i = 0; i < (int)xmB.size(); i++) C[i] -= coef * xmB[i];
266             if (2 * L <= n) {
267                 L = n + 1 - L; B = T; b = d; m = 1;
268             } else {
269                 m++;
270             }
271         }
272     }
273     return C;
274 }
275
276 mint FSPE(poly F, poly G, ll t) {
277     // find  $[x^t] F/G$ 
278     assert(t >= 0 && !G.empty() && G[0]);
279     if(F.empty()) return 0;
280     while (t) {
281         poly G1 = G;
282         for (int i = 1; i < (int)G1.size(); i += 2) G1[i] = -G1[i];
283         F = F * G1; G = G * G1;
284         poly f, g;
285         for (int i = t & 1; i < (int)F.size(); i += 2) f.push_back(F[i]);
286         for (int i = 0; i < (int)G.size(); i += 2) g.push_back(G[i]);
287         t >>= 1;
288         F = f; G = g;
289         if(F.empty()) return 0;
290     }
291     return F[0] / G[0];
292 }
293
294 mint RSPE(poly f, ll x) {

```

```

295 // find f[x] in  $O(n^2)$ , note that f must be recursion
296 poly g = BM(f);
297 poly s(g.size()-1);
298 for (int i = 0; i < (int)s.size(); i++)
299     for (int j = 0; j <= i && j < (int)f.size(); j++)
300         s[i] += f[j] * g[i - j];
301 return FSPE(s, g, x);
302 }
303
304 poly lagrange(vector<mint> x,vector<mint> y){
305     int n=x.size();assert(y.size()==x.size());
306     if(!n)return {};
307     poly g(n+1),q(n),f(n);g[0]=1;
308     for(int i=0;i<n;i++){
309         for(int j=i+1;j>=1;j--)g[j]=g[j-1]-x[i]*g[j];
310         g[0]*=-x[i];
311     }
312     for(int i=0;i<n;i++){
313         q[n-1]=g[n];
314         for(int j=n-2;j>=0;j--)q[j]=g[j+1]+x[i]*q[j+1];
315         mint d=value(q,x[i]);assert(d);
316         mint c=y[i]/d;for(int j=0;j<n;j++)f[j]+=q[j]*c;
317     }
318     return f;
319 }
320 // use init(n) before accessing fac / ifac / inv directly

```

6.2 polynomial/poly-new.cpp

带长除、多点求值的多项式

什么时候用

同样固定模数 998244353，提供常用幂级数和 Sqrt、Div、Eval 等接口。

调用顺序与接口

- `poly f={a0,a1,...}` 表示 $a_0+a_1x+\dots$ ，下标就是次数。
- `f*g` 是卷积；加减和数乘也支持。`Inv(f)` 返回模 x^n 的倒数， $n=f.size()$ ，`f[0]` 必须非零。
- `Ln(f)` 要求 `f[0]=1`；`Exp(f)` 要求 `f[0]=0`；返回长度与输入相同。
- `diff/integ` 求导/积分，积分常数取零；`value(f,x)` 是单点求值。
- `BM(seq)` 求线性递推的分母多项式，`FSPE(F,G,k)` 求 F/G 的第 k 项，`RSPE(seq,k)` 从已有递推样本推第 k 项， k 从 0 开始。
- `Sqrt(f)` 当前仅接受 `f[0]=1`，取常数项为 1 的根。
- `Div(f,g,q,r)` 得到多项式长除的商与余式， g 最高次系数非零。`Eval(f,xs)` 返回各取值点的函数值。

最小示例（放入 main）：

```

1 poly f{1,1,0,0};
2 auto g=Inv(f); auto h=f*g; h.resize(4);
3 assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
4 auto e=Exp(Ln(f));
5 for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
6 assert(FSPE({0,1},{1,-1,-1},10).x==55);

```

```

7 poly q,r; Div({1,2,1},{1,1},q,r);
8 assert(q.size()==2 && q[0].x==1 && q[1].x==1);
9 auto v=Eval({1,2,1},{0,1,2});
10 assert(v[0].x==1 && v[1].x==4 && v[2].x==9);

```

使用边界

精度由输入 vector 长度决定，想求前 n 项就先 `resize(n)`，不要删掉需要保留的高位零。`operator/` 是相关运算 $(f/g)[i]=\sum_j f[i+j]*g[j]$ ，不是多项式除法。形式幂级数相除要 `f*Inv(g)` 后截断。各多项式文件均内置模数类型和全局表，只选一份，不要再粘贴 `ModInt`。不支持任意常数项的模平方根。长除前清除除数尾部多余的零。`lagrange` 要求互异取值点，允许零点。NTT 上限为 2^{23} 。

考场版：本节已换成普通迭代 NTT；`fft` 输出按自然顺序排列，`invFft` 接受同样顺序。不能与原版变换内核混搭。

完整代码

```

1 //
2 // template for ModInt, prepared by Otomachi Una
3 // version 3.0 (Last Update Jun 15th, 2026)
4 //
5 // usage:
6 //   init(n); mint a,b; poly f={...};
7 //
8 template <unsigned _M> struct ModInt{
9     static constexpr unsigned MOD=_M;
10    static_assert(1<MOD&&MOD<=INT_MAX);
11    unsigned x;
12    constexpr ModInt():x(0){}
13    constexpr ModInt(unsigned y):x(y%MOD){}
14    constexpr ModInt(int y):x((y%=static_cast<int>(MOD))<0?y+MOD:y){}
15    constexpr ModInt(unsigned long long y):x(y%MOD){}
16    constexpr ModInt(long long y):x((y%=static_cast<long long>(MOD))<0?y+MOD:y){}
17    ModInt& operator +=(const ModInt &a){x=((x+=a.x)>=MOD)?x-MOD:x;return *this;}
18    ModInt& operator -=(const ModInt &a){x=((x-=a.x)>=MOD)?x+MOD:x;return *this;}
19    ModInt& operator *=(const ModInt &a){x=static_cast<unsigned long long>(x)*a.x%MOD;return *this;}
20    ModInt& operator /=(const ModInt &a){return (*this)*=a.inv();}
21    bool operator ==(const ModInt &a)const{return x==a.x;}
22    bool operator !=(const ModInt &a)const{return x!=a.x;}
23    explicit operator bool() const { return x != 0; }
24    bool operator!() const { return x == 0; }
25    ModInt inv() const {
26        unsigned a=MOD,b=x;ll y=0,z=1;
27        while(b){
28            const unsigned q=a/b,c=a-q*b;
29            a=b,b=c;
30            const ll w=y-(ll)q*z;
31            y=z,z=w;
32        }
33        assert(a==1);return ModInt(y);
34    }
35    ModInt pow(long long k) const {
36        unsigned long long e=k;
37        ModInt a=*this,b=1;
38        if(k<0){a=a.inv();e=0-e;}
39        for(;;e>>=1){if(e&1)b*=a;a*=a;}
40        return b;
41    }

```

```

42 ModInt& operator++() { *this+=1; return *this; }
43 ModInt& operator--() { *this-=1; return *this; }
44 ModInt operator +()const { return *this; }
45 ModInt operator -()const { ModInt a; a.x=(x?MOD-x:0u); return a; }
46 ModInt operator +(const ModInt &a)const { return ModInt(*this)+=a; }
47 ModInt operator -(const ModInt &a)const { return ModInt(*this)-=a; }
48 ModInt operator *(const ModInt &a)const { return ModInt(*this)*=a; }
49 ModInt operator /(const ModInt &a)const { return ModInt(*this)/=a; }
50
51 template<typename T> ModInt friend operator +(T a, const ModInt &b) { return ModInt(a)+=b; }
52 template<typename T> ModInt friend operator -(T a, const ModInt &b) { return ModInt(a)-=b; }
53 template<typename T> ModInt friend operator *(T a, const ModInt &b) { return ModInt(a)*=b; }
54 template<typename T> ModInt friend operator /(T a, const ModInt &b) { return ModInt(a)/=b; }
55 friend istream& operator >> (istream &i, ModInt &x) { ll y; if (i>>y) x=ModInt(y); return i; }
56 friend ostream& operator << (ostream &o, const ModInt &x) { o<<x.x; return o; }
57 };
58
59 const int MOD=998244353;
60 using mint = ModInt<MOD>;
61 vector<mint> fac{1}, ifac{1}, inv{0,1};
62 void init(int n=0) {
63     assert(0<=n&&n<MOD);
64     int m=fac.size(); if (n<m) return;
65     n=max(n, (int)min(2ll*m, (ll)MOD-1));
66     fac.resize(n+1); ifac.resize(n+1); inv.resize(max(n+1, 2));
67     for (int i=m; i<=n; i++) {
68         fac[i]=fac[i-1]*i;
69         if (i>1) inv[i]=-(MOD/i)*inv[MOD%i];
70         ifac[i]=ifac[i-1]*inv[i];
71     }
72 }
73 inline mint C(int x, int y) {
74     // choose x from y
75     if (x<0 || y<x) return 0;
76     init(y); return fac[y]*ifac[x]*ifac[y-x];
77 }
78 inline mint binom(int y, int x) { return C(x, y); }
79
80
81 //
82 // template for polynomial, prepared by Otomachi Una
83 // version 6.1 (Last Update Jun 13th, 2026)
84 // NTT core replaced with the "tech" version
85 //
86 // usage:
87 //     init(n);
88 //     Inv, Ln, Exp, Div, Sqrt, Eval, BM;
89 //
90
91 using poly = vector<mint>;
92
93 // ----- NTT constants and roots (tech version) -----
94 constexpr unsigned M0 = 998244353U;
95 constexpr unsigned M02 = 2U * M0;
96 constexpr int FFT_MAX=23;

```

```

97 void ntt(mint *a,int n,bool inverse){
98     assert(1<=n&&n<=(1<<FFT_MAX)&&!(n&(n-1)));
99     for(int i=1,j=0;i<n;i++){
100         int k=n>>1;
101         for(;j&k;k>>=1)j^=k;
102         j^=k;if(i<j)swap(a[i],a[j]);
103     }
104     for(int len=2;len<=n;len<<=1){
105         mint step=mint(3).pow((MOD-1)/len);
106         if(inverse)step=step.inv();
107         for(int i=0;i<n;i+=len){
108             mint cur=1;
109             for(int j=0;j<len/2;j++,cur*=step){
110                 mint x=a[i+j],y=a[i+j+len/2]*cur;
111                 a[i+j]=x+y;a[i+j+len/2]=x-y;
112             }
113         }
114     }
115     if(inverse){mint v=mint(n).inv();for(int i=0;i<n;i++)a[i]*=v;}
116 }
117 void fft(mint *a,int n){ntt(a,n,false);}
118 void invFft(mint *a,int n){ntt(a,n,true);}
119
120 void fft(poly &as) {
121     fft(as.data(), as.size());
122 }
123 void invFft(poly &as) {
124     invFft(as.data(), as.size());
125 }
126
127 // -----
128
129 poly operator *(poly f, poly g) {
130     if(f.empty()||g.empty())return {};
131     assert(f.size()+g.size()-1<=(1<<FFT_MAX));
132     int n = f.size() + g.size();
133     int l = 0; while ((1 << l) < n-1) ++l;
134     f.resize(1 << l); g.resize(1 << l);
135     fft(f); fft(g);
136     for (int i = 0; i < (1 << l); i++) f[i] *= g[i];
137     invFft(f);
138     f.resize(n - 1);
139     return f;
140 }
141
142 poly operator /(poly f, poly g) {
143     if(f.empty()||g.empty())return poly(f.size());
144     int m = g.size();
145     reverse(g.begin(), g.end());
146     g = f * g;
147     for (int i = 0; i < (int)f.size(); i++) f[i] = g[i + m - 1];
148     return f;
149 }
150
151 void operator +=(poly &f, const poly &g) {

```

```

152   if (f.size() < g.size()) f.resize(g.size());
153   for (int i = 0; i < (int)g.size(); i++) f[i] += g[i];
154 }
155
156 poly operator +(poly f, const poly &g) { f += g; return f; }
157
158 void operator -=(poly &f, const poly &g) {
159     if (f.size() < g.size()) f.resize(g.size());
160     for (int i = 0; i < (int)g.size(); i++) f[i] -= g[i];
161 }
162
163 poly operator -(poly f, const poly &g) { f -= g; return f; }
164
165 poly operator *(poly f, mint x) {
166     for (mint &i : f) i *= x;
167     return f;
168 }
169
170 poly operator *(mint x, poly f) {
171     for (mint &i : f) i *= x;
172     return f;
173 }
174
175 mint value(const poly &f, mint x){
176     mint ans=0;
177     for(int i=f.size()-1;i>=0;i--) ans=ans*x+f[i];
178     return ans;
179 }
180
181 poly Inv(poly f) {
182     assert(!f.empty()&&f[0]);
183     int n = f.size();
184     int l = 0; while ((1 << l) < n) ++l;
185     f.resize(1 << l);
186     poly g{f[0].inv()}, _f;
187     for (int i = 1; i <= l; i++) {
188         _f = poly(begin(f), begin(f) + (1 << i));
189         g.resize(1 << (i+1)); _f.resize(1 << (i+1));
190         fft(g); fft(_f);
191         for (int j = 0; j < (1 << (i+1)); j++)
192             g[j] = mint(2) * g[j] - g[j] * g[j] * _f[j];
193         invFft(g);
194         fill(begin(g) + (1 << i), end(g), 0);
195     }
196     g.resize(n);
197     return g;
198 }
199
200 poly integ(poly f) {
201     int n = f.size(); ::init(n);f.resize(n + 1);
202     for (int i = n; i >= 1; i--) f[i] = f[i - 1] * ::inv[i];
203     f[0] = 0;
204     return f;
205 }
206

```

```

207 poly diff(poly f) {
208     if(f.empty())return {};
209     int n = f.size();
210     for (int i = 0; i < n - 1; i++)
211         f[i] = f[i + 1] * (i + 1);
212     f.pop_back();
213     return f;
214 }
215
216 poly Ln(poly f) {
217     assert(!f.empty() && f[0].x==1);
218     poly f_ = diff(f), _f = Inv(f);
219     int n = f_.size(), m = _f.size();
220     int l = 0; while ((1 << l) < n + m) ++l;
221     f_.resize(1 << l); _f.resize(1 << l);
222     fft(f_); fft(_f);
223     for (int i = 0; i < (1 << l); i++) f_[i] *= _f[i];
224     invFft(f_);
225     f_ = integ(f_);
226     f_.resize(f.size());
227     return f_;
228 }
229
230 poly Exp(poly f) {
231     assert(!f.empty() && !f[0]);
232     poly g{1}, _f, _g;
233     int n = f.size();
234     int l = 0; while ((1 << l) < n) ++l;
235     f.resize(1 << l);
236     for (int i = 1; i <= l; i++) {
237         _f = poly(begin(f), begin(f) + (1 << i));
238         _g = Ln(g);
239         g.resize(1 << (i+1)); _f.resize(1 << (i+1)); _g.resize(1 << (i+1));
240         fft(g); fft(_f); fft(_g);
241         for (int j = 0; j < (1 << (i+1)); j++)
242             g[j] *= mint(1) - _g[j] + _f[j];
243         invFft(g);
244         fill(begin(g) + (1 << i), end(g), 0);
245     }
246     g.resize(n);
247     return g;
248 }
249
250 poly Sqrt(poly f) {
251     assert(!f.empty() && f[0].x==1);
252     poly g{1}, _f, _g;
253     int n = f.size();
254     int l = 0; while ((1 << l) < n) ++l;
255     f.resize(1 << l);
256     mint inv2 = mint(2).inv();
257     for (int i = 1; i <= l; i++) {
258         _f = poly(begin(f), begin(f) + (1 << i));
259         _g = Inv(g);
260         g.resize(1 << (i+1)); _f.resize(1 << (i+1)); _g.resize(1 << (i+1));
261         fft(_f); fft(_g); fft(g);

```

```

262     for (int j = 0; j < (1 << (i+1)); j++)
263         g[j] = (_f[j] + g[j] * g[j]) * inv2 * _g[j];
264     invFft(g);
265     fill(begin(g) + (1 << i), end(g), 0);
266 }
267 g.resize(n);
268 return g;
269 }
270
271 void Div(poly f, poly g, poly &q, poly &r) {
272     assert(!g.empty() && g.back());
273     if(f.size() < g.size()){q={0};r=f;return;}
274     int n = f.size() - 1, m = g.size() - 1;
275     reverse(f.begin(), f.end());
276     reverse(g.begin(), g.end());
277     g.resize(n + 1);
278     q = f * Inv(g);
279     q.resize(n - m + 1);
280     reverse(q.begin(), q.end());
281     g.resize(m + 1);
282     reverse(g.begin(), g.end());
283     reverse(f.begin(), f.end());
284     poly h = q * g;
285     r.resize(m);
286     for (int i = 0; i < m; i++) r[i] = f[i] - h[i];
287 }
288
289 poly Eval(poly f, poly x) {
290     if(x.empty())return {};
291     vector<poly> func;
292     map<pair<int, int>, int> mp;
293     function<poly(int, int)> prod = [&](int l, int r) {
294         poly ans;
295         if (l == r) ans = poly{-x[l], 1};
296         else {
297             int mid = (l + r) >> 1;
298             ans = prod(l, mid) * prod(mid + 1, r);
299         }
300         func.push_back(ans);
301         mp[{l, r}] = func.size() - 1;
302         return ans;
303     };
304     prod(0, x.size() - 1);
305
306     function<poly(poly, int, int)> solve = [&](poly f, int l, int r) {
307         if (l == r) {
308             mint val = 0;
309             for (int i = f.size() - 1; i >= 0; i--)
310                 val = val * x[l] + f[i];
311             return poly{val};
312         }
313         int mid = (l + r) >> 1;
314         poly p0 = func[mp[{l, mid}]], p1 = func[mp[{mid + 1, r}]];
315         poly d, q;
316         Div(f, p0, d, q);

```

```

317     poly ans = solve(q, l, mid);
318     Div(f, p1, d, q);
319     poly _ans = solve(q, mid + 1, r);
320     for (mint i : _ans) ans.push_back(i);
321     return ans;
322 };
323 return solve(f, 0, x.size() - 1);
324 }
325
326 poly BM(poly a) {
327     poly C{1}, B{1};
328     int L = 0, m = 1;
329     mint b = 1;
330     for (int n = 0; n < (int)a.size(); n++) {
331         mint d = 0;
332         for (int i = 0; i <= L; i++) d += C[i] * a[n - i];
333         if (!d) {
334             m++;
335         } else {
336             poly T = C;
337             mint coef = d * b.inv();
338             poly xB(m, 0);
339             for (mint val : B) xB.push_back(val);
340             if (C.size() < xB.size()) C.resize(xB.size(), 0);
341             for (int i = 0; i < (int)xB.size(); i++) C[i] -= coef * xB[i];
342             if (2 * L <= n) {
343                 L = n + 1 - L; B = T; b = d; m = 1;
344             } else {
345                 m++;
346             }
347         }
348     }
349     return C;
350 }
351
352 mint FSPE(poly F, poly G, ll t) {
353     // find  $[x^t] F/G$ 
354     assert(t >= 0 && !G.empty() && G[0]);
355     if (F.empty()) return 0;
356     while (t) {
357         poly G1 = G;
358         for (int i = 1; i < (int)G1.size(); i += 2) G1[i] = -G1[i];
359         F = F * G1; G = G * G1;
360         poly f, g;
361         for (int i = t & 1; i < (int)F.size(); i += 2) f.push_back(F[i]);
362         for (int i = 0; i < (int)G.size(); i += 2) g.push_back(G[i]);
363         t >>= 1;
364         F = f; G = g;
365         if (F.empty()) return 0;
366     }
367     return F[0] / G[0];
368 }
369
370 mint RSPE(poly f, ll x) {
371     // find  $f[x]$  in  $O(n^2)$ , note that f must be recursion

```

```

372 poly g = BM(f);
373 poly s(g.size()-1);
374 for (int i = 0; i < (int)s.size(); i++)
375     for (int j = 0; j <= i && j < (int)f.size(); j++)
376         s[i] += f[j] * g[i - j];
377 return FSPE(s, g, x);
378 }
379
380 poly lagrange(vector<mint> x,vector<mint> y){
381     int n=x.size();assert(y.size()==x.size());
382     if(!n)return {};
383     poly g(n+1),q(n),f(n);g[0]=1;
384     for(int i=0;i<n;i++){
385         for(int j=i+1;j>=1;j--)g[j]=g[j-1]-x[i]*g[j];
386         g[0]*=-x[i];
387     }
388     for(int i=0;i<n;i++){
389         q[n-1]=g[n];
390         for(int j=n-2;j>=0;j--)q[j]=g[j+1]+x[i]*q[j+1];
391         mint d=value(q,x[i]);assert(d);
392         mint c=y[i]/d;for(int j=0;j<n;j++)f[j]+=q[j]*c;
393     }
394     return f;
395 }
396
397 poly to_ex(poly f){ // f(e^x)
398     int n=f.size();if(!n)return {};::init(n);
399     function<pair<poly,poly>(int,int)> solve=[&](int l,int r){
400         if(l==r) return make_pair(poly{f[l]},poly{1,MOD-1});
401         int mid=l+r>>1;
402         auto ls=solve(l,mid),rs=solve(mid+1,r);
403         return make_pair(ls.first*rs.second+ls.second*rs.first,ls.second*rs.second);
404     };
405     auto ans=solve(0,n-1);
406     poly g=ans.first*Inv(ans.second);
407     g.resize(n);
408     for(int i=0;i<n;i++) g[i]=g[i]*ifac[i];
409     return g;
410 }
411
412 poly S2line(int n){
413     // return S(n,i)
414     assert(n>=0);::init(n);
415     poly F(n+1),G(n+1);
416     // S(n,i) * binom(j,i) -> F(j)
417     for(int i=0;i<=n;i++){
418         G[i]=ifac[i];F[i]=mint(i).pow(n)*ifac[i];
419         if(i&1) F[i]*=-1;
420     }
421     F=F*G;
422     F.resize(n+1);
423     for(int i=0;i<=n;i++){
424         if(i&1) F[i]*=-1;
425     }
426     return F;

```

427 }

6.3 polynomial/poly-old.cpp

ll 系数的旧版多项式

什么时候用

保留旧题适配接口，系数类型为 ll，默认 MOD=998244353。新题优先用 poly-fast 或 MTT。

调用顺序与接口

- poly 实际是 `vector<long long>`，输入系数应先归一化到 $[0, \text{MOD})$ 。
- `f*g` 为卷积，`inv/ln/exp/sqrt` 使用小写命名。
- `init(n)` 准备 fac、ifac、Inv 表；注意 Inv 是整数逆元数组，inv 才是幂级数求逆函数。

最小示例（放入 main）：

```
1 init(10); poly f{1,2},g{3,4}; auto h=f*g;
2 assert(h[0]==3 && h[1]==10 && h[2]==8);
3 poly a{1,1,0,0}; auto b=inv(a);
4 assert(b[0]==1 && b[1]==MOD-1);
```

使用边界

不要根据其他版本的接口大小写直接替换。旧版保留许多历史实现，本手册的可运行示例验证乘法和求逆；做递推推项优先选已核对接口的 MTT/poly-fast，避免依赖旧示例 solve。文件含 poly、LL、MP 等宏，组合时应留意污染。

完整代码

```
1 #define LL __int128
2 #define MP make_pair
3 #define poly vector<long long>
4 //
5 // template for polynomial (ll NTT)
6 // version 1.0 (last update: May 24th, 2026)
7 //
8 // usage:
9 //   init(n); poly f,g; f*g; ln(f);
10 //
11 const ll MOD=998244353,inv2=(MOD+1)/2;
12 vector<ll> fac{1},ifac{1},Inv{0,1};
13 vector<int> bfly(1);vector<ll> inver{0,1};
14 int clogg(int x){assert(x>0);return x==1?0:32-__builtin_clz(x-1);}
15 ll ksm(ll a,int b){ll res=1;while(b){if(b&1)res=res*a%MOD;a=a*a%MOD,b>>=1;}return res;}
16 void butterfly(int l){
17     assert(0<=l&&l<=23);
18     static int las=-1;
19     if(las!=l){
20         las=l;bfly.assign(1<<l,0);
21         for(int i=1;i<(1<<l);i++)
22             bfly[i]=(bfly[i>>1]>>1)|((i&1)<<l-1);
23     }
24 }
25 void NTT(poly &f,int l,int typ){
26     butterfly(l);f.resize(1<<l);
27     for(int i=0;i<(1<<l);i++)
```

```

28     if(bfly[i]<i) swap(f[i],f[bfly[i]]);
29     for(int i=0;i<l;i++){
30         ll step=ksm(3,MOD-1+(MOD-1>>i+1)*typ);
31         for(int j=0;j<(1<<l);j+=(1<<i+1)){
32             ll cur=1,l=j+(1<<i);
33             for(int k=j;k<l;k++){
34                 ll u=f[k],v=f[k|(1<<i)]*cur%MOD;
35                 f[k]=(u+v>=MOD?u+v-MOD:u+v);
36                 f[k|(1<<i)]=(u>=v?u-v:u-v+MOD);
37                 cur=cur*step%MOD;
38             }
39         }
40     }
41     if(typ== -1){
42         ll val=ksm(1<<l,MOD-2);
43         for(int i=0;i<(1<<l);i++)
44             f[i]=val*f[i]%MOD;
45     }
46     return;
47 }
48 poly operator *(poly f,poly g){
49     // h[i+j] <- f[i]*g[j]
50     if(f.empty()||g.empty())return {};
51     assert(f.size()+g.size()-1<=(1<<23));
52     int n=f.size()+g.size(),l=clogg(f.size()+g.size()-1);
53     if(n<=64){
54         poly h(n-1);
55         for(int i=0;i<f.size();i++)
56             for(int j=0;j<g.size();j++)
57                 h[i+j]+=f[i]*g[j]%MOD;
58         for(ll &i:h) i%=MOD;
59         return h;
60     }
61     NTT(f,l,1);NTT(g,l,1);
62     for(int i=0;i<(1<<l);i++)
63         f[i]=f[i]*g[i]%MOD;
64     NTT(f,l,-1);f.resize(n-1);
65     return f;
66 }
67 poly operator /(poly f,poly g){
68     if(f.empty()||g.empty())return poly(f.size());
69     // h[i-j] <- f[i]*g[j]
70     int m=g.size();
71     reverse(g.begin(),g.end());
72     g=f*g;
73     for(int i=0;i<f.size();i++) f[i]=g[i+m-1];
74     return f;
75 }
76 void operator +=(poly &f,const poly &g){
77     int n=max(f.size(),g.size());
78     f.resize(n);
79     for(int i=0;i<g.size();i++)
80         f[i]=(f[i]+g[i]>=MOD?f[i]+g[i]-MOD:f[i]+g[i]);
81     return;
82 }

```

```

83 poly operator +(poly f,poly g){
84     f+=g;
85     return f;
86 }
87 void operator -=(poly &f,const poly &g){
88     int n=max(f.size(),g.size());
89     f.resize(n);
90     for(int i=0;i<g.size();i++)
91         f[i]=(f[i]-g[i]<0?f[i]-g[i]+MOD:f[i]-g[i]);
92     return;
93 }
94 poly operator -(poly f,poly g){
95     f-=g;
96     return f;
97 }
98 poly operator *(poly f,ll x){
99     for(int i=0;i<f.size();i++) (f[i]*=x)%=MOD;
100    return f;
101 }
102 poly operator *(ll x,poly f){
103     for(int i=0;i<f.size();i++) (f[i]*=x)%=MOD;
104     return f;
105 }
106 poly inv(poly f){
107     assert(!f.empty()&&f[0]);
108     int n=f.size(),l=clogg(n);
109     f.resize(1<<l);
110     poly g{ksm(f[0],MOD-2)},_f;
111     for(int i=1;i<=l;i++){
112         _f=poly(begin(f),begin(f)+(1<<i));
113         NTT(g,i+1,1);NTT(_f,i+1,1);
114         for(int j=0;j<(1<<i+1);j++)
115             g[j]=(2*g[j]-g[j]*g[j]%MOD*_f[j]%MOD+MOD)%MOD;
116         NTT(g,i+1,-1);
117         fill(begin(g)+(1<<i),end(g),0);
118     }
119     g.resize(n);
120     return g;
121 }
122 poly integ(poly f){
123     int n=f.size();f.resize(n+1);
124     assert(n<MOD);
125     int m=inver.size();
126     if(n>=m){
127         int k=max(n,(int)min(2ll*m,MOD-1));inver.resize(k+1);
128         for(int i=m;i<=k;i++)inver[i]=MOD-(MOD/i)*inver[MOD%i]%MOD;
129     }
130     for(int i=n;i>=1;i--) f[i]=f[i-1]*inver[i]%MOD;
131     f[0]=0;
132     return f;
133 }
134 poly diff(poly f){
135     if(f.empty())return {};
136     int n=f.size();
137     for(int i=0;i<n-1;i++)

```

```

138     f[i]=f[i+1]*(i+1)%MOD;
139     f.pop_back();
140     return f;
141 }
142 poly ln(poly f){
143     assert(!f.empty()&&f[0]==1);
144     poly f_=diff(f),_f=inv(f);
145     int n=f_.size(),m=_f.size();
146     int l=clogg(n+m);
147     NTT(f_,l,1);NTT(_f,l,1);
148     for(int i=0;i<(1<<l);i++)
149         f_[i]=f_[i]*_f[i]%MOD;
150     NTT(f_,l,-1);
151     f_=integ(f_);
152     f_.resize(f.size());
153     return f_;
154 }
155 poly exp(poly f){
156     assert(!f.empty()&&f[0]==0);
157     poly g{1},_f,_g;
158     int n=f.size(),l=clogg(n);
159     f.resize(1<<l);
160     for(int i=1;i<=l;i++){
161         _f=poly(begin(f),begin(f)+(1<<i));
162         _g=ln(g);
163         NTT(g,i+1,1);NTT(_f,i+1,1);NTT(_g,i+1,1);
164         for(int j=0;j<(1<<i+1);j++)
165             g[j]=g[j]*(1-_g[j]+_f[j]+MOD)%MOD;
166         NTT(g,i+1,-1);
167         fill(begin(g)+(1<<i),end(g),0);
168     }
169     g.resize(n);
170     return g;
171 }
172 poly sqrt(poly f){
173     assert(!f.empty()&&f[0]==1);
174     poly g{1},_f,_g;
175     int n=f.size(),l=clogg(n);
176     f.resize(1<<l);
177     for(int i=1;i<=l;i++){
178         _f=poly(begin(f),begin(f)+(1<<i));
179         _g=inv(g);
180         NTT(_f,i+1,1);NTT(_g,i+1,1);NTT(g,i+1,1);
181         for(int j=0;j<(1<<i+1);j++)
182             g[j]=(_f[j]+g[j]*g[j]%MOD)*inv2%MOD*_g[j]%MOD;
183         NTT(g,i+1,-1);
184         fill(begin(g)+(1<<i),end(g),0);
185     }
186     g.resize(n);
187     return g;
188 }
189 void div(poly f,poly g,poly &q,poly &r){//  $\hat{p}, q \in \mathbb{F}_p[x]$   $f=g*q+r$ ;
190     assert(!g.empty()&&g.back());
191     if(f.size()<g.size()){q={0};r=f;return;}
192     int n=f.size()-1,m=g.size()-1;

```

```

193 reverse(f.begin(),f.end());
194 reverse(g.begin(),g.end());
195 g.resize(n+1);
196 q=(f*inv(g));q.resize(n-m+1);
197 reverse(q.begin(),q.end());
198 g.resize(m+1);
199 reverse(g.begin(),g.end());
200 reverse(f.begin(),f.end());
201 poly h=q*g;
202 r.resize(m);
203 for(int i=0;i<m;i++) r[i]=(f[i]-h[i]+MOD)%MOD;
204 return;
205 }
206 poly eval(poly f,poly x){
207 if(x.empty())return {};
208 vector<poly> func; map<pair<int,int>,int> mp;
209 function<poly(int,int)> prod=[&](int l,int r){
210     poly ans;
211     if(l==r) ans=(poly){MOD-x[l],1};
212     else {
213         int mid=l+r>>1;
214         ans=prod(l,mid)*prod(mid+1,r);
215     }
216     func.push_back(ans);
217     mp[MP(l,r)]=func.size()-1;
218     return ans;
219 };prod(0,x.size()-1);
220 int t=-1;
221 function<poly(poly,int,int)> solve=[&](poly f,int l,int r){
222     if(l==r){
223         ll val=0;
224         for(int i=f.size()-1;i>=0;i--)
225             val=(val*x[l]+f[i])%MOD;
226         return (poly){val};
227     }
228     int mid=l+r>>1;
229     poly p0=func[mp[MP(l,mid)]],p1=func[mp[MP(mid+1,r)]];
230     poly d,q;
231     div(f,p0,d,q);
232     poly ans=solve(q,l,mid);
233     div(f,p1,d,q);
234     poly _ans=solve(q,mid+1,r);
235     for(int i:_ans) ans.push_back(i);
236     return ans;
237 };
238 return solve(f,0,x.size()-1);
239 }
240 poly BM(poly a){
241     poly c{1},b{1};int l=0,m=1;ll last=1;
242     for(int i=0;i<(int)a.size();i++){
243         ll d=0;for(int j=0;j<=l;j++)d=(d+c[j]*a[i-j])%MOD;
244         if(!d){m++;continue;}
245         poly t=c;ll v=d*ksm(last,MOD-2)%MOD;
246         if(c.size()<b.size()+m)c.resize(b.size()+m);
247         for(int j=0;j<(int)b.size();j++)c[j+m]=(c[j+m]-v*b[j]%MOD+MOD)%MOD;

```

```

248     if(2*l<=i){l=i+1-l;b=move(t);last=d;m=1;}
249     else m++;
250 }
251 return c;
252 }
253 ll FSPE(poly F,poly G,ll t){// [x^t]F(x)/G(x)
254     assert(t>=0&&!G.empty()&&G[0]);
255     if(F.empty())return 0;
256     while(t){
257         poly G1=G;
258         for(int i=1;i<G1.size();i+=2) G1[i]=MOD-G1[i];
259         F=F*G1,G=G*G1;
260         poly f,g;
261         for(int i=t&1;i<F.size();i+=2) f.push_back(F[i]);
262         for(int i=0;i<G.size();i+=2) g.push_back(G[i]);
263         t>>=1;
264         F=f,G=g;
265         if(F.empty())return 0;
266     }
267     return F[0]*ksm(G[0],MOD-2)%MOD;
268 }
269 }
270 ll RSPE(poly f,ll x){
271     poly g=BM(f),s(g.size()-1);
272     for(int i=0;i<(int)s.size();i++)
273         for(int j=0;j<=i&&j<(int)f.size();j++)s[i]=(s[i]+f[j]*g[i-j])%MOD;
274     return FSPE(s,g,x);
275 }
276 mt19937 rnd(time(0));
277 void init(int n=0){
278     assert(0<=n&&n<MOD);
279     int m=fac.size();if(n<m)return;
280     n=max(n,(int)min(2ll*m,MOD-1));
281     fac.resize(n+1);ifac.resize(n+1);Inv.resize(max(n+1,2));
282     for(int i=m;i<=n;i++){
283         fac[i]=fac[i-1]*i%MOD;
284         if(i>=2)Inv[i]=MOD-Inv[MOD%i]*(MOD/i)%MOD;
285         ifac[i]=ifac[i-1]*Inv[i]%MOD;
286     }
287 }
288 ll C(int x,int y){if(x<0||x>y)return 0;init(y);return fac[y]*ifac[x]%MOD*ifac[y-x]%MOD;}

```

6.4 polynomial/MTT.cpp

非 NTT 模数下的多项式

什么时候用

默认目标模数 10^9+7 ，三模 NTT 后 CRT 还原；常用 FPS 功能与 poly-fast 对齐，但每次卷积都会回到目标模数。

调用顺序与接口

- $\text{poly } f=\{a_0,a_1,\dots\}$ 表示 $a_0+a_1x+\dots$ ，下标就是次数。
- $f*g$ 是卷积；加减和数乘也支持。Inv(f) 返回模 x^n 的倒数， $n=f.size()$ ， $f[0]$ 必须非零。
- Ln(f) 要求 $f[0]=1$ ；Exp(f) 要求 $f[0]=0$ ；返回长度与输入相同。

- `diff/integ` 求导/积分，积分常数取零；`value(f,x)` 是单点求值。
- `BM(seq)` 求线性递推的分母多项式，`FSPE(F,G,k)` 求 F/G 的第 k 项，`RSPE(seq,k)` 从已有递推样本推第 k 项， k 从 0 开始。
- `Poly` 与 `poly` 是同一个类型；`lagrange(xs,ys)` 支持互异取值点，包括零点，返回次数小于点数的插值多项式。

最小示例（放入 `main`）：

```
1 poly f{1,1,0,0};
2 auto g=Inv(f); auto h=f*g; h.resize(4);
3 assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
4 auto e=Exp(Ln(f));
5 for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
6 assert(FSPE({0,1},{1,-1,-1},10).x==55);
7 auto p=lagrange({0,1,2},{1,4,9});
8 assert(p[0].x==1 && p[1].x==2 && p[2].x==1);
```

使用边界

精度由输入 `vector` 长度决定，想求前 n 项就先 `resize(n)`，不要删掉需要保留的高位零。`operator/` 是相关运算 $(f/g)[i]=\sum_j f[i+j]*g[j]$ ，不是多项式除法。形式幂级数相除要 `f*Inv(g)` 后截断。各多项式文件均内置模数类型和全局表，只选一份，不要再粘贴 `ModInt`。卷积结果长度最多 2^{23} ，FPS 输入长度最多 2^{22} ，`Ln/Exp` 长度还要小于目标质数。没有 `poly-new` 的 `Sqrt/Div/Eval`。`RSPE` 需要输入确实足以确定递推，通常至少给 2 倍阶数的样本，不适用于任意序列外推。

考场版：本节已换成普通迭代 NTT；`fft` 输出按自然顺序排列，`invFft` 接受同样顺序。不能与原版变换内核混搭。

完整代码

```
1 //
2 // template for ModInt, use x.pow(k) / x.inv()
3 // version 2.0 (Last Update Jun 13th, 2026)
4 //
5 // usage:
6 //   init(n); mint a,b; C(n,k); MOD=1e9+7
7 //
8 template <unsigned _M> struct ModInt{
9     static constexpr unsigned MOD=_M;
10    static_assert(1<MOD&&MOD<=INT_MAX);
11    unsigned x;
12    constexpr ModInt():x(0){}
13    constexpr ModInt(unsigned y):x(y%MOD){}
14    constexpr ModInt(int y):x((y%=static_cast<int>(MOD))<0?y+MOD:y){}
15    constexpr ModInt(unsigned long long y):x(y%MOD){}
16    constexpr ModInt(long long y):x((y%=static_cast<long long>(MOD))<0?y+MOD:y){}
17    ModInt& operator +=(const ModInt &a){x=((x+=a.x)>=MOD)?x-MOD:x;return *this;}
18    ModInt& operator -=(const ModInt &a){x=((x-=a.x)>=MOD)?x+MOD:x;return *this;}
19    ModInt& operator *=(const ModInt &a){x=static_cast<unsigned long long>(x)*a.x%MOD;return *this;}
20    ModInt& operator /=(const ModInt &a){return (*this)*=a.inv();}
21    bool operator ==(const ModInt &a)const{return x==a.x;}
22    bool operator !=(const ModInt &a)const{return x!=a.x;}
23    explicit operator bool() const { return x != 0; }
24    bool operator!() const { return x == 0; }
25    ModInt inv() const {
26        unsigned a=MOD,b=x;ll y=0,z=1;
27        while(b){
28            const unsigned q=a/b,c=a-q*b;
29            a=b,b=c;
```

```

30     const ll w=y-(ll)q*z;
31     y=z,z=w;
32 }
33 assert(a==1);return ModInt(y);
34 }
35 ModInt pow(long long k) const {
36     unsigned long long e=k;
37     ModInt a=*this,b=1;
38     if(k<0){a=a.inv();e=0-e;}
39     for(;e>=1){if(e&1)b*=a;a*=a;}
40     return b;
41 }
42 ModInt& operator++() {*this+=1;return *this;}
43 ModInt& operator--() {*this-=1;return *this;}
44 ModInt operator +()const{return *this;}
45 ModInt operator -()const{ModInt a;a.x=(x?MOD-x:0u);return a;}
46 ModInt operator +(const ModInt &a)const{return ModInt(*this)+=a;}
47 ModInt operator -(const ModInt &a)const{return ModInt(*this)-=a;}
48 ModInt operator *(const ModInt &a)const{return ModInt(*this)*=a;}
49 ModInt operator /(const ModInt &a)const{return ModInt(*this)/=a;}
50
51 template<typename T> ModInt friend operator +(T a,const ModInt &b){return ModInt(a)+=b;}
52 template<typename T> ModInt friend operator -(T a,const ModInt &b){return ModInt(a)-=b;}
53 template<typename T> ModInt friend operator *(T a,const ModInt &b){return ModInt(a)*=b;}
54 template<typename T> ModInt friend operator /(T a,const ModInt &b){return ModInt(a)/=b;}
55 friend istream& operator >> (istream &i,ModInt &x){ll y;if(i>>y)x=ModInt(y);return i;}
56 friend ostream& operator << (ostream &o,const ModInt &x){o<<x.x;return o;}
57 };
58 // Basic function, fac, ifac, binom
59 // init(n) prepares fac / ifac / inv through n
60 const int MOD=1e9+7;
61 using mint=ModInt<MOD>;
62 using poly=vector<mint>;
63 using Poly=poly;
64 vector<mint> fac{1},ifac{1},inv{0,1};
65 void init(int n=0){
66     assert(0<=n&&n<MOD);
67     int m=fac.size();if(n<m)return;
68     n=max(n,(int)min(2ll*m,(ll)MOD-1));
69     fac.resize(n+1);ifac.resize(n+1);inv.resize(max(n+1,2));
70     for(int i=m;i<=n;i++){
71         fac[i]=fac[i-1]*i;
72         if(i>1)inv[i]=-(MOD/i)*inv[MOD%i];
73         ifac[i]=ifac[i-1]*inv[i];
74     }
75 }
76 inline mint C(int x,int y){
77     // choose x from y
78     if(x<0||y<x)return 0;
79     init(y);return fac[y]*ifac[x]*ifac[y-x];
80 }
81 inline mint binom(int y,int x){return C(x,y);}
82
83 //
84 // template for MTT (three-prime CRT)

```

```

85 // usage:
86 //   poly h=f*g; Inv(f); Ln(f); Exp(f);
87 //   diff, integ, value, BM, FSPE, RSPE, lagrange;
88 //   MOD is prime; Ln needs f[0]=1, Exp needs f[0]=0
89 //   convolution length <= 2^23; FPS length <= 2^22
90 //
91 const int FFT_MAX=23;
92 template<unsigned MO> struct MTT_NTT{
93 using num=ModInt<MO>;
94 void ntt(num *a,int n,bool inverse)const{
95     assert(1<=n&&n<=(1<<FFT_MAX)&&!(n&(n-1)));
96     for(int i=1,j=0;i<n;i++){
97         int k=n>>1;
98         for(;j&k;k>>=1)j^=k;
99         j^=k;if(i<j)swap(a[i],a[j]);
100     }
101     for(int len=2;len<=n;len<=<=1){
102         num step=num(3).pow((MO-1)/len);
103         if(inverse)step=step.inv();
104         for(int i=0;i<n;i+=len){
105             num cur=1;
106             for(int j=0;j<len/2;j++,cur*=step){
107                 num x=a[i+j],y=a[i+j+len/2]*cur;
108                 a[i+j]=x+y;a[i+j+len/2]=x-y;
109             }
110         }
111     }
112     if(inverse){num v=num(n).inv();for(int i=0;i<n;i++)a[i]*=v;}
113 }
114 void fft(num *a,int n)const{ntt(a,n,false);}
115 void invFft(num *a,int n)const{ntt(a,n,true);}
116 };
117
118 template<unsigned MO> vector<ModInt<MO>> convolution(const poly &f,const poly &g,int n){
119     static const MTT_NTT<MO> ntt;
120     vector<ModInt<MO>> a(n),b(n);
121     for(int i=0;i<(int)f.size();i++)a[i]=f[i].x;
122     for(int i=0;i<(int)g.size();i++)b[i]=g[i].x;
123     ntt.fft(a.data(),n);ntt.fft(b.data(),n);
124     for(int i=0;i<n;i++)a[i]*=b[i];
125     ntt.invFft(a.data(),n);a.resize(f.size()+g.size()-1);return a;
126 }
127 poly operator *(const poly &f,const poly &g){
128     if(f.empty()||g.empty())return {};
129     assert(f.size()+g.size()-1<=(1<<FFT_MAX));
130     int m=f.size()+g.size()-1;
131     poly h(m);
132     if(min(f.size(),g.size())<=32){
133         for(int i=0;i<(int)f.size();i++)for(int j=0;j<(int)g.size();j++)h[i+j]+=f[i]*g[j];
134         return h;
135     }
136     constexpr unsigned mod1=998244353,mod2=167772161,mod3=469762049;
137     constexpr unsigned long long mod12=1ull*mod1*mod2;
138     static const unsigned inv1=ModInt<mod2>(mod1).inv().x;
139     static const unsigned inv2=ModInt<mod3>(mod12).inv().x;

```

```

140  assert((__int128)min(f.size(),g.size())*(MOD-1)*(MOD-1)<((__int128)mod12*mod3);
141  int n=1;while(n<m)n<=1;
142  auto a=convolution<mod1>(f,g,n);
143  auto b=convolution<mod2>(f,g,n);
144  auto c=convolution<mod3>(f,g,n);
145  for(int i=0;i<m;i++){
146      unsigned long long x=(b[i].x+mod2-a[i].x%mod2)*1ull*inv1%mod2;
147      x=x*mod1+a[i].x;
148      unsigned long long y=(c[i].x+mod3-x%mod3)*inv2%mod3;
149      h[i]=x%MOD+(mod12%MOD)*y%MOD;
150  }
151  return h;
152 }
153 // Transposed multiplication: (f/g)[i] = sum_j f[i+j]*g[j].
154 poly operator /(const poly &f,poly g){
155     if(f.empty()||g.empty())return poly(f.size());
156     int m=g.size();reverse(g.begin(),g.end());g=f*g;
157     return poly(g.begin()+m-1,g.end());
158 }
159 void operator +=(poly &f,const poly &g){
160     if(f.size()<g.size())f.resize(g.size());
161     for(int i=0;i<(int)g.size();i++)f[i]+=g[i];
162 }
163 poly operator +(poly f,const poly &g){f+=g;return f;}
164 void operator -=(poly &f,const poly &g){
165     if(f.size()<g.size())f.resize(g.size());
166     for(int i=0;i<(int)g.size();i++)f[i]-=g[i];
167 }
168 poly operator -(poly f,const poly &g){f-=g;return f;}
169 poly operator *(poly f,mint x){for(auto &y:f)y*=x;return f;}
170 poly operator *(mint x,poly f){for(auto &y:f)y*=x;return f;}
171 mint value(const poly &f,mint x){
172     mint ans=0;for(int i=(int)f.size()-1;i>=0;i--)ans=ans*x+f[i];return ans;
173 }
174 poly Inv(const poly &f){
175     assert(!f.empty()&&f[0]&&f.size()<=(1<<(FFT_MAX-1)));
176     int n=f.size();poly g{f[0].inv()};
177     while((int)g.size()<n){
178         int m=min(n,(int)g.size()*2);
179         poly h=poly(f.begin(),f.begin()+m)*g;h.resize(m);
180         for(auto &x:h)x=-x;
181         h[0]+=2;g=g*h;g.resize(m);
182     }
183     return g;
184 }
185 poly diff(poly f){
186     if(f.empty())return {};
187     for(int i=1;i<(int)f.size();i++)f[i-1]=f[i]*i;
188     f.pop_back();return f;
189 }
190 poly integ(poly f){
191     int n=f.size();init(n);f.resize(n+1);
192     for(int i=n;i-->0)f[i]=f[i-1]*inv[i];
193     f[0]=0;return f;
194 }

```

```

195 poly Ln(const poly &f){
196     assert(!f.empty()&&f[0].x==1&&f.size()<(size_t)MOD&&f.size()<=(1<<(FFT_MAX-1)));
197     poly g=diff(f)*Inv(f);g.resize(f.size()-1);return integ(move(g));
198 }
199 poly Exp(const poly &f){
200     assert(!f.empty()&&!f[0]&&f.size()<(size_t)MOD&&f.size()<=(1<<(FFT_MAX-1)));
201     int n=f.size();poly g{1};
202     while((int)g.size()<n){
203         int m=min(n,(int)g.size()*2);poly h=g;h.resize(m);h=Ln(h);
204         for(int i=0;i<m;i++)h[i]=f[i]-h[i];
205         h[0]+=1;g=g*h;g.resize(m);
206     }
207     return g;
208 }
209 poly BM(poly a) {
210     poly C{1}, B{1};
211     int L = 0, m = 1;
212     mint b = 1;
213     for (int n = 0; n < (int)a.size(); n++) {
214         mint d = 0;
215         for (int i = 0; i <= L; i++) d += C[i] * a[n - i];
216         if (!d) {
217             m++;
218         } else {
219             poly T = C;
220             mint coef = d * b.inv();
221             poly xmb(m, 0);
222             for (mint val : B) xmb.push_back(val);
223             if (C.size() < xmb.size()) C.resize(xmb.size(), 0);
224             for (int i = 0; i < (int)xmb.size(); i++) C[i] -= coef * xmb[i];
225             if (2 * L <= n) {
226                 L = n + 1 - L; B = T; b = d; m = 1;
227             } else {
228                 m++;
229             }
230         }
231     }
232     return C;
233 }
234
235 mint FSPE(poly F, poly G, ll t) {
236     // find  $[x^t] F/G$ 
237     assert(t>=0&&!G.empty()&&G[0]);
238     if(F.empty())return 0;
239     while (t) {
240         poly G1 = G;
241         for (int i = 1; i < (int)G1.size(); i += 2) G1[i] = -G1[i];
242         F = F * G1; G = G * G1;
243         poly f, g;
244         for (int i = t & 1; i < (int)F.size(); i += 2) f.push_back(F[i]);
245         for (int i = 0; i < (int)G.size(); i += 2) g.push_back(G[i]);
246         t >>= 1;
247         F = f; G = g;
248         if(F.empty())return 0;
249     }

```

```

250     return F[0] / G[0];
251 }
252
253 mint RSPE(poly f, ll x) {
254     // find f[x] in O(n^2), note that f must be recursion
255     poly g = BM(f);
256     poly s(g.size()-1);
257     for (int i = 0; i < (int)s.size(); i++)
258         for (int j = 0; j <= i && j < (int)f.size(); j++)
259             s[i] += f[j] * g[i - j];
260     return FSPE(s, g, x);
261 }
262
263
264 poly lagrange(const vector<mint> &x, const vector<mint> &y){
265     int n=x.size();assert(y.size()==x.size());
266     poly g(n+1),f(n),q(n);g[0]=1;
267     for(int i=0;i<n;i++){
268         for(int j=i+1;j-->0)g[j]=g[j-1]-x[i]*g[j];
269         g[0]*=-x[i];
270     }
271     for(int i=0;i<n;i++){
272         q[n-1]=g[n];
273         for(int j=n-2;j>=0;j-->0)q[j]=g[j+1]+x[i]*q[j+1];
274         mint d=value(q,x[i]);assert(d);
275         mint c=y[i]/d;
276         for(int j=0;j<n;j++)f[j]+=q[j]*c;
277     }
278     return f;
279 }

```

6.5 polynomial/FWT.cpp

按位 AND / OR / XOR 卷积

什么时候用

求 $c_k = \sum_{i \oplus j = k} a_i b_j$, 其中 op 为按位 AND、OR 或 XOR。长度 $n = 2^k$, 每次变换 $O(n \log n)$, 额外空间 $O(1)$ 。

调用顺序与接口

- `convolution_and(a,b)`、`convolution_or(a,b)`、`convolution_xor(a,b)` 直接返回对应卷积, 不修改传入数组。支持长度不同、非二次幂长度; 任一输入为空则返回空数组, 否则结果长度为不小于两者最大长度的最小二次幂。
- `fwt_and(a)`、`fwt_or(a)`、`fwt_xor(a)` 原地正变换; 第二个参数传 `true` 做逆变换。
- 支持 `vector<int>`、`vector<ll>` 和 `vector<mint>`。mint 需先抄模整数定义, FWT 本身不依赖多项式文件。
- 两个数组补零到相同的非零二次幂长度 n, 使 n 大于最大下标; 不必补到两者长度之和。
- 两边做同一种正变换, 逐点相乘, 再做对应逆变换, 即得卷积。

最小示例 (放入 main):

```

1 vector<int> a{1,2},b{3,4};
2 auto c=convolution_and(a,b); assert((c==vector<int>{13,8}));
3 c=convolution_or(a,b); assert((c==vector<int>{3,18}));
4 c=convolution_xor(a,b); assert((c==vector<int>{11,10}));
5 // vector<mint> works with the same interfaces.

```

使用边界

下标从 0 开始。int/ll 表示普通整数运算，不会自动取模，正变换和逐点乘法的中间值也必须不溢出。整数 XOR 逆变换逐层除以 2，要求每步可整除，不能把任意整数数组当成合法频域数据。XOR 模运算要求 2 可逆，通常使用奇数模数；AND/OR 无此限制。不需要归一化第二次。

完整代码

```

1 //
2 // template for FWT and bitwise convolution
3 //
4 // usage:
5 //   auto c=convolution_and(a,b); // convolution_or / convolution_xor
6 //   fwt_and(a); fwt_and(a,true); // forward / inverse; also or / xor
7 //   vector<int>, vector<ll> or vector<mint>; 0-indexed
8 //   convolution pads to a power of 2; empty input returns {}
9 //   transform size must be a positive power of 2
10 //   integer intermediates must fit the type; mint XOR needs inverse of 2
11 //
12 template<typename T>
13 void fwt_and(vector<T> &a,bool inverse=false){
14     assert(!a.empty()&&a.size()<=INT_MAX&&!(a.size()&(a.size()-1)));
15     int n=a.size();
16     for(int k=1;k<n;k<=<=1)
17         for(int i=0;i<n;i+=k<=<1)
18             for(int j=0;j<k;j++){
19                 if(inverse)a[i+j]-=a[i+j+k];
20                 else a[i+j]+=a[i+j+k];
21             }
22 }
23 template<typename T>
24 void fwt_or(vector<T> &a,bool inverse=false){
25     assert(!a.empty()&&a.size()<=INT_MAX&&!(a.size()&(a.size()-1)));
26     int n=a.size();
27     for(int k=1;k<n;k<=<=1)
28         for(int i=0;i<n;i+=k<=<1)
29             for(int j=0;j<k;j++){
30                 if(inverse)a[i+j+k]-=a[i+j];
31                 else a[i+j+k]+=a[i+j];
32             }
33 }
34 template<typename T>
35 void fwt_xor(vector<T> &a,bool inverse=false){
36     assert(!a.empty()&&a.size()<=INT_MAX&&!(a.size()&(a.size()-1)));
37     int n=a.size();T inv2=1;
38     if constexpr(!is_integral_v<T>)if(inverse)inv2=T(1)/T(2);
39     for(int k=1;k<n;k<=<=1)
40         for(int i=0;i<n;i+=k<=<1)
41             for(int j=0;j<k;j++){
42                 if constexpr(is_integral_v<T>){
43                     using W=conditional_t<(sizeof(T)<=4),long long,__int128>;
44                     W x=a[i+j],y=a[i+j+k],u=x+y,v=x-y;
45                     if(inverse){assert(u%2==0&&v%2==0);u/=2;v/=2;}
46                     assert(numeric_limits<T>::lowest()<=u&&u<=numeric_limits<T>::max());
47                     assert(numeric_limits<T>::lowest()<=v&&v<=numeric_limits<T>::max());
48                     a[i+j]=T(u);a[i+j+k]=T(v);
49                 }else{

```

```
50         T x=a[i+j],y=a[i+j+k];
51         a[i+j]=x+y;a[i+j+k]=x-y;
52         if(inverse){a[i+j]*=inv2;a[i+j+k]*=inv2;}
53     }
54 }
55 }
56
57 template<typename T>
58 vector<T> convolution_and(vector<T> a,vector<T> b){
59     if(a.empty()||b.empty())return {};
60     assert(max(a.size(),b.size())<=(1u<<30));
61     int n=1;while(n<(int)max(a.size(),b.size()))n<<=1;
62     a.resize(n);b.resize(n);
63     fwt_and(a);fwt_and(b);
64     for(int i=0;i<n;i++)a[i]*=b[i];
65     fwt_and(a,true);return a;
66 }
67
68 template<typename T>
69 vector<T> convolution_or(vector<T> a,vector<T> b){
70     if(a.empty()||b.empty())return {};
71     assert(max(a.size(),b.size())<=(1u<<30));
72     int n=1;while(n<(int)max(a.size(),b.size()))n<<=1;
73     a.resize(n);b.resize(n);
74     fwt_or(a);fwt_or(b);
75     for(int i=0;i<n;i++)a[i]*=b[i];
76     fwt_or(a,true);return a;
77 }
78
79 template<typename T>
80 vector<T> convolution_xor(vector<T> a,vector<T> b){
81     if(a.empty()||b.empty())return {};
82     assert(max(a.size(),b.size())<=(1u<<30));
83     int n=1;while(n<(int)max(a.size(),b.size()))n<<=1;
84     a.resize(n);b.resize(n);
85     fwt_xor(a);fwt_xor(b);
86     for(int i=0;i<n;i++)a[i]*=b[i];
87     fwt_xor(a,true);return a;
88 }
```

7 字符串

7.1 string/sa.cpp

后缀数组和 $O(1)$ LCP

什么时候用

计数排序倍增构建后缀排序；附带 SA_LCP，用 height 数组和 ST 表查询两个后缀的最长公共前缀。

调用顺序与接口

- SA sa; sa.build(" "+s) 输入前面补一个占位空格，真实字符下标 1..n。
- sa.sa[r] 是排名第 r 的后缀起点；sa.rk[i] 是后缀 i 的排名。
- SA_LCP l; l.build(sa)，然后 l.lcp(i,j) 或 l.ask(i,j) 查询，i,j 是原串位置。
- 也可 l.build(" "+s) 直接建。height[r] 为排名 r 与 r-1 的后缀 LCP，height[1]=0。

最小示例（放入 main）：

```
1 SA sa; sa.build(" banana");
2 assert(sa.sa[1]==6);
3 SA_LCP l; l.build(sa);
4 assert(l.lcp(2,4)==3); // anana / ana
5 assert(l.lcp(3,3)==4);
```

使用边界

SA 构建 $O(n \log n)$ ，工作空间 $O(n)$ ，LCP 结构构建与空间 $O(n \log n)$ ，查询 $O(1)$ 。不要把后缀排名传给 lcp。默认字符编码范围 1..127，任意非零字节传 M=256；不支持串内零字节。修改原串后须重建两者。

完整代码

```
1 struct SA{
2     int n=0,m=0;
3     vector<int> sa,rk;
4     vector<unsigned char> s;
5     void set(){n=m=0;sa.clear();rk.clear();s.clear();}
6     void build(string S,int M=128){
7         assert(!S.empty()&&S[0]!=' ' &&1<=M&&M<=256);
8         assert(S.size()<INT_MAX/2);
9         n=S.size()-1;m=M;
10        sa.assign(n+1,0);rk.assign(n+1,0);s.assign(n+1,0);
11        vector<int> y(n+1),old(n+1),cnt(max(n+1,M),0);
12        for(int i=1;i<=n;i++){
13            s[i-1]=(unsigned char)S[i];
14            assert(0<s[i-1]&&s[i-1]<M);
15            rk[i]=s[i-1];cnt[rk[i]]++;
16        }
17        for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1];
18        for(int i=n;i;i--)sa[cnt[rk[i]]--]=i;
19        for(int k=1;k<=1){
20            int p=0;
21            for(int i=n-k+1;i<=n;i++)y[++p]=i;
22            for(int i=1;i<=n;i++)if(sa[i]>k)y[++p]=sa[i]-k;
23            fill(cnt.begin(),cnt.end(),0);
24            for(int i=1;i<=n;i++)cnt[rk[i]]++;
25            for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1];
26            for(int i=n;i;i--)sa[cnt[rk[y[i]]]--]=y[i];
```

```

27     old=rk;p=0;
28     for(int i=1;i<=n;i++){
29         int x=sa[i],z=sa[i-1];
30         if(i==1||old[x]!=old[z]||
31             (x+k<=n?old[x+k]:0)!=(z+k<=n?old[z+k]:0))p++;
32         rk[x]=p;
33     }
34     m=p+1;if(p==n)break;
35 }
36 for(int i=1;i<=n;i++)rk[sa[i]]=i;
37 }
38 void output(){
39     cerr<<"string: ";for(int i=0;i<n;i++)cerr<<s[i];cerr<<"\n";
40     cerr<<"sa: ";for(int i=1;i<=n;i++)cerr<<sa[i]<<" \n"[i==n];
41     cerr<<"rk: ";for(int i=1;i<=n;i++)cerr<<rk[i]<<" \n"[i==n];
42 }
43 void solve(){for(int i=1;i<=n;i++)cout<<sa[i]<<" \n"[i==n];}
44 };
45 // SA_LCP L; L.build(sa); L.lcp(i,j); // 1-indexed
46 // O(n log n) build, O(1) query
47 struct SA_LCP{
48     int n=0;
49     vector<int> rk,height;
50     vector<vector<int>> st;
51     void build(const SA &S){
52         n=S.n;rk=S.rk;height.assign(n+1,0);st.clear();
53         assert((int)S.sa.size()==n+1&&(int)rk.size()==n+1);
54         for(int i=1,k=0;i<=n;i++){
55             int r=rk[i];
56             if(r==1){k=0;continue;}
57             int j=S.sa[r-1];
58             while(i+k<=n&&j+k<=n&&S.s[i+k-1]==S.s[j+k-1])k++;
59             height[r]=k;if(k)k--;
60         }
61         if(!n)return;
62         st.push_back(height);
63         for(int k=1;(1<<k)<=n;k++){
64             st.emplace_back(n-(1<<k)+2);
65             for(int i=1;i+(1<<k)-1<=n;i++)st[k][i]=min(st[k-1][i],st[k-1][i+(1<<(k-1))]);
66         }
67     }
68     void build(const string &s,int M=128){SA S;S.build(s,M);build(S);}
69     int lcp(int i,int j)const{
70         assert(1<=min(i,j)&&max(i,j)<=n);
71         if(i==j)return n-i+1;
72         int l=rk[i],r=rk[j];if(l>r)swap(l,r);l++;
73         int k=__lg(r-l+1);return min(st[k][l],st[k][r-(1<<k)+1]);
74     }
75     int ask(int i,int j)const{return lcp(i,j);}
76 };

```

7.2 string/sam.cpp

单串后缀自动机

什么时候用

构建单个小写字母串的子串状态图，并统计每个状态对应子串的出现次数。时间及空间 $O(n)$ ，字母表固定 26。

调用顺序与接口

- SAM t; t.build(" "+s) 每次重新构建。
- 根为 0; ch[u][c] 为字符 c 的转移，0 表示不存在。
- 状态 u 对应长度区间 (len[fail[u]],len[u])，同一状态内的子串出现次数均为 siz[u]。

最小示例（放入 main）：

```
1 SAM t; t.build(" ababa");
2 int u=0; for(char c:string("aba"))u=t.ch[u][c-'a'];
3 assert(u && t.siz[u]==2);
4 ll distinct=0;
5 for(int i=1;i<=t.tot;i++)distinct+=t.len[i]-t.len[t.fail[i]];
6 assert(distinct==9);
```

使用边界

匹配模式串时遇到不存在转移应立即判定不出现。build 内已累计 siz，不能再次累计。裸调用 extend 不会自动维护整套出现次数；多串应用请用 GSAM。solve 是“重复子串长度乘次数最大值”的特定示例，不是通用查询。

完整代码

```
1 // string algorithm template by Otomachi Una (Junlin Ye)
2 // version 1.0 (last update May 24th 2026)
3 //
4 // usage:
5 //   SAM sam; sam.build(" abaab"); // leading space, lowercase
6 //   sam.len[u], sam.fail[u], sam.siz[u]
7 //
8 // your string should index from 1, begin with a space
9 struct SAM{
10 // this SAM rooted at 0
11 string s;
12 int n=0,tot=0,lst=0;
13 vector<array<int,26>> ch;
14 vector<int> fail,len,siz;
15 vector<vector<int>> son;
16 SAM(){set();}
17 void set(int m=0){
18   assert(0<=m&&m<(INT_MAX-1)/2);n=tot=lst=0;
19   ch.assign(1,{});fail.assign(1,-1);len.assign(1,0);siz.assign(1,0);son.assign(1,{});
20   ch.reserve(2*m+1);fail.reserve(2*m+1);len.reserve(2*m+1);
21   siz.reserve(2*m+1);son.reserve(2*m+1);
22 }
23 int new_node(){
24   assert(tot<INT_MAX-1);ch.push_back({});fail.push_back(0);
25   len.push_back(0);siz.push_back(0);son.emplace_back();return ++tot;
26 }
27 inline int clone(int x){
28   new_node();ch[tot]=ch[x];
29   fail[tot]=fail[x];
```

```

30     return tot;
31 }
32 void extend(char c){
33     assert('a'<=c&&c<='z');c-='a'; // attention the alphabet
34     int cur=new_node();len[cur]=len[lst]+1;
35     while(~lst&&!ch[lst][c]) ch[lst][c]=tot,lst=fail[lst];
36     if(lst==~-1) fail[cur]=0;
37     else{
38         int p=ch[lst][c];
39         if(len[p]==len[lst]+1) fail[cur]=p;
40         else{
41             int q=clone(p);len[q]=len[lst]+1;
42             while(~lst&&ch[lst][c]==p) ch[lst][c]=q,lst=fail[lst];
43             fail[p]=fail[cur]=q;
44         }
45     }
46     lst=cur;
47 }
48 void output(){
49     // this allow you to see the structure of SAM
50     // output fail, ed, len etc.
51     cerr<<"string:"<<s<<endl;
52     cerr<<"# node = "<<tot<<endl;
53     cerr<<"fail: ";for(int i=0;i<=tot;i++) cerr<<fail[i]<<" \n"[i==tot];
54     cerr<<"len: ";for(int i=0;i<=tot;i++) cerr<<len[i]<<" \n"[i==tot];
55     // other else?
56     cerr<<"siz: ";for(int i=0;i<=tot;i++) cerr<<siz[i]<<" \n"[i==tot];
57 }
58 void dfs(int u){
59     vector<int> q{u};
60     for(int i=0;i<(int)q.size();i++)for(int v:son[q[i]])q.push_back(v);
61     for(int i=(int)q.size()-1;i>0;i--)siz[fail[q[i]]]+=siz[q[i]];
62 }
63 void build(string S){
64     assert(!S.empty()&&S[0]==' ' &&S.size()<(INT_MAX-1)/2);
65     set<(int)>S.size()-1;s=move(S);n=(int)s.size()-1;
66     for(int i=1;i<=n;i++){
67         extend(s[i]);
68         // something record here
69         siz[lst]=1;
70     }
71     for(int i=1;i<=tot;i++) son[fail[i]].push_back(i);
72     dfs(0);
73     // output();
74 }
75 void solve(){
76     // this is the main work
77     ll ans=0;
78     for(int i=1;i<=tot;i++) if(siz[i]>1) ans=max(ans,1ll*siz[i]*len[i]);
79     cout<<ans<<"\n";
80 }
81 };

```

7.3 string/ac.cpp

多模式串 AC 自动机

什么时候用

一次加入多个模式串，再统计它们在文本中的出现次数。允许重叠出现；构建 $O(26 \times \text{节点数})$ ，每次 count 为 $O(\text{文本长度} + \text{节点数})$ 。

调用顺序与接口

- AC ac; id=ac.insert(" "+pattern) 返回该模式串末节点编号。
- 插入完所有模式串后调用 build，之后不能再插入，除非 set 清空重建。
- auto cnt=ac.count(" "+text)，模式串答案是 cnt[id]。
- build 后 ch 是补全后的转移，可直接用于自动机 DP；fail 是失配链接。

最小示例（放入 main）：

```
1 AC ac; int a=ac.insert(" aba"),b=ac.insert(" ba");
2 ac.build(); auto cnt=ac.count(" ababa");
3 assert(cnt[a]==2 && cnt[b]==2);
```

使用边界

模式和文本都只支持小写字母，前导空格不参加匹配。重复模式串返回同一编号，但它们各自的答案仍是该模式在文本中的出现次数。空模式返回根编号 0，次数为文本长度 +1。不同 count 之间不会累计。

完整代码

```
1 //
2 // template for Aho-Corasick
3 // usage:
4 //   AC ac; int id=ac.insert(" aba"); ac.build();
5 //   auto cnt=ac.count(" ababa"); cnt[id];
6 //   leading space, lowercase; insert before build
7 //
8 struct AC{
9   int tot=0;
10  bool built=false;
11  vector<array<int,26>> ch;
12  vector<int> fail,q;
13  AC(){set();}
14  void set(int m=0){
15    assert(0<=m&&m<INT_MAX-1);tot=0;built=false;
16    ch.assign(1,{});fail.assign(1,0);q.clear();
17    ch.reserve(m+1);fail.reserve(m+1);
18  }
19  int new_node(){
20    assert(tot<INT_MAX-1);ch.push_back({});fail.push_back(0);return ++tot;
21  }
22  int insert(const string &s){
23    assert(!built&&!s.empty()&&s[0]!=' ' && s.size()<INT_MAX);
24    int u=0;
25    for(int i=1;i<(int)s.size();i++){
26      assert('a'<=s[i]&&s[i]<='z');int c=s[i]-'a';
27      if(!ch[u][c]){int v=new_node();ch[u][c]=v;}
28      u=ch[u][c];
29    }
30    return u;
```

```

31 }
32 void build(){
33     if(built)return;
34     q.clear();q.reserve(tot+1);q.push_back(0);
35     for(int i=0;i<(int)q.size();i++){
36         int u=q[i];
37         for(int c=0;c<26;c++){
38             int v=ch[u][c];
39             if(v){fail[v]=u?ch[fail[u]][c]:0;q.push_back(v);}
40             else if(u)ch[u][c]=ch[fail[u]][c];
41         }
42     }
43     built=true;
44 }
45 vector<ll> count(const string &s)const{
46     assert(built&&!s.empty()&&s[0]!=' ' &&s.size()<INT_MAX);
47     vector<ll> cnt(tot+1);cnt[0]=1;int u=0;
48     for(int i=1;i<(int)s.size();i++){
49         assert('a'<=s[i]&&s[i]<='z');u=ch[u][s[i]-'a'];cnt[u]++;
50     }
51     for(int i=(int)q.size()-1;i;i--)cnt[fail[q[i]]]+=cnt[q[i]];
52     return cnt;
53 }
54 };

```

7.4 string/gsam.cpp

广义后缀自动机

什么时候用

维护多个小写串的所有子串；出现次数是这些串中次数之和，重复插入的串也重复计数，不允许跨串边界匹配。

调用顺序与接口

- GSAM t; t.insert(" "+s) 加入一条字符串，之后 t.build() 累计出现次数。
- count(" "+p) 返回模式的总出现次数；find(" "+p) 返回状态编号，不存在为 -1。
- distinct() 返回所有插入串合起来的不同非空子串数。
- 可继续 insert，但下一次 count 前必须重新 build。

最小示例（放入 main）：

```

1 GSAM t; t.insert(" aba"); t.insert(" bab");
2 t.build(); assert(t.count(" ab")==2);
3 assert(t.count(" abab")==0 && t.distinct()==6);
4 t.insert(" aba"); t.build(); assert(t.count(" aba")==2);

```

使用边界

count 不是“包含该子串的字符串个数”。根为 0，fail[0]=-1；别把普通 SAM 简单重置 lst 的做法当成这里的插入实现。set(m) 的 m 用于预留总字符量级的空间，build 的计数按当前全部插入内容重新计算。

完整代码

```

1 //
2 // template for Generalized SAM
3 // usage:
4 //   GSAM sam; sam.insert(" aba"); sam.insert(" bab"); sam.build();
5 //   sam.count(" ab"); sam.distinct();
6 //   leading space, lowercase; count includes repeated strings
7 //
8 struct GSAM{
9   int n=0,tot=0;
10  ll strings=0;
11  bool built=false;
12  vector<array<int,26>> ch;
13  vector<int> fail,len;
14  vector<ll> cnt,siz;
15  GSAM(){set();}
16  void set(int m=0){
17    assert(0<=m&&m<(INT_MAX-1)/2);n=tot=0;strings=0;built=false;
18    ch.assign(1,{});fail.assign(1,-1);len.assign(1,0);cnt.assign(1,0);siz.clear();
19    ch.reserve(2*m+1);fail.reserve(2*m+1);len.reserve(2*m+1);cnt.reserve(2*m+1);
20  }
21  int new_node(){
22    assert(tot<INT_MAX-1);ch.push_back({});fail.push_back(0);len.push_back(0);cnt.push_back(0);return ++
        tot;
23  }
24  int clone(int x,int l){
25    int y=new_node();ch[y]=ch[x];fail[y]=fail[x];len[y]=1;return y;
26  }
27  int extend(int p,int c){
28    if(ch[p][c]){
29      int q=ch[p][c];if(len[q]==len[p]+1)return q;
30      int r=clone(q,len[p]+1);
31      while(p!=-1&&ch[p][c]==q)ch[p][c]=r,p=fail[p];
32      fail[q]=r;return r;
33    }
34    int cur=new_node();len[cur]=len[p]+1;
35    while(p!=-1&&!ch[p][c])ch[p][c]=cur,p=fail[p];
36    if(p==-1)fail[cur]=0;
37    else{
38      int q=ch[p][c];
39      if(len[q]==len[p]+1)fail[cur]=q;
40      else{
41        int r=clone(q,len[p]+1);
42        while(p!=-1&&ch[p][c]==q)ch[p][c]=r,p=fail[p];
43        fail[q]=fail[cur]=r;
44      }
45    }
46    return cur;
47  }
48  int insert(const string &s){
49    assert(!s.empty()&&s[0]!=' '&&s.size()<INT_MAX);
50    int u=0;n=max(n,(int)s.size()-1);strings++;built=false;
51    for(int i=1;i<(int)s.size();i++){
52      assert('a'<=s[i]&&s[i]<='z');u=extend(u,s[i]-'a');cnt[u]++;
53    }

```

```

54     return u;
55 }
56 void build(){
57     vector<int> c(n+1),q(tot+1);siz=cnt;
58     for(int i=0;i<=tot;i++)c[len[i]]++;
59     for(int i=1;i<=n;i++)c[i]+=c[i-1];
60     for(int i=0;i<=tot;i++)q[--c[len[i]]]=i;
61     for(int i=tot;i;i--)siz[fail[q[i]]]+=siz[q[i]];
62     siz[0]+=strings;built=true;
63 }
64 int find(const string &s)const{
65     assert(!s.empty()&&s[0]!=' '&&s.size()<INT_MAX);int u=0;
66     for(int i=1;i<(int)s.size();i++){
67         assert('a'<=s[i]&&s[i]<='z');u=ch[u][s[i]-'a'];if(!u)return -1;
68     }
69     return u;
70 }
71 ll count(const string &s)const{
72     assert(built);int u=find(s);return u==-1?0:siz[u];
73 }
74 ll distinct()const{
75     ll ans=0;for(int i=1;i<=tot;i++)ans+=len[i]-len[fail[i]];return ans;
76 }
77 };

```

7.5 string/kmp.cpp

前缀函数和 Z 函数

什么时候用

线性时间计算字符串的 border 与各后缀和全串的公共前缀长度。

调用顺序与接口

- `border(" "+s)` 返回 `pi`, `pi[i]` 为 `s[1..i]` 的最长真 border 长度。
- `z_function(" "+s)` 返回 `z`, `z[i]` 为 `s[i..n]` 与 `s[1..n]` 的 LCP。
- 数组长度 `n+1`, 0 号是占位。

最小示例（放入 main）:

```

1 auto pi=border(" ababa"); auto z=z_function(" ababa");
2 assert(pi[5]==3 && z[3]==3);

```

使用边界

这个实现 `z[1]=0`，而不是全串长度。没有封装“模式串在文本的所有位置”：可自行 KMP 扫描，或拼接 `pattern+ 分隔符 +text` 后用 Z；分隔符不能出现在两个原串里。

完整代码

```

1 // string algorithm template by Otomachi Una (Junlin Ye)
2 // version 1.0 (last update May 24th 2026)
3 //
4 // usage:
5 //     string s = " ababa"; auto b = border(s); auto z = z_function(s);
6 //     1-indexed with leading space

```

```

7 //
8 // your string should index from 1, begin with a space
9 vector<int> border(string s){
10 // s is a string with length n+1,
11 // this function return a vector with length n+1
12 // indication the border array of s
13 assert(!s.empty()&&s.size()<=INT_MAX);
14 int n=(int)s.length()-1;
15 assert(!s.empty()&&s[0]==' ');
16 vector<int> a(n+1);
17 for(int i=2,j=0;i<=n;i++){
18     while(j&&s[j+1]!=s[i]) j=a[j];
19     if(s[j+1]==s[i]) j++;
20     a[i]=j;
21 }
22 return a;
23 }
24 vector<int> z_function(string s){
25 // s is a string with length n+1,
26 // this function return a vector with length n+1
27 // indication the a[i]=lcp(s[i:],s[])
28 assert(!s.empty()&&s.size()<=INT_MAX);
29 int n=(int)s.length()-1;
30 assert(!s.empty()&&s[0]==' ');
31 vector<int> z(n+1);
32 for(int i=2,l=1,r=0;i<=n;i++){
33     z[i]=r<i?0:min(r-i+1,z[i-l+1]);
34     while(i+z[i]<=n&&s[1+z[i]]==s[i+z[i]]) z[i]++;
35     if(i+z[i]>r) l=i,r=i+z[i]-1;
36 }
37 return z;
38 }

```

7.6 string/hash.cpp

双模前缀哈希

什么时候用

提供子串指纹与拼接，不是无碰撞的字符串判等。构建前缀数组 $O(n)$ ，常规子串取值 $O(1)$ 。

调用顺序与接口

- `auto h=Hash_of(" "+s)` 产生前缀哈希。
- 子串 $[l,r]$ 为 `h[r]-h[l-1]`。
- `a+b` 拼接两段，`h.v()` 打包哈希值；`.l` 是长度。
- `init_hash(n)` 可预留幂次；实际操作也会自动扩容。

最小示例（放入 main）：

```

1 auto h=Hash_of(" ababa");
2 auto a=h[3]-h[0],b=h[5]-h[2];
3 assert(a.l==b.l && a.v()==b.v());

```

使用边界

比较子串时同时比较长度和哈希。底数每个进程随机，不能把数值用于跨进程永久存储。减法只表示删去前缀，不是任意删除；Hash 对象应使用 {} 零初始化。

完整代码

```

1 //
2 // template for Hash, prepared by Otomachi_Una
3 // version 1.2 (Last Update Jun 13th, 2026)
4 //
5 // usage:
6 //   init_hash(); auto H = Hash_of(" abc"); Hash sub = H[r]-H[l-1];
7 //   sub.v() for compare; random bases each run
8 //
9 mt19937 rnd(time(0));
10 const int MOD1 = 998244353;
11 const int MOD2 = 1e9 + 7;
12 const int B1 = 256 + rnd() % (MOD1-257);
13 const int B2 = 256 + rnd() % (MOD2-257);
14
15 vector<ll> pw1{1}, pw2{1};
16 void init_hash(int n=0);
17
18 struct Hash {
19     int l=0;
20     ll h1=0,h2=0;
21     ll v() const { return h1 << 30 | h2; }
22 };
23
24 inline Hash operator+(const Hash &x, const Hash &y) {
25     // append y to end of x
26     assert(x.l>=0&&y.l>=0&&x.l<=INT_MAX-y.l);init_hash(y.l);
27     return {x.l + y.l,
28             (x.h1 * pw1[y.l] + y.h1) % MOD1,
29             (x.h2 * pw2[y.l] + y.h2) % MOD2};
30 }
31
32 inline Hash operator-(const Hash &x, const Hash &y) {
33     // y is prefix of x, delete y from x
34     assert(0<=y.l&&y.l<=x.l);init_hash(x.l-y.l);
35     return {x.l - y.l,
36             (x.h1 - y.h1 * pw1[x.l - y.l] % MOD1 + MOD1) % MOD1,
37             (x.h2 - y.h2 * pw2[x.l - y.l] % MOD2 + MOD2) % MOD2};
38 }
39
40 template<typename T>
41 inline Hash operator+(const Hash &h, const T &x) {
42     // append x to end of h
43     assert(0<=h.l&&h.l<INT_MAX);
44     return {h.l + 1,
45             (h.h1 * B1 + x) % MOD1,
46             (h.h2 * B2 + x) % MOD2};
47 }
48
49 template<typename T>
50 inline void operator+=(Hash &h, const T &x) {
51     // append x to end of h

```

```

52  h = h + x;
53  }
54
55  void init_hash(int n) {
56      assert(0<=n&&n<INT_MAX);
57      int m=pw1.size();if(n<m)return;
58      n=max(n,(int)min(2ll*m,(ll)INT_MAX-1));
59      pw1.resize(n+1);pw2.resize(n+1);
60      for(int i=m;i<=n;i++){
61          pw1[i]=pw1[i-1]*B1%MOD1;pw2[i]=pw2[i-1]*B2%MOD2;
62      }
63  }
64
65  vector<Hash> Hash_of(const string &s) {
66      int n = (int)s.length() - 1;
67      assert(!s.empty()&&s[0]!=' ' &&s.size()<=INT_MAX);init_hash(n);
68      vector<Hash> a(n+1);
69      for (int i = 1; i <= n; ++i)
70          a[i] = a[i-1] + (unsigned char)s[i];
71      return a;
72  }

```

7.7 string/pam.cpp

回文自动机

什么时候用

在线维护不同回文子串、最长回文后缀及回文出现次数。字符集为小写字母，建树与空间均为 $O(n)$ 。

调用顺序与接口

- PAM p; p.build(s) 从整串重建; s 带前导空格。也可 p.set(n) 预留容量, 再逐字 p.extend(c), 返回当前最长回文后缀结点。
- 0 是长度 0 的根, 1 是长度 -1 的根; 真正的回文结点为 2..tot, 不同回文数为 tot-1。lst 是当前最长回文后缀。
- len[u] 为长度, fail[u] 为最长真回文后缀; num[u] 为该回文的非空回文后缀个数。因此每次加入后的 num[lst] 是当前前缀末尾的回文数。
- pos[u] 为首次出现的右端点, 首次区间是 [pos[u]-len[u]+1,pos[u]]。cnt=p.count() 返回每个回文的总出现次数, 可重叠; 根的次数置 0。

最小示例 (放入 main):

```

1 PAM p; p.build(" ababa");
2 auto cnt=p.count();
3 assert(p.tot-1==5 && p.len[p.lst]==5);
4 assert(p.num[p.lst]==3 && cnt[p.lst]==1);
5 int u=p.ch[1]['a'-'a']; assert(cnt[u]==3);
6 p.extend('b'); cnt=p.count(); // still safe after count()
7 assert(p.len[p.lst]==5 && cnt[u]==3);

```

使用边界

siz 只记录结点作为最长后缀的次数, 不能直接作为总出现次数。count 在副本上沿 fail 累加, 单次 $O(n)$, 可以重复调用, 也能继续 extend; 不要每加入一个字符都调用 count。所有回文出现次数之和用 ll 保存。空串写作一个空格。

完整代码

```

1  //////////////////////////////////////
2  //
3  // template for Palindromic Automaton
4  // usage:
5  //   PAM pam; pam.build(" ababa"); auto cnt=pam.count();
6  //   Or pam.set(n), then pam.extend(c); returns the longest suffix node.
7  //   Leading space, lowercase; real nodes are 2..tot, count() is repeatable.
8  //   len / fail / num / pos: length, suffix link, suffix count, first end.
9  //   O(n) build / space for a fixed alphabet, O(n) count.
10 //
11 //////////////////////////////////////
12 struct PAM{
13 int n=0,tot=1,lst=0;
14 string s;
15 vector<array<int,26>> ch;
16 vector<int> len,fail,num,pos,siz;
17 PAM(){set();}
18 void set(int m=0){
19     assert(0<=m&&m<INT_MAX-2);n=0;tot=1;lst=0;s=" ";
20     ch.assign(2,{});len={0,-1};fail={1,1};
21     num.assign(2,0);pos.assign(2,0);siz.assign(2,0);
22     s.reserve(m+1);ch.reserve(m+2);len.reserve(m+2);fail.reserve(m+2);
23     num.reserve(m+2);pos.reserve(m+2);siz.reserve(m+2);
24 }
25 int get_fail(int u)const{
26     while(s[n-len[u]-1]!=s[n])u=fail[u];
27     return u;
28 }
29 int extend(char c){
30     assert('a'<=c&&c<='z'&&n<INT_MAX-3);s+=c;n++;c-='a';
31     int u=get_fail(lst);
32     if(!ch[u][c]){
33         int v=++tot;
34         ch.push_back({});len.push_back(len[u]+2);
35         fail.push_back(len[v]==1?0:ch[get_fail(fail[u])][c]);
36         num.push_back(num[fail[v]]+1);pos.push_back(n);siz.push_back(0);
37         ch[u][c]=v;
38     }
39     lst=ch[u][c];siz[lst]++;return lst;
40 }
41 void build(string S){
42     assert(!S.empty()&&S[0]==' ' &&S.size()<INT_MAX-2);
43     set((int)S.size()-1);
44     for(int i=1;i<(int)S.size();i++)extend(S[i]);
45 }
46 vector<int> count()const{
47     vector<int> cnt=siz;
48     for(int u=tot;u>=2;u--)cnt[fail[u]]+=cnt[u];
49     cnt[0]=cnt[1]=0;return cnt;
50 }
51 };
52 // end for string/pam.cpp
53 //////////////////////////////////////

```

7.8 string/manacher.cpp

Manacher 回文半径

什么时候用

线性求出所有奇数、偶数长度回文的最大半径，并以 $O(1)$ 判断某个子串是否回文。

调用顺序与接口

- `Manacher t; t.build(s);` `s` 带前导空格，实际下标 $1..n$ ；`set()` 清空。
- `d1[i]` 包含中心字符：最长奇回文为 $[i-d1[i]+1, i+d1[i]-1]$ ，长度 $2*d1[i]-1$ 。
- `d2[i]` 的中心在 $i-1$ 与 i 之间：最长偶回文为 $[i-d2[i], i+d2[i]-1]$ ，长度 $2*d2[i]$ ，半径可以为 0。
- `ask(l,r)` 判断非空闭区间是否回文，要求 $1 \leq l \leq r \leq n$ 。所有回文出现次数为各位置 `d1+d2` 的总和，不是不同回文数。

最小示例（放入 main）：

```
1 Manacher t; t.build(" abba");
2 assert(t.d2[3]==2 && t.d1[2]==1);
3 assert(t.ask(1,4) && t.ask(2,3));
4 assert(!t.ask(1,3));
5 long long cnt=0;
6 for(int i=1;i<=t.n;i++)cnt+=t.d1[i]+t.d2[i];
7 assert(cnt==6);
```

使用边界

时间、空间均为 $O(n)$ ；不插入分隔字符，可处理任意字节。奇偶半径含义不同，不能直接把 `d1` 当成回文长度。空串允许 build，但没有可查询的非空区间。求最长回文取所有 $2*d1[i]-1$ 与 $2*d2[i]$ 的最大值，空串答案为 0。

完整代码

```
1 //////////////////////////////////////
2 //
3 // template for Manacher
4 // usage:
5 //   Manacher t; t.build(" abba"); t.ask(1,4);
6 //   Leading space, 1-indexed; ask(l,r) checks a nonempty closed interval.
7 //   Odd: [i-d1[i]+1,i+d1[i]-1]; even: [i-d2[i],i+d2[i]-1].
8 //   O(n) build / space, O(1) query.
9 //
10 //////////////////////////////////////
11 struct Manacher{
12     int n=0;
13     vector<int> d1,d2;
14     void set(){n=0;d1.clear();d2.clear();}
15     void build(const string &s){
16         assert(!s.empty()&&s[0]!=' ' &&s.size()<INT_MAX/2);
17         n=(int)s.size()-1;d1.assign(n+1,0);d2.assign(n+1,0);
18         for(int i=1,l=1,r=0;i<=n;i++){
19             int k=i>r?1:min(d1[l+r-i],r-i+1);
20             while(i-k>=1&&i+k<=n&&s[i-k]==s[i+k])k++;
21             d1[i]=k--;
22             if(i+k>r)l=i-k,r=i+k;
23         }
24         for(int i=1,l=1,r=0;i<=n;i++){
25             int k=i>r?0:min(d2[l+r-i+1],r-i+1);
26             while(i-k-1>=1&&i+k<=n&&s[i-k-1]==s[i+k])k++;
```

```

27     d2[i]=k;
28     if(i+k-1>r)l=i-k,r=i+k-1;
29 }
30 }
31 bool ask(int l,int r)const{
32     assert(1<=l&&l<=r&&r<=n);int k=r-l+1;
33     return k&1?d1[(l+r)/2]>=k/2+1:d2[(l+r+1)/2]>=k/2;
34 }
35 };
36 // end for string/manacher.cpp
37 ///////////////////////////////////

```

7.9 string/runs.cpp

Runs 极大周期子串

什么时候用

枚举所有长度至少为最小周期两倍、且不能保持该周期向左或向右扩展的子串。返回若干三元组 (l, r, p) ，其中 p 必须是最小周期。

调用顺序与接口

先抄 string/sa.cpp 和 basic/ST table (linear).cpp，再抄本文件。调用 `Runs t; auto a=t.build(s)`，返回 `vector<array<int,3>>`，依次为 l, r, p 。输入有前导空格，区间为 1 下标闭区间，结果按三元组字典序排序并去重。空串、单字符及无重复的串返回空向量。

最小示例（放入 main）：

```

1 Runs t; auto a=t.build(" ababab");
2 assert((a==vector<array<int,3>>{{1,6,2}}));
3 a=t.build(" aabaa");
4 assert((a==vector<array<int,3>>{{1,2,1},{4,5,1}}));
5 assert(t.build(" abc").empty());

```

使用边界

实际字符限字节 1..255，0 留给 SA 哨兵。采用两种字母序下的最长 Lyndon 前缀，加正反串 LCP 向两侧扩展；LCP 使用四毛子，不使用哈希。源码版时间 $O(n \log n)$ （最后排序），空间 $O(n)$ ；考场倍增 SA 也满足此界。Runs 不包括所有周期区间，只给出极大区间；长度不必是 p 的倍数，例如 ababa 返回 (1,5,2)。

完整代码

```

1 ///////////////////////////////////
2 //
3 // template for Runs
4 // usage:
5 //   Paste string/sa.cpp and basic/ST table (linear).cpp first.
6 //   Runs t; auto a=t.build(" ababab"); // {1,6,2}
7 //   Leading space, bytes 1..255; returns vector<array<int,3>> of {l,r,p}.
8 //   [l,r] is maximal with least period p, r-l+1>=2*p; 1-indexed.
9 //   Sorted and unique; O(n log n) time, O(n) space.
10 //
11 ///////////////////////////////////
12 struct Runs{
13 private:
14     struct LCP{
15         int n=0;
16         vector<int> rk;

```

```

17  Linear_RMQ<int> st;
18  void build(const string &s){
19      SA t;t.build(s,256);n=t.n;rk=move(t.rk);
20      vector<int> h(n+1);
21      for(int i=1,k=0;i<=n;i++){
22          int r=rk[i];if(r==1){k=0;continue;}
23          int j=t.sa[r-1];
24          while(i+k<=n&&j+k<=n&&s[i+k]==s[j+k])k++;
25          h[r]=k;if(k)k--;
26      }
27      st.build(h);
28  }
29  int ask(int i,int j)const{
30      if(i>n||j>n)return 0;
31      if(i==j)return n-i+1;
32      int l=rk[i],r=rk[j];if(l>r)swap(l,r);
33      return st.ask(l+1,r);
34  }
35 };
36 public:
37 vector<array<int,3>> build(const string &s)const{
38     assert(!s.empty()&&s[0]==' '&&s.size()<(INT_MAX-1024)/2);
39     int n=(int)s.size()-1;
40     for(int i=1;i<=n;i++)assert((unsigned char)s[i]>0);
41     vector<array<int,3>> ans;if(n<2)return ans;
42     LCP a,b;a.build(s);
43     string t=s;reverse(t.begin()+1,t.end());b.build(t);
44     vector<int> st;st.reserve(n);
45     for(int op=0;op<2;op++){
46         st.clear();
47         for(int i=n;i>=1;i--){
48             while(!st.empty()){
49                 int j=st.back(),k=a.ask(i,j);
50                 // Reversing the alphabet does not reverse the prefix rule.
51                 if(j+k>n)break;
52                 bool less=(unsigned char)s[i+k]<(unsigned char)s[j+k];
53                 if(less==bool(op))break;
54                 st.pop_back();
55             }
56             if(!st.empty()){
57                 int j=st.back(),p=j-i;
58                 int l=b.ask(n-i+2,n-j+2),r=a.ask(i,j);
59                 if(l+r>=p)ans.push_back({i-l,j+r-1,p});
60             }
61             st.push_back(i);
62         }
63     }
64     sort(ans.begin(),ans.end());ans.erase(unique(ans.begin(),ans.end()),ans.end());
65     return ans;
66 }
67 };
68 // end for string/runs.cpp
69 //////////////////////////////////////////////////

```

7.10 string/lyndon.cpp

Lyndon 分解

什么时候用

用 Duval 将串唯一分解为字典序不增的 Lyndon 串。Lyndon 串严格小于自己的每个非空真后缀，相同因子可以连续出现。

调用顺序与接口

`auto a=lyndon(s);` `s` 带前导空格。返回 `vector<pair<int,int>>`，各项为一个因子的 1 下标闭区间 `[l,r]`，按原串从左到右排列且恰好覆盖全串。需要内容时用 `s.substr(l,r-l+1)`。

最小示例（放入 main）：

```
1 string s=" ababab"; auto a=lyndon(s);
2 assert((a==vector<pair<int,int>>{{1,2},{3,4},{5,6}}));
3 assert((lyndon(" cba")==vector<pair<int,int>>{{1,1},{2,2},{3,3}}));
4 assert(lyndon(" ").empty());
```

使用边界

时间 $O(n)$ ，除返回区间外额外空间 $O(1)$ 。比较按 `unsigned char` 的字节顺序，可处理任意字节；不要求小写字母。同样的因子不会合并，`ababab` 分为三个 `ab`，而 `aaaa` 分为四个 `a`。空串返回空向量。本接口是整串分解，不是每个位置的最长 Lyndon 前缀数组。

完整代码

```
1 //////////////////////////////////////
2 //
3 // template for Lyndon Factorization
4 // usage:
5 //   auto a=lyndon(" ababab"); // [1,2], [3,4], [5,6]
6 //   Leading space, 1-indexed closed intervals, ordered from left to right.
7 //   The factors are Lyndon words in non-increasing lexicographic order.
8 //    $O(n)$  time,  $O(1)$  extra space excluding the returned intervals.
9 //
10 //////////////////////////////////////
11 vector<pair<int,int>> lyndon(const string &s){
12     assert(!s.empty()&&s[0]!=' '&&s.size()<INT_MAX);
13     int n=(int)s.size()-1;vector<pair<int,int>> ans;
14     for(int i=1;i<=n;){
15         int j=i+1,k=i;
16         while(j<=n&&(unsigned char)s[k]<=(unsigned char)s[j]){
17             k=s[k]==s[j]?k+1:i;j++;
18         }
19         int p=j-k;
20         while(i<=k){ans.emplace_back(i,i+p-1);i+=p;}
21     }
22     return ans;
23 }
24 // end for string/lyndon.cpp
25 //////////////////////////////////////
```

8 线性代数

8.1 math/gauss.cpp

线性方程组与行列式

什么时候用

用消元求 $Ax=b$ 的一组解，或计算方阵行列式。支持质数模数下的 `mint` 以及浮点类型；不能用 `int/ll` 实例化。

调用顺序与接口

- `Gauss<T> g; res=g.solve(a,m)`, `a` 是 n 行 $m+1$ 列的增广矩阵，最后一列为 `b`。
- 返回 `-1` 无解 (`x` 被清空), `0` 多解, `1` 唯一解; 解存 `g.x`, 长度 `m`, 下标 `0..m-1`。
- 多解时自由变量取 `0`; `g.rank` 是系数矩阵秩, `where[j]=-1` 表示自由变量。
- `g.det(a)` 接受 $n*n$ 方阵, 返回 `T`, 不改变 `x/rank`。空方阵行列式为 `1`。

最小示例 (放入 `main`):

```
1 Gauss<long double> g;
2 // x+y=3, 2x-y=0
3 int res=g.solve({{1,1,3},{2,-1,0}},2);
4 assert(res==1 && abs(g.x[0]-1)<1e-9L);
5 assert(abs(g.x[1]-2)<1e-9L);
6 assert(abs(g.det({{1,2},{3,4}})+2)<1e-9L);
```

使用边界

`Gauss<mint>` 需要先粘贴 `ModInt` 或一个内置 `mint` 的多项式文件。浮点使用最大绝对值选主元和绝对阈值 `eps` (默认 `1e-12`)，尺度差异大或病态方程需要自行缩放并检查残差。方阵时间 $O(n^3)$ ，复制矩阵后原输入保持不变；多个 RHS 要自行扩展。

完整代码

```
1 //
2 // template for Gaussian elimination and determinant
3 // usage:
4 //   Gauss<mint> G; // prime modulus, or Gauss<long double>
5 //   int res=G.solve(a,m); // augmented matrix, m variables
6 //   -1: no solution, 0: multiple, 1: unique; answer in G.x
7 //   free variables are set to zero; G.rank, G.where
8 //   G.det(a); // square matrix; floating point uses G.eps
9 //
10 template<typename T> struct Gauss{
11     static_assert(!is_integral_v<T>,"use a field type");
12     long double eps=1e-12L;
13     int rank=0;
14     vector<int> where;
15     vector<T> x;
16     bool zero(const T &v)const{
17         if constexpr(is_floating_point_v<T>)return abs(v)<=eps;
18         else return !v;
19     }
20     int pivot(const vector<vector<T>> &a,int r,int c)const{
21         int p=-1;
22         for(int i=r;i<(int)a.size();i++)if(!zero(a[i][c])){
23             if constexpr(is_floating_point_v<T>){if(p==-1||abs(a[i][c])>abs(a[p][c]))p=i;}
24             else return i;
```

```

25 }
26 return p;
27 }
28 int solve(vector<vector<T>> a,int m){
29     assert(m>0&&m<INT_MAX&&a.size()<INT_MAX&&eps>=0);
30     int n=a.size();for(auto &v:a)assert(v.size()==(size_t)m+1);
31     rank=0;where.assign(m,-1);x.assign(m,T(0));
32     for(int c=0;c<m&&rank<n;c++){
33         int p=pivot(a,rank,c);if(p==-1)continue;
34         swap(a[p],a[rank]);where[c]=rank;
35         T z=T(1)/a[rank][c];a[rank][c]=T(1);
36         for(int j=c+1;j<=m;j++)a[rank][j]*=z;
37         for(int i=rank+1;i<n;i++){
38             T v=a[i][c];a[i][c]=T(0);if(zero(v))continue;
39             for(int j=c+1;j<=m;j++)a[i][j]-=v*a[rank][j];
40         }
41         rank++;
42     }
43     for(int i=rank;i<n;i++)if(!zero(a[i][m])){x.clear();return -1;}
44     for(int c=m-1;c>=0;c--)if(where[c]!=-1){
45         int r=where[c];x[c]=a[r][m];
46         for(int j=c+1;j<m;j++)x[c]-=a[r][j]*x[j];
47     }
48     return rank==m?1:0;
49 }
50 T det(vector<vector<T>> a)const{
51     assert(a.size()<INT_MAX&&eps>=0);int n=a.size();
52     for(auto &v:a)assert(v.size()==(size_t)n);
53     T ans=1;
54     for(int c=0;c<n;c++){
55         int p=pivot(a,c,c);if(p==-1)return T(0);
56         if(p!=c){swap(a[p],a[c]);ans=-ans;}
57         ans*=a[c][c];T z=T(1)/a[c][c];
58         for(int i=c+1;i<n;i++){
59             T v=a[i][c];a[i][c]=T(0);if(zero(v))continue;v*=z;
60             for(int j=c+1;j<n;j++)a[i][j]-=v*a[c][j];
61         }
62     }
63     return ans;
64 }
65 };

```

8.2 math/linear basis/xor.cpp

异或线性基 调用顺序与接口

维护 64 位无符号整数的异或张成空间。Xor_Basis B 默认空基；set() 清空，insert(x) 返回是否增秩，rank 为当前秩。contains(x) 判断能否异或得到 x；ask(x=0) 返回 x 与基中任意子集异或的最大值，允许空集。

最小示例（放入 main）：

```

1 Xor_Basis B;
2 assert(B.insert(1) && B.insert(2));
3 assert(!B.insert(3) && B.rank==2);
4 assert(B.contains(3) && !B.contains(4));

```

```
5 assert(B.ask()==3 && B.ask(4)==7);
```

使用边界

插入、查询均 $O(64)$ ，空间 $O(64)$ 。输入用 ull，不按有符号大小比较；第 63 位也可用。零向量不增秩。a 中存的是消元后的向量，不是原输入元素。

完整代码

```
1 // Xor_Basis B; B.insert(x); B.contains(x); B.ask(x);
2 struct Xor_Basis{
3     int rank=0;
4     vector<ull> a=vector<ull>(64);
5     void set(){rank=0;fill(a.begin(),a.end(),0);}
6     bool insert(ull x){
7         for(int i=63;i>=0;i--)if(x>>i&1){
8             if(a[i]x^=a[i];
9             else{a[i]=x;rank++;return true;}
10        }
11        return false;
12    }
13    bool contains(ull x)const{
14        for(int i=63;i>=0;i--)if(x>>i&1)x^=a[i];
15        return x==0;
16    }
17    ull ask(ull x=0)const{
18        for(int i=63;i>=0;i--)x=max(x,x^a[i]);
19        return x;
20    }
21 };
```

8.3 math/linear basis/xor-prefix.cpp

前缀异或线性基 调用顺序与接口

Prefix_Xor_Basis B; 按加入顺序调用 insert(x,id)，id 必须为严格递增的正整数，可不连续。增秩返回 0，否则返回本次相关关系中最早且可删除的元素编号；零向量返回自身编号。set() 清空并重置编号。

最小示例（放入 main）：

```
1 Prefix_Xor_Basis B;
2 assert(B.insert(1,1)==0 && B.insert(2,2)==0);
3 assert(B.insert(2,3)==2); // removing id=1 would lower rank
4 assert(B.insert(3,4)==1);
5 assert(B.get_rank(3)==2 && B.contains(1,3));
6 assert(!B.contains(1,4) && B.ask(0,4)==3);
7 assert(B.insert(0,5)==5);
```

使用边界

处理到右端点 r 后，get_rank(l)、contains(x,l)、ask(x,l) 分别查询编号 [l,r] 的秩、可表示性和最大异或值。l 默认 1；l 大于 r 时视为空基。rank 是全部已插入元素的秩。pos 中的正编号是保留的原元素编号，a 是消元后的基。单次 $O(64)$ ，空间 $O(64)$ ；查询历史右端点须离线按 r 扫描或自己保存副本，没有持久化。

完整代码

```

1 // Prefix_Xor_Basis B; int removed=B.insert(x,id);
2 // id must increase; 0: rank increased, otherwise removed id
3 struct Prefix_Xor_Basis{
4     int rank=0,last=0;
5     vector<ull> a=vector<ull>(64);
6     vector<int> pos=vector<int>(64);
7     void set(){rank=last=0;fill(a.begin(),a.end(),0);fill(pos.begin(),pos.end(),0);}
8     int insert(ull x,int id){
9         assert(id>last);last=id;
10        for(int i=63;i>=0;i--)if(x>>i&1){
11            if(!a[i]){a[i]=x;pos[i]=id;rank++;return 0;}
12            if(pos[i]<id){swap(a[i],x);swap(pos[i],id);}
13            x^=a[i];
14        }
15        return id;
16    }
17    int get_rank(int l=1)const{
18        assert(l>=1);int ans=0;
19        for(int i=0;i<64;i++)ans+=pos[i]>=l;
20        return ans;
21    }
22    bool contains(ull x,int l=1)const{
23        assert(l>=1);
24        for(int i=63;i>=0;i--)if((x>>i&1)&&pos[i]>=l)x^=a[i];
25        return x==0;
26    }
27    ull ask(ull x=0,int l=1)const{
28        assert(l>=1);
29        for(int i=63;i>=0;i--)if(pos[i]>=l)x=max(x,x^a[i]);
30        return x;
31    }
32 };

```

8.4 math/linear basis/linear.cpp

模意义线性基 调用顺序与接口

先抄一份 `ModInt; Linear_Basis<mint> B; B.setN(n)` 设置向量维数。`insert(x)` 返回是否增秩；`contains(x)` 判断 x 是否在张成空间中；`rank` 是当前秩。输入 `vector` 长度必须为 n ，坐标下标 $0..n-1$ 。

最小示例（放入 `main`）：

```

1 Linear_Basis<mint> B; B.setN(2);
2 assert(B.insert({1,0}) && B.insert({0,1}));
3 assert(!B.insert({2,3}) && B.rank==2);
4 assert(B.contains({7,9}));
5 B.setN(2); assert(!B.contains({1,0}));

```

使用边界

T 要求精确域上的运算，通常使用质数模数 `mint`；不支持普通 `int/ll`、合数模数或浮点数。插入和查询 $O(n^2)$ ，空间 $O(n^2)$ ，占用的主元行按实际长度存储。 $n=0$ 允许，空向量不增秩。只维护张成空间，不返回具体线性组合系数。

完整代码

```

1 // Linear_Basis<mint> B; B.setN(n); B.insert(x); B.contains(x);
2 // T must be an exact field type; coordinates are 0..n-1
3 template<typename T> struct Linear_Basis{
4     static_assert(!is_integral_v<T>&&!is_floating_point_v<T>,"use an exact field type");
5     int n=0,rank=0;
6     vector<vector<T>> a;
7     void setN(int _n){assert(_n>=0);n=_n;rank=0;a.assign(n,{});}
8     bool insert(vector<T> x){
9         assert(x.size()==(size_t)n);
10        for(int i=n-1;i>=0;i--)if(x[i]){
11            if(a[i].empty()){
12                T v=T(1)/x[i];for(int j=0;j<i;j++)x[j]*=v;
13                x[i]=T(1);x.resize(i+1);a[i]=move(x);rank++;return true;
14            }
15            T v=x[i];for(int j=0;j<i;j++)x[j]-=v*a[i][j];
16            x[i]=T(0);
17        }
18        return false;
19    }
20    bool contains(vector<T> x)const{
21        assert(x.size()==(size_t)n);
22        for(int i=n-1;i>=0;i--)if(x[i]){
23            if(a[i].empty())return false;
24            T v=x[i];for(int j=0;j<i;j++)x[j]-=v*a[i][j];
25            x[i]=T(0);
26        }
27        return true;
28    }
29 };

```

8.5 math/linear basis/linear-prefix.cpp

前缀模意义线性基 调用顺序与接口

先抄 ModInt。Prefix_Linear_Basis<mint> B; B.setN(n)，依次 insert(x,id)。id 必须为严格递增的正整数；增秩返回 0，否则返回最早可替换元素的编号。规则与前缀异或版相同，只是异或消元换成域上的线性消元。

最小示例（放入 main）：

```

1 Prefix_Linear_Basis<mint> B; B.setN(2);
2 assert(B.insert({1,0},1)==0);
3 assert(B.insert({0,1},2)==0);
4 assert(B.insert({0,2},3)==2);
5 assert(B.insert({1,2},4)==1);
6 assert(B.get_rank(3)==2 && B.contains({1,0},3));
7 assert(!B.contains({1,0},4));
8 assert(B.insert({0,0},5)==5);

```

使用边界

处理到编号 r 时，get_rank(l) 查询 $[l,r]$ 的秩，contains(x,l) 判断 x 是否在该区间张成空间中。 l 默认 1，超出 r 时为空基。插入和可表示性查询 $O(n^2)$ ，秩查询 $O(n)$ ，空间 $O(n^2)$ 。只用于精确域，通常用质数模数 mint；setN 清空所有状态。pos 保留原编号，a 是消元后的向量。不是任意删除或持久化结构。

完整代码

```

1 // Prefix_Linear_Basis<mint> B; B.setN(n); int removed=B.insert(x,id);
2 // id must increase; 0: rank increased, otherwise removed id
3 template<typename T> struct Prefix_Linear_Basis{
4 static_assert(!is_integral_v<T>&&!is_floating_point_v<T>,"use an exact field type");
5 int n=0,rank=0,last=0;
6 vector<vector<T>> a;
7 vector<int> pos;
8 void setN(int _n){assert(_n>=0);n=_n;rank=last=0;a.assign(n,{});pos.assign(n,0);}
9 int insert(vector<T> x,int id){
10 assert(x.size()==(size_t)n&&id>last);last=id;
11 for(int i=n-1;i>=0;i--)if(x[i]){
12     if(a[i].empty()){
13         T v=T(1)/x[i];for(int j=0;j<i;j++)x[j]*=v;
14         x[i]=T(1);x.resize(i+1);a[i]=move(x);pos[i]=id;rank++;return 0;
15     }
16     if(pos[i]<id){
17         x.resize(i+1);swap(a[i],x);swap(pos[i],id);
18         T v=T(1)/a[i][i];for(int j=0;j<i;j++)a[i][j]*=v;
19         a[i][i]=T(1);
20     }
21     T v=x[i];for(int j=0;j<i;j++)x[j]-=v*a[i][j];
22     x[i]=T(0);
23 }
24 return id;
25 }
26 int get_rank(int l=1)const{
27     assert(l>=1);int ans=0;for(int p:pos)ans+=p>=l;return ans;
28 }
29 bool contains(vector<T> x,int l=1)const{
30     assert(x.size()==(size_t)n&&l>=1);
31     for(int i=n-1;i>=0;i--)if(x[i]){
32         if(pos[i]<l)return false;
33         T v=x[i];for(int j=0;j<i;j++)x[j]-=v*a[i][j];
34         x[i]=T(0);
35     }
36     return true;
37 }
38 };

```

9 计算几何

9.1 geometry/geometry-main.cpp

整数二维几何基础

什么时候用

点坐标和叉积均为 `ll`，提供凸包、面积与基本相交判断；需要另定义 `ld=long double`。

调用顺序与接口

- `Point(x,y)`: 点向量加减; `a*b` 是叉积, `a^b` 是点积。
- `convex(points)` 求凸包, `convec_area(hull)` 返回两倍面积。
- `on_segment`、`strict_intersect`、`nonstrict_intersect` 分别判断在线段上、严格相交、含端点相交。

粘贴本文件之前:

```
1 using ld=long double;
```

最小示例 (放入 main):

```
1 vector<Point> a{{0,0},{2,0},{2,2},{0,2}};
2 auto h=convex(a); assert(convec_area(h)==8);
3 assert(on_segment(Point(1,0),Point(0,0),Point(2,0)));
```

使用边界

`convex` 自动排序去重。`min_cross` 要求顶点按凸包环序排列且没有共线中间点，支持旋转起点、顺逆时针， $O(\log n)$ 。射线包含端点，零方向按单点处理；退化三角形按三条边处理。坐标叉积、点积仍须保证不溢出 `ll`。`Point` 输出两个坐标，中间有空格；`output` 用于调试。

完整代码

```
1 //
2 // template for 2D geometry
3 // version 1.0 (Last Update Jun 16th, 2026)
4 //
5 // usage:
6 //   Point A,B; convex(hull); convec_area(hull);
7 //   needs bits/stdc++.h, typedef ll/ld
8 //
9 const ld pi=acos(-1),dx=1e-9;
10 struct Point{
11     ll x,y;
12     friend istream& operator >>(istream &i,Point &P){i>>P.x>>P.y;return i;}
13     friend ostream& operator <<(ostream &o,const Point &P){o<<P.x<<' '<<P.y;return o;}
14     // void read(){cin>>x>>y;}
15     // void output(){cout<<x<<' '<<y<<'\n';}
16     void output(string s=""){cerr<<"Point "<<s<<": ("<<x<<","<<y<<")"<<endl;}
17     double arg(){return atan2(y,x);}
18     Point (ll _x=0,ll _y=0){x=_x,y=_y;}
19 };
20
21 inline bool operator ==(const Point &x,const Point &y){return x.x==y.x&&x.y==y.y;}
22 inline bool operator !=(const Point &x,const Point &y){return x.x!=y.x||x.y!=y.y;}
23
24 inline bool operator <(const Point &x,const Point &y){return (x.x!=y.x?x.x<y.x:x.y<y.y);}
25 inline Point operator -(const Point &x,const Point &y){return Point(x.x-y.x,x.y-y.y);}
```

```

26 inline Point operator +(const Point &x,const Point &y){return Point(x.x+y.x,x.y+y.y);}
27 inline void operator --(Point &x,const Point &y){x=x-y;}
28 inline void operator +=(Point &x,const Point &y){x=x+y;}
29 inline Point operator *(const Point &x,ll y){return Point(x.x*y,x.y*y);}
30 inline void operator *=(Point &x,ll y){x=x*y;}
31 inline Point operator *(ll y,const Point &x){return Point(x.x*y,x.y*y);}
32 inline void operator *=(ll y,Point &x){x=x*y;}
33
34 inline ll operator *(const Point &x,const Point &y){
35     // return OX * OY * sin(XOY) = 2 * area(OXY)
36     return x.x*y.y-x.y*y.x;
37 }
38 inline ll operator ~(const Point &x,const Point &y){
39     // return OX * OY * cos(XOY)
40     return x.x*y.x+x.y*y.y;
41 }
42
43 int sgn(ll x){return (x==0?0:(x>0?1:-1));}
44
45 inline int sgn(const Point &p){
46     return p.y<0?-1:p.y>0||p.x<0?1:0;
47 }
48 inline bool cmp(const Point &x,const Point &y){
49     if(sgn(x)!=sgn(y)) return sgn(x)<sgn(y);
50     return x*y>0;
51 }
52 inline ll area(const Point &x,const Point &y,const Point &z){
53     // return triangle XYZ area **times 2**
54     return abs((x-y)*(x-z));
55 }
56
57 inline ld dist(const Point &x){return sqrt((ld)x.x*x.x+(ld)x.y*x.y);}
58 inline ll dist2(const Point &x){return x.x*x.x+x.y*x.y;}
59
60 inline int dir(const Point &x,const Point &y,const Point &z){
61     // return is YX clockwise direction to YZ
62     return sgn((x-y)*(y-z));
63 }
64 inline int suf(int x,int n){return (x+1==n?0:x+1);}
65 inline int pre(int x,int n){return (x==0?n-1:x-1);}
66 ll convex_area(const vector<Point> &a){
67     // return convex A area **times 2**
68     ll s=0;int n=a.size();
69     for(int i=0;i<n;i++) s+=area(a[0],a[i],a[suf(i,n)]);
70     return s;
71 }
72 vector<Point> convex(vector<Point> a){
73     sort(a.begin(),a.end());a.erase(unique(a.begin(),a.end()),a.end());
74     int n=a.size();
75     if(n<=2)return a;
76     vector<Point> b;int s=0,t=0;
77     for(int i=2;i--;s=t,reverse(a.begin(),a.end())){
78         for(Point p:a){
79             while(t>=s+2&&dir(b[t-2],b[t-1],p)>=0) t--,b.pop_back();
80             b.push_back(p);t++;

```

```

81     }
82     b.pop_back();t--;
83 }
84 return b;
85 }
86 inline bool on_segment(const Point &X,const Point &A,const Point &B){
87     // check if X on segment AB
88     if((A-B)*(X-B)!=0) return false;
89     return min(A.x,B.x)<=X.x&&X.x<=max(A.x,B.x)&&
90         min(A.y,B.y)<=X.y&&X.y<=max(A.y,B.y);
91 }
92 inline bool strict_intersect(const Point &A,const Point &B,
93     const Point &C,const Point &D){
94     // check if AB,CD (except endpoint)strictly intersect
95     return dir(A,C,D)*dir(B,C,D)==-1&&dir(C,A,B)*dir(D,A,B)==-1;
96 }
97 inline bool nonstrict_intersect(const Point &A,const Point &B,
98     const Point &C,const Point &D){
99     // check if AB,CD (include endpoints) intersect
100    return (dir(A,C,D)*dir(B,C,D)==-1&&dir(C,A,B)*dir(D,A,B)==-1)||
101        on_segment(A,C,D)||on_segment(B,C,D)||on_segment(C,A,B)||on_segment(D,A,B);
102 }
103 inline bool parallel(Point A,Point B,
104     Point C,Point D){
105     // check if AB // CD
106     B-=A,D-=C;A-=C;C=-1*D;
107     return sgn(B*C)==0&&sgn(A*B)!=0;
108 }
109 inline bool ray_intersect(Point A,Point B,Point C,Point D){
110     // closed rays; a zero direction is a single point
111     Point u=B-A,v=D-C,w=C-A;
112     if(u==Point())return v==Point()?A==C:((A-C)*v==0&&((A-C)^v)>=0);
113     if(v==Point())return w*u==0&&(w^u)>=0;
114     ll d=u*v;
115     if(!d)return w*u==0&&((u^v)>0||(w^u)>=0);
116     return sgn(w*v)*sgn(d)>=0&&sgn(w*u)*sgn(d)>=0;
117 }
118 inline bool in_triangle(const Point &P,const Point &A,
119     const Point &B,const Point &C){
120     // check if P in triangle ABC
121     // P may lie on bound
122     if(!area(A,B,C))return on_segment(P,A,B)||on_segment(P,B,C)||on_segment(P,C,A);
123     return area(A,B,C)==area(P,A,B)+area(P,B,C)+area(P,C,A);
124 }
125 template<typename F> int convex_min(int n,F val){
126     assert(n>0);
127     auto low=[&](int i){return val(i)<=val((i+n-1)%n)&&val(i)<=val((i+1)%n);};
128     if(low(0))return 0;
129     int l=0,r=n;
130     while(l+1<r){
131         int mid=(l+r)>>1;
132         if(low(mid))return mid;
133         bool a=val((l+1)%n)>=val(l),b=val((mid+1)%n)>=val(mid);
134         if(a!=b){if(a)l=mid;else r=mid;}
135         else if((val(mid)>val(l))==a)l=mid;

```

```
136     else r=mid;
137 }
138 return r%n;
139 }
140 inline ll min_cross(const vector<Point> &I,Point X){
141     // convex polygon in cyclic order, without collinear intermediate points
142     auto val=[&](int i){return I[i]*X;};
143     return val(convex_min(I.size(),val));
144 }
145 inline ld min_cross(const vector<Point> &I,ld x,ld y){
146     auto val=[&](int i){return I[i].x*x+I[i].y*y;};
147     return val(convex_min(I.size(),val));
148 }
```